

Dispense di Applicazioni Informatiche del Machine Learning

Marco Galletti

A.A. 2025/2026

Sommario

Questa dispensa contiene un riassunto dei principali concetti trattati nel corso di Informatica per il Machine Learning con il professore Fabrizio Silvestri del corso di laurea di "Scienze matematiche per l'intelligenza artificiale" presso l'Università Roma "La Sapienza".

Indice

1	Supervised Learning	2
1.1	Funzioni di Loss	4
1.2	Testing	7
2	Alberi di decisione	7
3	Regressione lineare	9
3.1	Regressione univariata	9
3.2	Regressione lineare multivariata	11
4	Logistic Regression	13
5	Valutazione del modello	15
5.1	Metriche di ranking	16
6	Ensemble Learning	18
6.1	Bagging & Random Forest	19
6.2	Random Forest	20
6.3	Boosting	21
6.4	Gradient Boosting	22
6.5	Learning to Rank (LeToR)	22
7	Sistemi di raccomandazione	24
8	Neural Networks	25
8.1	Shallow Neural Networks	25
8.2	Deep Neural Networks	27
9	Allenare reti neurali	30
9.1	Gradient Descent	30
9.2	Stochastic Gradient Descent	31
9.3	Backpropagation	31
10	Sentiment Analysis	35

11 Modeling a language	36
11.0.1 Language Models in IR	37
11.1 Neural Language Models	37
11.1.1 Codifica delle parole	38
11.1.2 Approccio con MLP	38
11.2 Recurrent Neural Networks LMs	38
11.2.1 Funzionamento	38
11.2.2 Efficienza dei parametri	39
11.2.3 Unrolling e Training	40
11.3 Word Embeddings	40
11.3.1 Word2Vec	40
11.3.2 Proprietà degli Embeddings e Valutazione	42
12 Recurrent Neural Networks	43
12.1 Unfolding (Srotolamento)	43
12.2 Addestramento: Backpropagation Through Time	44
12.3 Vanishing ed Exploding Gradient	44
12.4 Long Short-Term Memory (LSTM)	44
12.5 Gated Recurrent Unit (GRU)	46
12.6 Sequence to Sequence (Seq2Seq)	48
12.7 Modelli Autoregressivi	49
12.8 ELMo	49
13 Attention Mechanism	50
13.1 Encoder Bidirezionale	50
13.2 Context Vector Dinamico	50
13.3 Calcolo degli Alignment Scores	50
13.4 Decoder con Attention	51
13.5 Self-Attention	51
13.6 Transformer	52
13.7 Multi-Head Attention	52
13.8 Transformer Architecture	52
13.9 BERT	54
14 CNN - Convolutional Neural Networks	56
14.1 Introduzione e Motivazioni	56
14.2 Operazione di Convoluzione	56
14.3 Iperparametri della Convoluzione	57
14.4 1D ConvNets	58
14.5 Input 2D e Layer Conv2D	59
14.6 Applicazioni delle ConvNets	61
14.7 Architetture per Semantic Segmentation	67

1 Supervised Learning

L'apprendimento supervisionato vuole trovare un “meccanismo” che consenta di ottenere un output voluto a partire da un determinato input. Alla base vi sono i dati di addestramento (**training data**) che il modello usa per trovare il legame tra input e output.

Per mappa intendiamo una funzione $f : X \rightarrow Y$ che associa un oggetto $\mathbf{x} \in X$ un opportuno output $\mathbf{y} \in Y$. Le principali categorie di modelli sono:

- **Modelli di regressione:** $Y \in \mathbb{R}$

- **Modelli di classificazione:** $Y \in A \subset \mathbb{N}$

Definizione 1.1 (Inferenza). L'inferenza è il processo di predizione dell'output di un modello a partire da un input non presente nei dati di addestramento. Si differenzia dunque da quest'ultimo per il fatto che l'output non viene mai utilizzato per l'addestramento.

Per la ricerca del modello f si assume che esso sia codificato da un insieme di parametri θ

$$f(\mathbf{x}; \theta) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \theta^T \mathbf{x}$$

Per misurare la “qualità” dei parametri utilizziamo la **loss function** $\mathcal{L}$. In questo corso useremo principalmente due loss function:

- **Mean Squared Error:** $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - f(\mathbf{x}_i; \theta))^2$

La MSE è tipicamente usata per i modelli di regressione. A volte si usa anche il valore assoluto della differenza (MAE). Notiamo anche che la MSE è **differenziabile**.

- **Cross Entropy:** $\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N y_i \log(f(\mathbf{x}_i; \theta))$

La Binary Cross Entropy (BCE) è tipicamente usata per i modelli di classificazione.

$$\mathcal{L}(\theta) = \sum_{i=1}^N -(1 - y_i) \log(1 - \text{sigmoid}(f(\mathbf{x}_i; \theta))) - y_i \log(\text{sigmoid}(f(\mathbf{x}_i; \theta)))$$

Questa è detta **Binary Cross Entropy with logits**.

La funzione sigmoide è definita come:

$$\text{sig}(x) = \frac{1}{1 + e^{-x}}$$

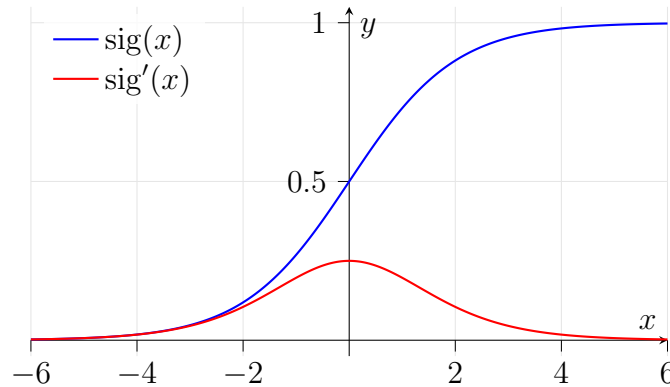


Figura 1: Funzione sigmoide e sua derivata.

La funzione di loss $\mathcal{L}(\theta)$ è dipendente dai parametri θ del modello f . Considerando il dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ siamo interessati a minimizzare:

$$\frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}_i; \theta), y_i)$$

1.1 Funzioni di Loss

La definizione di una loss deve partire da un dataset di training $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$. La funzione ritorna un numero che descrive la “distanza” tra le predizioni e le etichette reali. Durante il processo di apprendimento si toccano i parametri θ al fine di minimizzarla e quindi fare in modo che le predizioni siano sempre più vicine alle etichette reali.

$$\tilde{\theta} = \arg \min_{\theta} \mathcal{L}(\theta)$$

Dal punto di vista statistico la loss function è una funzione di verosimiglianza, ovvero:

$$\mathbf{y} = f(\mathbf{x}; \theta) \Rightarrow \mathbb{P}\{\mathbf{y}|\mathbf{x}; \theta\}$$

Il fatto di avere una distribuzione di probabilità (funzione di verosimiglianza) ci permette di scrivere la probabilità del dataset rispetto al prodotto:

$$\hat{\theta} = \arg \max_{\theta} \left[\prod_{i=1}^N \mathbb{P}\{\mathbf{y}_i | f(\mathbf{x}_i; \theta)\} \right]$$

e massimizzare questa quantità è equivalente a minimizzare la loss function. Questa è detta **Maximum Likelihood**. Possiamo applicare anche il logaritmo naturale per ottenere una somma invece di un prodotto (è monotonicamente crescente). Si parla di **Log-Likelihood**.

$$\hat{\theta} = \arg \max_{\theta} \left[\sum_{i=1}^N \log \mathbb{P}\{\mathbf{y}_i | f(\mathbf{x}_i; \theta)\} \right]$$

Quindi possiamo passare alla **Negative Log-Likelihood**, in modo da passare a un problema di minimizzazione.

$$\hat{\theta} = \arg \min_{\theta} \left[- \sum_{i=1}^N \log \mathbb{P}\{\mathbf{y}_i | f(\mathbf{x}_i; \theta)\} \right] = \arg \min_{\theta} [\mathcal{L}(\theta)]$$

Si suppone inoltre che le etichette siano i.i.d. (indipendenti e identicamente distribuite). Per distribuzione intendiamo la legge che descrive i numeri.

Al tempo di **inferenza** la rete determina la distribuzione di probabilità su $\mathbf{y}$, quindi si determina il massimo della distribuzione

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} \mathbb{P}\{\mathbf{y} | f(\mathbf{x}; \hat{\theta})\}$$

Ricetta di una loss

1. Si sceglie una distribuzione $\mathbb{P}\{\mathbf{y}|\theta\}$ definita sul dominio delle predizioni $\mathbf{y}$ sui parametri θ .
2. Costruzione del modello $f(\mathbf{x}; \theta)$ per predire uno o più parametri θ .
3. Si allena la rete cercando $\arg \min_{\theta} \mathcal{L}(\theta)$ che minimizza, ad esempio, la negative log-likelihood.
4. Si valuta il modello in fase di inferenza.

Esempio 1.1 (Gaussiana $\Rightarrow$ MSE). Assumiamo $\mathbb{P}\{y_i | \mathbf{x}_i; \theta\} \sim \mathcal{N}(f(\mathbf{x}_i; \theta), \sigma^2)$ con varianza costante. La verosimiglianza del dataset i.i.d. è

$$\mathcal{L}(\theta) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f(\mathbf{x}_i; \theta))^2}{2\sigma^2}\right).$$

Facendo il log (prodotto $\rightarrow$ somma) otteniamo

$$\log \mathcal{L}(\theta) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - f(\mathbf{x}_i; \theta))^2.$$

Quindi la negative log-likelihood è

$$-\log \mathcal{L}(\theta) = \underbrace{\frac{N}{2} \log(2\pi\sigma^2)}_{\text{costante}} + \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - f(\mathbf{x}_i; \theta))^2,$$

e, trascurando il termine additivo costante e il fattore moltiplicativo $1/(2\sigma^2)$, minimizzare la NLL equivale a minimizzare la MSE. Questo vale solo sotto varianza costante e indipendente da θ .

Quindi in generale, quando facciamo MSE, stiamo assumendo che il modello stia seguendo una distribuzione gaussiana.

Possiamo anche considerare la **varianza** come un parametro del modello, per cui la funzione da minimizzare è

$$\hat{\theta} = \arg \min_{\theta, \sigma^2} \left[- \sum_{i=1}^N \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(y_i - f(\mathbf{x}_i; \theta))^2}{2\sigma^2} \right) \right] \right]$$

A tempo di inferenza il modello predice la media $\mu = f(\mathbf{x}; \hat{\theta})$, mentre la varianza è stata ottenuta durante l'addestramento. Quando l'incertezza del modello varia come una funzione dei dati di input si parla di **eteroschedasticità**.

Esempio 1.2 (Classificazione Binaria $\Rightarrow$ BCE). Assumiamo $y_i \in \{0, 1\}$ e un modello Bernoulli con parametro $\lambda_i = \lambda(\mathbf{x}_i; \theta)$. Con link logistico poniamo

$$\lambda_i = \text{sig}(f(\mathbf{x}_i; \theta)) = \frac{1}{1 + e^{-f(\mathbf{x}_i; \theta)}}.$$

La probabilità condizionata è

$$\mathbb{P}\{y_i \mid \lambda_i\} = \begin{cases} 1 - \lambda_i & \text{se } y_i = 0, \\ \lambda_i & \text{se } y_i = 1, \end{cases}$$

cioè, in forma compatta, $\mathbb{P}\{y_i \mid \lambda_i\} = \lambda_i^{y_i} (1 - \lambda_i)^{1-y_i}$. Per dati i.i.d. la verosimiglianza è

$$\mathcal{L}(\theta) = \prod_{i=1}^N \lambda_i^{y_i} (1 - \lambda_i)^{1-y_i}.$$

Prendendo il log otteniamo

$$\log \mathcal{L}(\theta) = \sum_{i=1}^N [y_i \log \lambda_i + (1 - y_i) \log(1 - \lambda_i)],$$

perciò la negative log-likelihood (da minimizzare) è

$$-\log \mathcal{L}(\theta) = - \sum_{i=1}^N [y_i \log \lambda_i + (1 - y_i) \log(1 - \lambda_i)],$$

che è la **Binary Cross Entropy**. Con il link logistico $\lambda_i = \text{sig}(f(\mathbf{x}_i; \theta))$ si ottiene la BCE standard (o “with logits” usando direttamente f e la sua forma equivalente).

Esempio 1.3 (Classificazione multiclasse). Vogliamo predire $y \in \{1, 2, \dots, K\}$. Scegliamo quindi la **distribuzione categorica**, che assegna una probabilità a ciascuna classe tramite K parametri $\lambda_1, \dots, \lambda_K$ (in realtà $K - 1$ dato che la somma deve essere 1). Questi parametri devono essere tra 0 e 1 e sommare a 1.

Usiamo una rete $\mathbf{f}[\mathbf{x}, \boldsymbol{\theta}] \in \mathbb{R}^K$ che restituisce K numeri a partire da $\mathbf{x}$. In generale questi numeri non rispettano i vincoli di probabilità, quindi li passiamo attraverso la **softmax**, che li mappa in $[0, 1]$ e li fa sommare a 1. L'uscita k -esima è

$$\text{softmax}_k(\mathbf{f}[\mathbf{x}, \boldsymbol{\theta}]) = \frac{\exp(f_k[\mathbf{x}, \boldsymbol{\theta}])}{\sum_{k'=1}^K \exp(f_{k'}[\mathbf{x}, \boldsymbol{\theta}])}.$$

La verosimiglianza (probabilità del dato) è quindi

$$\mathbb{P}\{y = k \mid \mathbf{x}\} = \text{softmax}_k(\mathbf{f}[\mathbf{x}, \boldsymbol{\theta}]).$$

La negative log-likelihood (da minimizzare) sul dataset è

$$\mathcal{L}(\boldsymbol{\theta}) = - \sum_{i=1}^N \log(\text{softmax}_{y_i}(\mathbf{f}[\mathbf{x}_i, \boldsymbol{\theta}])) = - \sum_{i=1}^N (f_{y_i}[\mathbf{x}_i, \boldsymbol{\theta}] - \log \sum_{k=1}^K e^{f_k[\mathbf{x}_i, \boldsymbol{\theta}]}).$$

In inferenza, la classe predetta è $\hat{y} = \arg \max_k \text{softmax}_k(\mathbf{f}[\mathbf{x}, \boldsymbol{\theta}])$.

Multiple Output Alle volte si vuole predire più di un output per uno stesso modello, quindi l'output è un vettore $\mathbf{y} \in \mathbb{R}^K$. Solitamente si preferisce trattare ciascuna predizione in maniera indipendente.

$$\mathbb{P}\{\mathbf{y}|\boldsymbol{\theta}\} = \prod_d \mathbb{P}\{y_d|\mathbf{x}_d[\mathbf{x}_d; \boldsymbol{\theta}]\}$$

Kullback-Leibler Divergence La **Kullback-Leibler Divergence** è una misura di “distanza” tra due distribuzioni di probabilità p e q :

$$\begin{aligned} D_{KL}(q||p) &= \int_{-\infty}^{+\infty} q(z) \log \frac{q(z)}{p(z)} dz \\ &= \int_{-\infty}^{+\infty} q(z) \log q(z) dz - \int_{-\infty}^{+\infty} q(z) \log p(z) dz. \end{aligned}$$

Si vuole dunque minimizzare la divergenza di Kullback-Leibler tra la distribuzione empirica $q(y)$ e la vera distribuzione del modello $\mathbb{P}\{y|\boldsymbol{\theta}\}$:

$$\begin{aligned} \hat{\boldsymbol{\theta}} &= \arg \min_{\boldsymbol{\theta}} \left[\int q(y) \log q(y) dy - \int q(y) \log \mathbb{P}\{y|\boldsymbol{\theta}\} dy \right] \\ &= \arg \min_{\boldsymbol{\theta}} \left[- \int q(y) \log \mathbb{P}\{y|\boldsymbol{\theta}\} dy \right]. \end{aligned}$$

Poiché la distribuzione empirica può essere scritta come una somma di **delta di Dirac** nei punti osservati $\{y_i\}_{i=1}^N$:

$$q(y) = \frac{1}{N} \sum_{i=1}^N \delta(y - y_i),$$

possiamo sostituirla nell'integrale:

$$\begin{aligned} - \int q(y) \log \mathbb{P}\{y|\boldsymbol{\theta}\} dy &= - \frac{1}{N} \sum_{i=1}^N \int \delta(y - y_i) \log \mathbb{P}\{y|\boldsymbol{\theta}\} dy \\ &= - \frac{1}{N} \sum_{i=1}^N \log \mathbb{P}\{y_i|\boldsymbol{\theta}\}. \end{aligned}$$

Tralasciando il fattore costante $1/N$, otteniamo la **negative log-likelihood (NLL)**:

$$\hat{\theta} = \arg \min_{\theta} \left[- \sum_{i=1}^N \log \mathbb{P}\{y_i|\theta\} \right].$$

Infine, considerando che il modello predice una distribuzione parametrizzata da $f(\mathbf{x}_i; \theta)$, possiamo scrivere:

$$\hat{\theta} = \arg \min_{\theta} \left[- \sum_{i=1}^N \log \mathbb{P}\{y_i|f(\mathbf{x}_i; \theta)\} \right].$$

Pertanto, minimizzare la **cross-entropy** equivale a massimizzare la verosimiglianza del modello. È per questo che la *cross-entropy loss* è comunemente usata nei problemi di classificazione.

1.2 Testing

Quando il modello è addestrato siamo interessati a conoscere le sue performance su dati reali e che non sono stati utilizzati per l'addestramento.

- **Underfitting**: il modello è troppo semplice e non riesce a catturare la complessità dei dati.
- **Overfitting**: il modello è troppo complesso e riesce a catturare i dati di training ma non riesce a generalizzare efficacemente.

2 Alberi di decisione

Questo tipo di modello non è governato da parametri ma da una struttura ad albero. Sono lo stato dell'arte per i task che hanno dati tabellati.

Un albero di decisione è una mappa $f : \mathbb{R}^n \rightarrow A$ dove A è una foglia dell'albero e ogni step intermedio è una decisione. Noi guarderemo principalmente a quelli di classificazione binaria.

Esempio 2.1 (Albero decisionale per la scelta di un ristorante). L'albero seguente rappresenta il processo decisionale per decidere se andare in un ristorante.

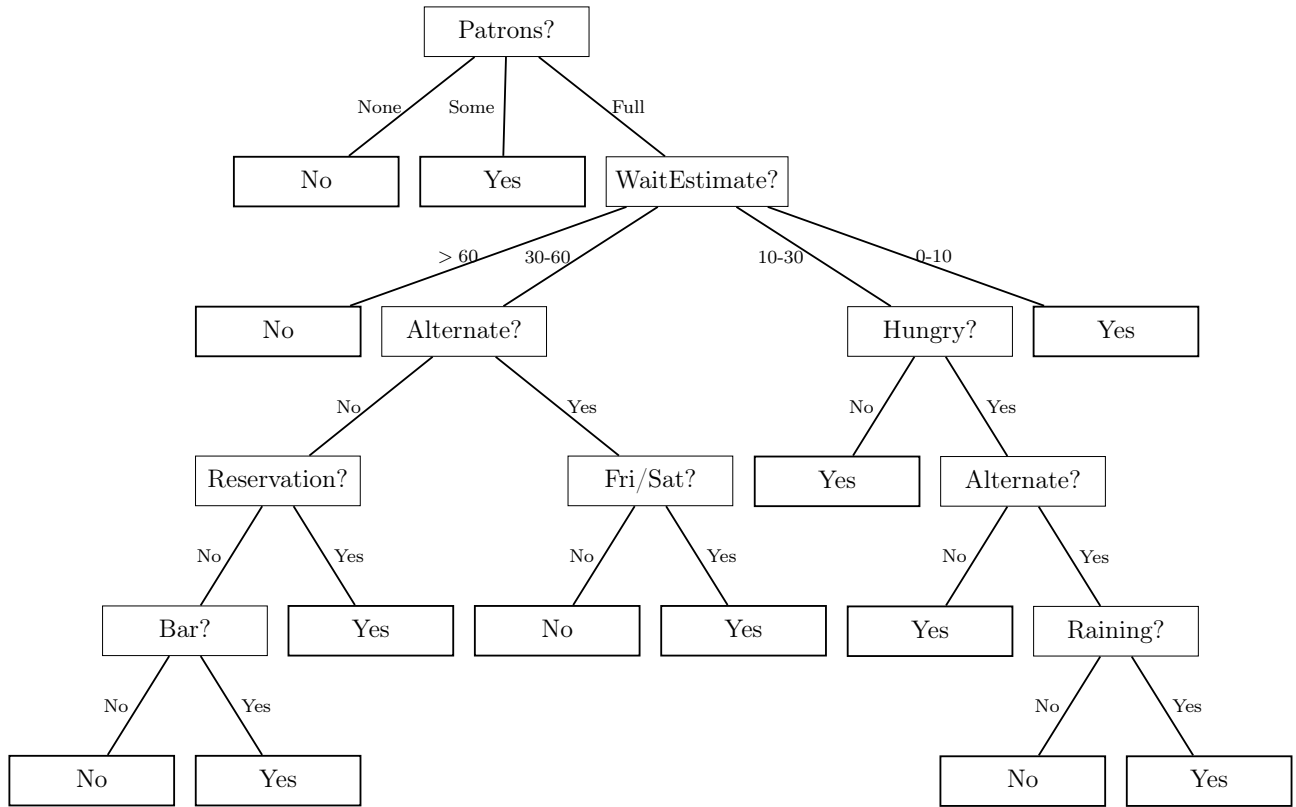


Figura 2: Albero decisionale per decidere se andare in un ristorante in base a vari criteri.

Un albero di decisione può essere descritto da una serie di regole logiche. Dato un dataset possono esserci più alberi che descrivono il processo decisionale. A seconda dell'ordine delle feature discriminatorie si possono ottenere alberi più o meno efficienti. Non esiste un albero ottimo per tutti i dataset.

Algorithm 1 LEARN-DECISION-TREE

Require: *examples*, *attributes*, *parent_examples*

Ensure: a tree

```

1: if examples is empty then
2:   return PLURALITY-VALUE(parent_examples)
3: else if all examples have the same classification then
4:   return the classification
5: else if attributes is empty then
6:   return PLURALITY-VALUE(examples)
7: else
8:    $A \leftarrow \arg \max_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$ 
9:   tree  $\leftarrow$  a new decision tree with root test A
10:  for each value v of A do
11:    exs  $\leftarrow \{e : e \in \text{examples} \text{ and } e.A = v\}$ 
12:    subtree  $\leftarrow$  LEARN-DECISION-TREE(exs, attributes - A, examples)
13:    add a branch to tree with label (A = v) and subtree subtree
14:  end for
15:  return tree
16: end if
  
```

L'importanza di un'attributo può essere definita in termini di **entropia**, che misura la quantità di incertezza di una variabile aleatoria. L'entropia di una variabile aleatoria V con valori v_k con

probabilità p_k è definita come

$$H(V) = \sum_k p_k \log \frac{1}{p_k} = - \sum_k p_k \log p_k$$

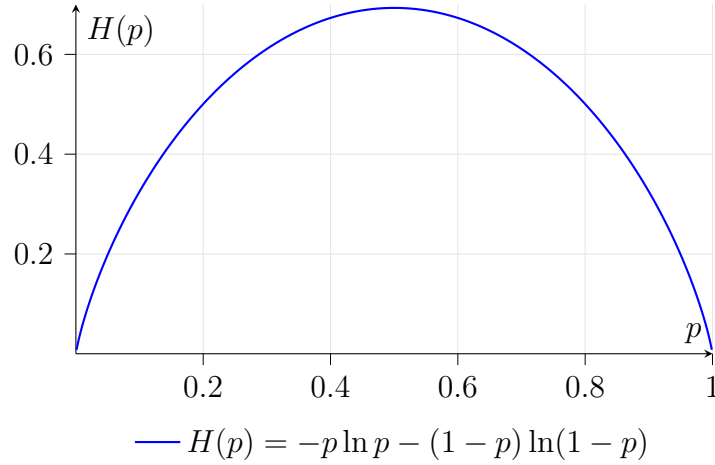


Figura 3: Funzione entropia binaria.

Sia $B(q)$ l'entropia di una variabile aleatoria con probabilità q :

$$B(q) = -q \ln q - (1 - q) \ln(1 - q)$$

Se un dataset contiene p esempi positivi e n esempi negativi allora l'entropia dell'output è

$$H(\text{output}) = B\left(\frac{p}{p+n}\right)$$

Un attributo A con d valori distinti divide il dataset di training E in d sottoinsiemi, per cui ciascun E_k ha p_k esempi positivi e n_k esempi negativi. Possiamo dunque calcolare l'entropia rimanente dopo la divisione su un attributo:

$$\text{Remainder}(A) = \sum_{k=1}^d \frac{|p_k + n_k|}{p+n} B\left(\frac{p_k}{p_k + n_k}\right)$$

E quindi possiamo calcolare il **gain** di un attributo, che ci serve per determinare l'IMPORTANCE di quest'ultimo:

$$\text{Gain}(A) = B\left(\frac{p}{p+n}\right) - \text{Remainder}(A)$$

Osservazione 2.1. *Un albero decisionale può overfittare se è troppo profondo rispetto al dataset.*

3 Regressione lineare

3.1 Regressione univariata

Una funzione lineare univariata (linea) con input x e output y si presenta nella forma:

$$y = h(x) = w_1 x + w_0$$

Dove w_0 e w_1 sono i coefficienti che il modello apprende. Questo tipo di modelli ha il vantaggio non banale di essere interpretabile. Nel caso multivariato possiamo definire $\mathbf{w} = \langle w_0, w_1 \rangle$ e dunque:

$$h(x) = \mathbf{w} \cdot \langle x, 1 \rangle$$

Generalmente utilizziamo la MSE sommando per ogni punto la differenza tra il valore predetto e il valore reale al quadrato:

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^N (y_i - h_{\mathbf{w}}(x_i))^2 = \sum_{i=1}^N (y_i - (w_1 x_i + w_0))^2$$

Riusciamo a minimizzare la loss utilizzando il gradiente e ponendolo uguale a 0:

$$\frac{\partial}{\partial w_0} \mathcal{L}(\mathbf{w}) = 0 \quad \frac{\partial}{\partial w_1} \mathcal{L}(\mathbf{w}) = 0$$

La cui soluzione in forma chiusa è:

$$w_1 = \frac{N \sum x_i y_i - \sum x_i \sum y_i}{N \sum x_i^2 - (\sum x_i)^2} \quad w_0 = \frac{\sum y_i - w_1 \sum x_i}{N}$$

ed è **univocamente** determinata (la loss è convessa, e questo è vero per ogni problema di regressione lineare con una loss L2). La regressione lineare, oltre che per task di prediction, viene utilizzata anche per classificazione, dove l'obiettivo è dividere lo spazio in regioni.

Queste funzioni vengono definite funzioni di **threshold** dove

$$h_{\mathbf{w}}(\mathbf{x}) = \text{threshold}(\mathbf{w} \cdot \mathbf{x}) = \begin{cases} 1 & \text{se } \mathbf{w} \cdot \mathbf{x} > 0 \\ 0 & \text{se } \mathbf{w} \cdot \mathbf{x} \leq 0 \end{cases}$$

tuttavia questa funzione ha il limite di non essere differenziabile, quindi per minimizzarla possiamo considerare un dato $(\mathbf{x}, y)$ e calcolare:

$$w_i \leftarrow w_i + \alpha(y_i - h_{\mathbf{w}}(\mathbf{x}))x_i$$

questa è la regola di update del percettrone

Nel caso della classificazione binaria tramite il percettrone, sia il valore vero y sia l'output dell'ipotesi $h_{\mathbf{w}}(\mathbf{x})$ possono assumere solo i valori 0 o 1. Possiamo quindi considerare tre casi possibili su ogni esempio nel dataset:

- **Se l'output è corretto** ($y = h_{\mathbf{w}}(\mathbf{x})$), i pesi *non vengono modificati*, poiché la previsione è già giusta.
- **Se $y = 1$ ma $h_{\mathbf{w}}(\mathbf{x}) = 0$** , allora il peso w_i viene *aumentato* quando l'input corrispondente x_i è positivo e *diminuito* quando x_i è negativo. Questo perché desideriamo aumentare il valore di $\mathbf{w} \cdot \mathbf{x}$, così che la prossima volta $h_{\mathbf{w}}(\mathbf{x})$ restituisca 1.
- **Se $y = 0$ ma $h_{\mathbf{w}}(\mathbf{x}) = 1$** , allora il peso w_i viene *diminuito* quando x_i è positivo e *aumentato* quando x_i è negativo. Questo ha senso perché vogliamo ridurre il valore di $\mathbf{w} \cdot \mathbf{x}$ in modo che $h_{\mathbf{w}}(\mathbf{x})$ restituisca 0.

Queste regole sono la base dell'algoritmo di aggiornamento dei pesi del percettrone, che permette al modello di correggersi iterativamente sugli errori commessi.

Discesa del gradiente Di seguito presentiamo l'algoritmo di discesa del gradiente

```

1:  $\mathbf{w} \leftarrow$  any point in the parameter space
2: while not converged do
3:   for each  $w_i$  in  $\mathbf{w}$  do
4:      $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \mathcal{L}(\mathbf{w})$ 
5:   end for
6: end while
7: return  $\mathbf{w}$ 

```

Questo algoritmo permette di minimizzare la loss function $\mathcal{L}(\mathbf{w})$ iterativamente fino a convergenza. Il valore di α è detto **learning rate**, e può essere costante o variare ad ogni iterazione.

Regola della catena Ricordiamo la regola della catena:

$$\frac{\partial g(f(x))}{\partial x} = g'(f(x)) \cdot \frac{\partial f(x)}{\partial x}$$

Applicando la regola alla loss della regressione lineare, otteniamo:

$$\frac{\partial \mathcal{L}}{\partial w_0} = -2(y - h_{\mathbf{w}}(\mathbf{x})) \quad \frac{\partial \mathcal{L}}{\partial w_1} = -2(y - h_{\mathbf{w}}(\mathbf{x})) \cdot x$$

Nella pratica preferiamo usare il **batch gradient descent** ovvero calcolare la loss su tutto il batch di training. Sapendo che il gradiente di una somma è la somma dei gradienti, possiamo calcolare il gradiente della loss come:

$$w_0 \leftarrow w_0 + \alpha \sum_{i=1}^N (y_i - h_{\mathbf{w}}(\mathbf{x}_i)) \quad w_1 \leftarrow w_1 + \alpha \sum_{i=1}^N (y_i - h_{\mathbf{w}}(\mathbf{x}_i)) \cdot x_i$$

Si parla di **epoca** quando il modello ha visto tutto il dataset di training.

3.2 Regressione lineare multivariata

Ci riferiamo all'insieme di funzioni del tipo:

$$h_{\mathbf{w}}(\mathbf{x}_j) = w_0 + w_1 x_{j1} + w_2 x_{j2} + \dots + w_n x_{jn} = w_0 + \sum_{i=1}^n w_i x_{ji}$$

Possiamo esprimere questa funzione anche come:

$$h_{\mathbf{w}}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^T \mathbf{x}_j = \sum_{i=0}^n w_i x_{ji}$$

e possiamo ottenere il miglior vettore dei pesi $\mathbf{w}^*$ minimizzando la loss function:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \sum_j L_2(y_j, \mathbf{w} \cdot \mathbf{x}_j)$$

Se chiamiamo $\mathbf{y}$ il vettore degli output reali e $\mathbf{X}$ la matrice degli input, scriviamo il vettore delle predizioni come:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

per cui ricaviamo la squared error loss:

$$\mathcal{L}(\mathbf{w}) = \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

da cui ricaviamo il gradiente:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = 0$$

che ha soluzione:

$$\mathbf{w}^* = \underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{\text{pseudo-inverse di Moore-Penrose}} \mathbf{y}$$

Overfitting nella regressione multivariata Mentre per la regressione lineare univariata non sussistono problemi di overfitting, per la regressione multivariata questo è possibile. Questo è più frequente quando il modello è più complesso rispetto al task da svolgere (e al dataset), ed è dunque possibile che il modello apprenda troppo il rumore del dataset.

Per ovviare a questo problema si usa la **regolarizzazione**. Questa consiste nell'aggiungere al costo totale del modello una penalità legata alla sua complessità, in modo da evitare che apprenda troppi dettagli che non hanno un reale significato:

$$\text{Cost}(h) = \text{EmpLoss}(h) + \lambda \text{Complexity}(h)$$

dove il primo termine misura l'errore sui dati di addestramento, mentre il secondo penalizza modelli troppo complessi (e.g., con pesi troppo grandi).

Nel caso di un modello lineare, la complessità può essere espressa come una funzione dei pesi:

$$\text{Complexity}(h_{\mathbf{w}}) = \mathcal{L}_q(\mathbf{w}) = \sum_{i=1}^n |w_i|^q$$

Per cui a seconda del valore di q si ottengono forme diverse di regolarizzazione.

Esempio 3.1 (L_1 vs L_2). **Idea di base.** La regolarizzazione controlla la complessità del modello aggiungendo alla perdita empirica una penalizzazione sui pesi:

$$\min_{\mathbf{w}} \underbrace{\text{MSE}(\mathbf{y}, \hat{\mathbf{y}})}_{\text{fit}} + \lambda \underbrace{\mathcal{L}_q(\mathbf{w})}_{\text{complessità}}, \quad L_2(\mathbf{w}) = \sum_i w_i^2, \quad L_1(\mathbf{w}) = \sum_i |w_i|.$$

Forma vincolata equivalente: $\text{MSE}(\mathbf{y}, \hat{\mathbf{y}})$ s.t. $\|\mathbf{w}\|_q \leq t$ (per opportuno $t = t(\lambda)$).

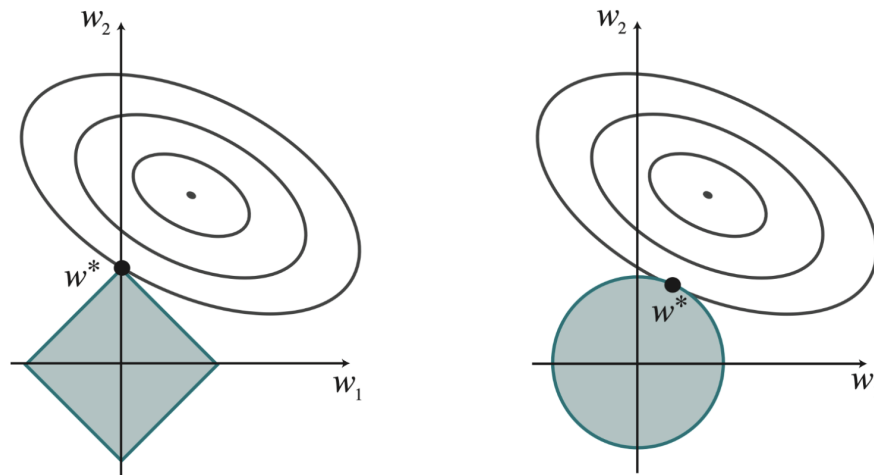


Figura 4: Le curve di livello della loss sono ellissi nello spazio dei pesi; il vincolo L_2 delimita un cerchio, quello L_1 un rombo. Il punto ottimo è dove una curva di livello *tangente* la frontiera del vincolo. Nel caso L_1 la frontiera ha *spigoli* sugli assi: il contatto cade spesso su coordinate nulle $\Rightarrow$ sparsità. Nel caso L_2 la frontiera è liscia: i pesi vengono ridotti ma tipicamente non annullati esattamente.

Ridge (L_2).

- Metodo utile per stimare i coefficienti in presenza di predittori *fortemente correlati*.
- Obiettivo: $L(\mathbf{y}, \hat{\mathbf{y}}) = \text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) + \lambda L_2(\mathbf{w})$, con $L_2(\mathbf{w}) = \sum_i w_i^2$.
- Esiste una forma chiusa per l'ottimo:

$$\hat{\mathbf{w}} = (X^\top X + \lambda I_p)^{-1} X^\top \mathbf{y}.$$

Lasso (L_1).

- *Least Absolute Shrinkage and Selection Operator*: concettualmente simile a Ridge.
- Obiettivo: $L(\mathbf{y}, \hat{\mathbf{y}}) = \text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) + \lambda L_1(\mathbf{w})$, con $L_1(\mathbf{w}) = \sum_j |w_j|$.
- Per valori elevati di λ , molti coefficienti diventano zero; questo non avviene con Ridge.
- Esiste anche per Lasso una forma chiusa, ma non è “pulita” quanto quella di Ridge.

Elastic Net.

Lasso presenta due limiti:

1. se $p > n$, il Lasso seleziona al massimo n variabili (il numero di coefficienti non nulli è limitato dal numero di campioni)
2. con variabili altamente correlate, il Lasso non fa “grouped selection”: tende a sceglierne una e a scartare le altre.

Elastic Net combina i benefici di L_1 (sparsità/selezione) e L_2 (stabilità con predittori correlati).

$$\min_{\mathbf{w}} \text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) + \lambda_1 L_1(\mathbf{w}) + \lambda_2 L_2(\mathbf{w}), \quad L_1(\mathbf{w}) = \sum_j |w_j|, \quad L_2(\mathbf{w}) = \sum_i w_i^2.$$

È comodo usare la parametrizzazione con un unico $\lambda \geq 0$ e un mixing $\alpha \in [0, 1]$:

$$\min_{\mathbf{w}} \text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) + \lambda \left[\alpha L_1(\mathbf{w}) + (1 - \alpha) L_2(\mathbf{w}) \right], \quad \alpha = \frac{\lambda_1}{\lambda_1 + \lambda_2}, \quad \lambda = \lambda_1 + \lambda_2.$$

- **Casi limite:** $\alpha = 1 \Rightarrow$ Lasso; $\alpha = 0 \Rightarrow$ Ridge.
- **Effetto pratico:** conserva la *sparsità* di L_1 , ma grazie al termine L_2 favorisce la *selezione di gruppi* di variabili correlate e può selezionare anche più di n variabili quando $p > n$.

4 Logistic Regression

La funzione logistica ha questa forma:

$$\text{Logistic}(z) = \frac{1}{1 + e^{-z}}$$

Il grafico è visionabile in Figura 1.

Questa funzione può rintracciare la funzione di threshold per il costo:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Logistic}(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{x}}}$$

L'andamento di una funzione tra 0 e 1 ci suggerisce che possiamo usare questa funzione per modellare la probabilità di un evento. Non esiste una forma chiusa per trovare il valore di $\mathbf{w}$ che minimizza la loss function, ma possiamo usare il metodo della discesa del gradiente.

Osservazione 4.1. Dato che il l'output della funzione può essere diverso da 0 o 1 (in generale lo è sempre) usiamo la $L2$ loss.

Per un singolo $(\mathbf{x}, y)$, la derivazione del gradiente è la stessa della regressione lineare fino al punto in cui viene inserita la forma effettiva di h :

$$\begin{aligned}
\frac{\partial}{\partial w_j} \text{Loss}(\mathbf{w}) &= \frac{\partial}{\partial w_j} (y - h_{\mathbf{w}}(\mathbf{x}))^2 \\
&= 2(y - h_{\mathbf{w}}(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (y - h_{\mathbf{w}}(\mathbf{x})) \\
&= -2(y - h_{\mathbf{w}}(\mathbf{x})) \cdot g'(\mathbf{w} \cdot \mathbf{x}) \cdot \frac{\partial}{\partial w_j} \mathbf{w} \cdot \mathbf{x} \\
&= -2(y - h_{\mathbf{w}}(\mathbf{x})) \cdot g'(\mathbf{w} \cdot \mathbf{x}) \cdot x_j.
\end{aligned}$$

La derivata g' della funzione logistica soddisfa $g'(z) = g(z)(1 - g(z))$, quindi abbiamo:

$$g'(\mathbf{w} \cdot \mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x})(1 - g(\mathbf{w} \cdot \mathbf{x})) = h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x}))$$

Osservazione 4.2. *Chiaramente anche nel caso della logistic regression possiamo usare la regolarizzazione.*

L'aggiornamento dei pesi per minimizzare la loss segue la direzione della differenza tra il valore reale e quello predetto, $(y - h_{\mathbf{w}}(\mathbf{x}))$, e la dimensione del passo dipende dai fattori α e dalla derivata della funzione logistica g' .

$$w_j \leftarrow w_j + \alpha(y - h_{\mathbf{w}}(\mathbf{x})) \cdot h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x})) \cdot x_j$$

Abbiamo osservato che ipotesi tratte da logistic threshold possono essere viste come probabilità, tuttavia si preferisce la **binary cross entropy loss**:

$$-y \log(h_{\mathbf{w}}(\mathbf{x})) - (1 - y) \log(1 - h_{\mathbf{w}}(\mathbf{x}))$$

Possiamo dunque trovare il gradiente della loss function:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = (h_{\mathbf{w}}(\mathbf{x}) - y) \cdot \mathbf{x}$$

e l'update dei pesi segue la direzione del gradiente:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot (h_{\mathbf{w}}(\mathbf{x}) - y) \cdot \mathbf{x}$$

Definizione 4.1 (Odd-ratio). Definiamo il **odds-ratio** come il rapporto tra la probabilità $P(x)$ e la probabilità $\overline{P(x)} = 1 - P(x)$:

$$\text{Odds-ratio} = \frac{P(x)}{1 - P(x)}$$

Possiamo dunque considerare il logaritmo naturale dell'odds-ratio:

$$\log \text{Odds-ratio} = \log \left(\frac{P(x)}{1 - P(x)} \right)$$

e dunque ottenere il log-odds-ratio come un modello lineare:

$$\log \left(\frac{P(x)}{1 - P(x)} \right) = \sum_{j=1}^N w_j x_j$$

Quindi possiamo ottenere la probabilità come:

$$P(\mathbf{x}) = \frac{1}{1 + \exp(-\sum_{j=1}^N w_j x_j)}$$

ovvero

$$P(\mathbf{x}) = \sigma \left(\sum_{j=1}^N w_j x_j \right)$$

5 Valutazione del modello

Esistono differenti metriche per valutare la “bontà” di un modello in base al tipo di task.

- **Regressione:** Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE). Chiaramente queste metriche hanno senso quando si stanno misurando due varianti di un modello che vogliono predire lo stesso output.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

- **Classificazione:** Accuracy, Confusion Matrix, Precision, Recall, F1-Score, ROC-AUC, Precision-Recall Curve.

Accuracy: ci da una misura di quanto possiamo affidarci alla predizione del modello.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{Numero di predizioni corrette}}{\text{Numero di predizioni totali}}$$

Si osservi che per dataset sbilanciati (i.e., dati su malattie rare) l'accuracy non è una buona metrica da usare.

Confusion Matrix: matrice che ha sulle righe le etichette reali e sulle colonne le etichette predette. Per il caso binario, la confusion matrix è:

$$\text{Confusion Matrix} = \begin{pmatrix} TP & FP \\ FN & TN \end{pmatrix}$$

Per il caso multiclasse, la confusion matrix è una matrice $K \times K$ dove K è il numero di classi.

Precision: ci da una misura di quanto possiamo affidarci a una predizione positiva del modello.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: ci da una misura di quanti dei veri positivi sono stati predetti correttamente.

$$\text{Recall} = \frac{TP}{TP + FN}$$

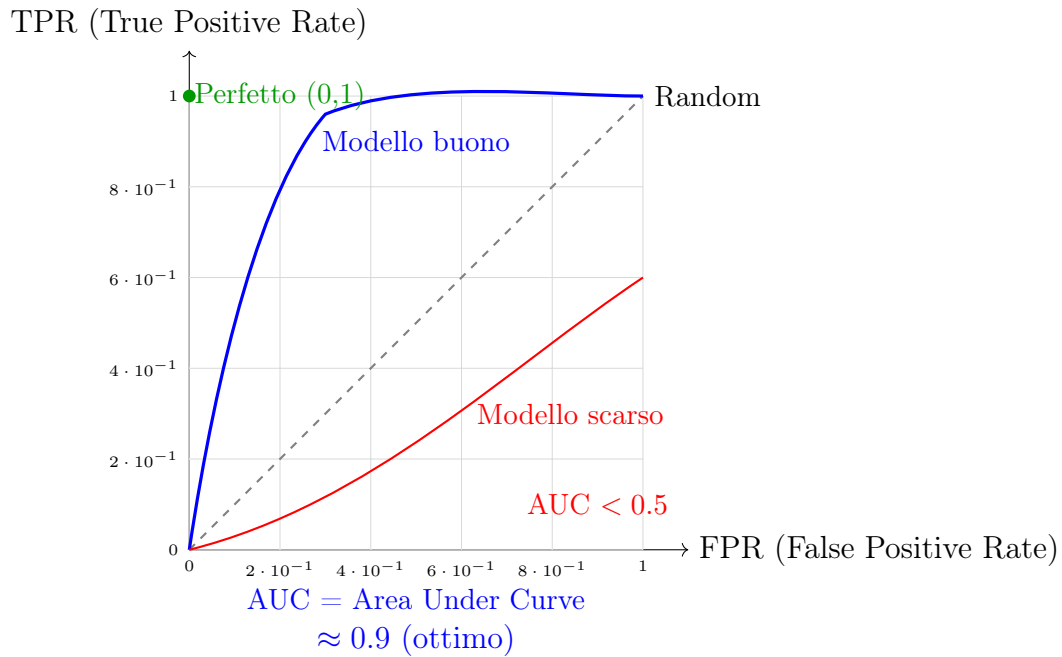
F1-Score: media armonica di precision e recall.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}}$$

Questa metrica vuole bilanciare precision e recall, in quanto non sono del tutto indipendenti.

ROC: Receiver Operating Characteristic Curve permette di visualizzare il trade-off tra falsi positivi e falsi negativi. Ovvero come varia la percentuale di true positive rate (TPR, che è la Recall) al variare della percentuale di false positive rate (FPR) (numero di volte in cui dico positivo quando in realtà è negativo).

$$\text{TPR} = \frac{TP}{TP + FN} \quad \text{FPR} = \frac{FP}{FP + TN}$$



Interpretazione: Un classificatore perfetto ha TPR=1 e FPR=0 (punto in alto a sinistra). La curva ROC mostra le performance al variare della soglia di decisione. L'area sotto la curva (AUC) quantifica la qualità: AUC=1 è perfetto, AUC=0.5 è casuale.

La AUC (Area Under the Curve) è una metrica che quantifica la qualità di un classificatore.

Esiste anche la **Precision-Recall Curve** che si basa sulla curva ROC ma sull'asse delle x abbiamo la recall mentre sull'asse delle y abbiamo la precision. La curva mostra come varia la precisione al variare della soglia di classificazione: un modello migliore mantiene alta precisione anche con alta recall. È più informativa quando i positivi sono più rari.

5.1 Metriche di ranking

Le metriche di classificazione tradizionali non catturano la qualità di un *ordinamento*: due sistemi che restituiscono gli stessi documenti rilevanti ma in ordine diverso hanno lo stesso F1-Score, pur differendo significativamente in utilità pratica. Le **metriche di ranking** misurano invece quanto i documenti rilevanti si trovino *in cima* alla lista, penalizzando quelli relegati nelle posizioni inferiori. Queste metriche trovano applicazione naturale nei motori di ricerca, nei sistemi di raccomandazione e, come vedremo, nell'apprendimento automatico applicato all'ordinamento (cfr. il Learning to Rank).

Precision@ k e Recall@ k Dato un sistema che restituisce una lista ordinata di documenti, si considerano solo i **top- k** risultati:

$$\text{Precision@}k = \frac{|\{\text{rilevanti tra i top-}k\}|}{k}, \quad \text{Recall@}k = \frac{|\{\text{rilevanti tra i top-}k\}|}{|\{\text{rilevanti nel corpus}\}|}.$$

Precision@ k risponde alla domanda “*quanti dei k risultati mostrati sono effettivamente utili?*”, mentre Recall@ k risponde a “*quale frazione di tutti i documenti rilevanti siamo riusciti a recuperare?*”. Le due quantità sono in trade-off: abbassando la soglia di rilevanza (o aumentando k) si tende ad avere recall più alta a scapito della precision.

La scelta di quale privilegiare dipende dal dominio applicativo. In sistemi di recupero documenti critici (es. ricerca medica o legale) è più grave perdere un documento rilevante che mostrarne uno irrilevante, per cui si privilegia la recall. Nei motori di ricerca generali, dove l'utente legge prevalentemente i primi risultati, conta di più la precision.

Esempio 5.1 (Precision@ k e Recall@ k). Una query ha 6 documenti rilevanti nel corpus. Il sistema restituisce i top-5 con rilevanza binaria: $[1, 0, 1, 0, 1]$ (dove 1 indica un documento rilevante). I rilevanti tra i top-5 sono 3, quindi:

$$\text{Precision@5} = \frac{3}{5} = 0.60, \quad \text{Recall@5} = \frac{3}{6} = 0.50.$$

MRR (Mean Reciprocal Rank) Quando all'utente interessa principalmente trovare *almeno un* risultato rilevante nel minor numero di passi possibile (tipicamente in sistemi di question answering o ricerche navigazionali), si usa il **Mean Reciprocal Rank**:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{R_i},$$

dove N è il numero di query e R_i è la posizione (rank) del *primo* documento rilevante per la query i . Se non vi è alcun documento rilevante tra i top- k , si pone $\frac{1}{R_i} = 0$, ovvero il contributo di quella query è nullo.

Esempio 5.2 (MRR). Per una query il sistema restituisce $[1, 0, 1, 0, 1]$: il primo documento rilevante si trova in posizione 1, quindi $\text{RR} = 1/1 = 1$. Se invece il sistema restituisse $[0, 0, 1, 0, 1]$, il primo rilevante sarebbe in posizione 3 e $\text{RR} = 1/3 \approx 0.333$.

Un limite intrinseco dell'MRR è che considera *soltanto* la posizione del primo documento rilevante, ignorando completamente la struttura del resto della lista. Due sistemi in cui il primo rilevante occupa la stessa posizione hanno MRR identico, anche se uno presenta molti altri documenti rilevanti nelle posizioni immediatamente successive e l'altro nessuno.

MAP (Mean Average Precision) La **MAP** (Mean Average Precision) supera il limite dell'MRR tenendo conto di *tutte* le posizioni dei documenti rilevanti. Per una singola query con R documenti rilevanti nel corpus si calcola prima la **Average Precision**:

$$\text{AP@}k = \frac{1}{\min(R, k)} \sum_{i=1}^k \text{Precision@}i \cdot \mathbf{1}[\text{doc}_i \text{ rilevante}],$$

dove il fattore $\mathbf{1}[\cdot]$ vale 1 se il documento in posizione i è rilevante e 0 altrimenti. La MAP è poi la media delle AP su tutte le N query:

$$\text{MAP} = \frac{1}{N} \sum_{j=1}^N \text{AP}_j.$$

Questa metrica premia i documenti rilevanti nelle posizioni alte (alta precision cumulativa in quel punto della lista) e penalizza quelli relegati in fondo, pur mantenendo una visione globale sull'intera lista.

Esempio 5.3 (Average Precision). Top-5 = $[1, 0, 1, 0, 1]$ con $R = 6$ rilevanti nel corpus.

$$\text{AP@5} = \frac{1}{\min(6, 5)} \left[\frac{1}{1} \cdot 1 + \frac{1}{2} \cdot 0 + \frac{2}{3} \cdot 1 + \frac{2}{4} \cdot 0 + \frac{3}{5} \cdot 1 \right] = \frac{1 + 0 + 0.667 + 0 + 0.6}{5} \approx 0.453.$$

Confronto tra le metriche di ranking Ogni metrica cattura un aspetto diverso della qualità dell'ordinamento:

- **Precision@ k / Recall@ k** : metriche semplici che valutano solo la “finestra” dei top- k risultati, senza considerare l'ordine interno.

- **MRR**: adatto quando basta trovare *un solo* buon risultato il prima possibile. Non distingue sistemi che differiscono nei risultati successivi al primo rilevante.
- **MAP**: considera tutti i documenti rilevanti e la loro distribuzione lungo la lista, ma assume rilevanza *binaria* (rilevante o irrilevante).
- **NDCG** (Normalized Discounted Cumulative Gain, trattato nel contesto del Learning to Rank): estende MAP a rilevanza *graduata* (es. scale 0–4) e applica uno sconto logaritmico in base alla posizione. È la metrica più usata nei sistemi di ricerca commerciali proprio perché cattura sia la qualità della rilevanza sia l'importanza della posizione.

Esempio 5.4 (Limite dell'MRR). Sistema A: primo documento rilevante al rank 1, nessun altro rilevante nelle posizioni successive. Sistema B: primo documento rilevante al rank 1, poi altri cinque documenti rilevanti nelle posizioni immediatamente successive. Entrambi hanno $MRR = 1.0$. L'NDCG di B è invece molto più alto, riuscendo a discriminare i due sistemi.

6 Ensemble Learning

L'idea che sottende l'ensemble learning è quella di considerare ipotesi individuali $(h_1, h_2, \dots, h_n)$ e combinarle in un'unica ipotesi $h = f(h_1, h_2, \dots, h_n)$. In questo senso f può essere una media, una combinazione lineare, una somma etc. L'ensemble learning può portare a ridurre bias e varianza.

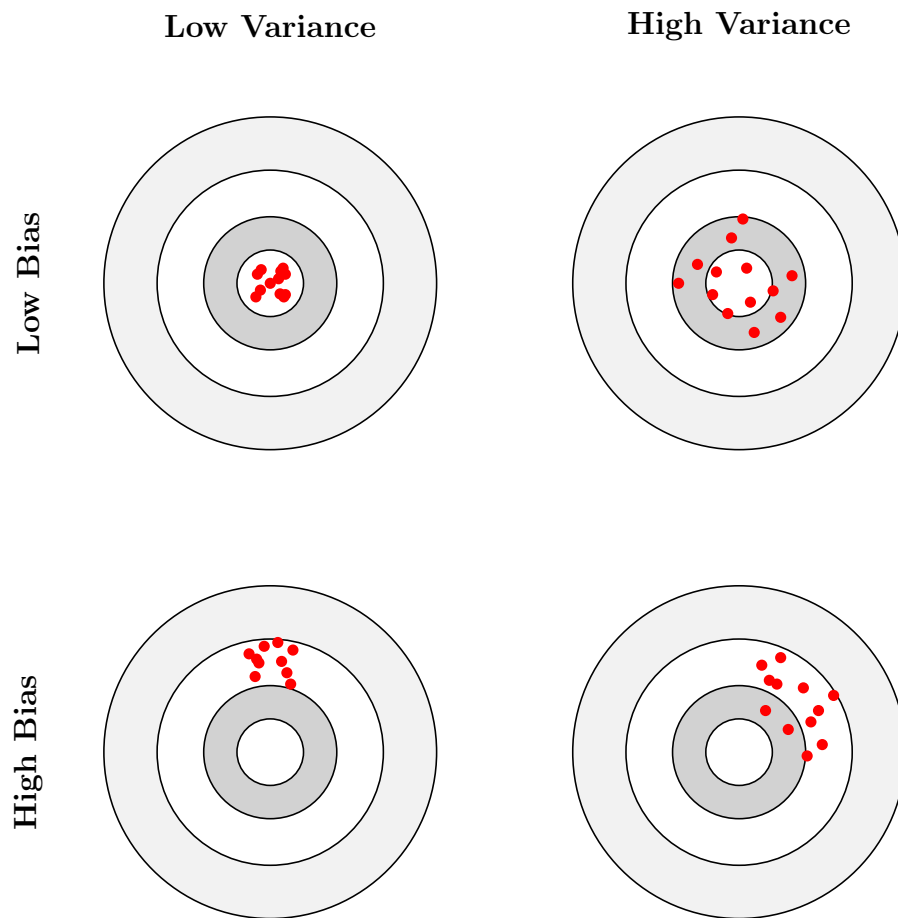


Figura 5: Bias-Variance

Vediamo dunque perché l'ensemble learning riduce la varianza e il bias.

- **Riduzione del bias**: Un singolo modello può avere uno **spazio delle ipotesi** limitato, cioè è in grado di rappresentare solo un insieme ristretto di funzioni. Questo vincolo strutturale

impone un *bias* elevato: il modello, anche se addestrato perfettamente, non può adattarsi bene a relazioni complesse nei dati.

Un esempio tipico è la **regressione logistica**, che cerca un confine decisionale lineare: se i dati non sono linearmente separabili, l'errore sistematico non può essere eliminato (alto bias).

I modelli **ensemble** combinano più classificatori (ad esempio n modelli lineari) ottenendo così uno spazio delle ipotesi più ricco e flessibile. In pratica, un insieme di più confini lineari può approssimare confini non lineari, riducendo il bias complessivo del modello.

Questa maggiore espressività comporta un costo computazionale che cresce circa in modo lineare con il numero di modelli (n volte maggiore), ma risulta comunque molto più efficiente rispetto ad ampliare lo spazio delle ipotesi di un singolo modello — operazione che richiederebbe una crescita *esponenziale* della complessità.

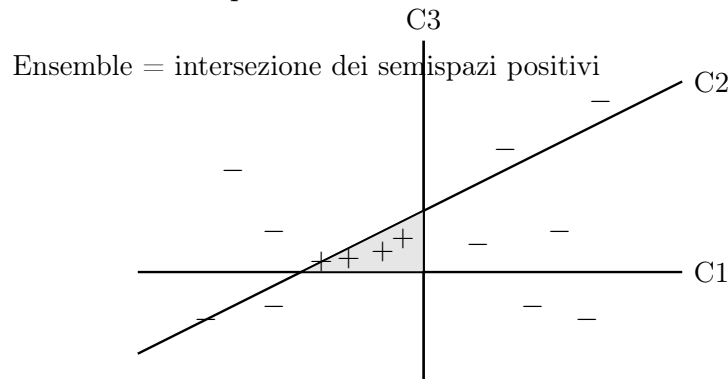


Figura 6: Ensemble (AND) di tre confini lineari. Positivo solo dove valgono $y > 1.6$, $y < 0.5x + 0.4$ e $x < 4.2$ simultaneamente (triangolo grigio). La combinazione di linee produce un confine non lineare (poligonale) più espressivo di una singola retta $\Rightarrow$ bias inferiore.

- **Riduzione della varianza:** consideriamo un ensemble di 5 classificatori indipendenti, ciascuno addestrato su un **sottoinsieme diverso** del dataset di training, e combinati tramite **majority voting**. Affinché l'ensemble commetta un errore su un nuovo campione, è necessario che almeno 3 classificatori su 5 sbaglino simultaneamente. Se ogni classificatore ha un'accuratezza del 75%, la probabilità complessiva che la maggioranza voti correttamente sale fino a circa l'89%. In generale, l'uso di più modelli addestrati su dati diversi riduce le fluttuazioni dovute al training e rende le predizioni più stabili, portando a una **riduzione della varianza** rispetto a un singolo modello.

Se non si può assumere che i classificatori siano indipendenti (nella gran parte dei casi) allora non possiamo considerare la predizione dei singoli come indipendenti. Per ovviare a questo problema possiamo considerare tre modi per creare un ensemble: **Bagging**, **Boosting** e **Stacking**.

6.1 Bagging & Random Forest

Data generation : si costruiscono K training sets distinti campionando dal training set originale con **replacement**, ovvero ogni esempio del training set originale può apparire più volte nel training set di ogni modello. Ogni sottoinsieme viene utilizzato per costruire un classificatore a sè stante, e l'output di ogni classificatore viene aggregato per produrre la predizione finale.

- Per la **classificazione** si usa la **majority voting**
- Per la **regressione** si usa la **media** $h(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^K h_i(\mathbf{x})$

Bagging e alberi decisionali: Il **bagging** (da *bootstrap aggregating*) è una tecnica di ensemble che consiste nel generare K modelli indipendenti, ognuno addestrato su un training set ottenuto campionando dal dataset originale **con replacement** (bootstrap sampling). Ciò significa che ogni esempio del dataset originale può essere selezionato più volte oppure non essere incluso affatto nel training set di un determinato modello.

Durante l'inferenza, le predizioni dei singoli modelli vengono poi combinate (*aggregating*) per ottenere la predizione finale.

Il bagging tende a ridurre la **varianza** ed è particolarmente utile quando i dati sono **limitati** o quando il modello base tende a **overfittare** sul training set. Può essere applicato a qualunque tipo di modello, ma risulta particolarmente efficace con gli **alberi decisionali**.

Questo perché gli alberi sono modelli intrinsecamente *instabili*: piccole variazioni nei dati di training possono produrre alberi strutturalmente molto diversi. Aggregando molti alberi addestrati su campioni bootstrap differenti, il bagging ne riduce la variabilità e aumenta la stabilità complessiva del modello.

Un ulteriore vantaggio è che gli alberi possono essere addestrati in parallelo, rendendo il bagging efficiente anche su architetture multi-core o distribuite.

Tuttavia, i modelli generati tendono a essere **altamente correlati**: se un attributo ha un forte potere discriminante, esso sarà spesso scelto come nodo radice in molti alberi, riducendo la diversità dell'ensemble.

6.2 Random Forest

I modelli di random forest sono una forma di bagging che usa alberi decisionali dove si usano step extra per costruire l'insieme dei K alberi in modo che siano meno correlati tra loro (riduce la varianza). Le random forest possono essere usate per classificazione e regressione.

L'idea è di **scegliere casualmente** gli attributi da usare e ogni volta calcolare quali hanno il più alto **gain** per la classificazione (sono più discriminanti). Se gli attributi sono n si sceglie:

- $m = \sqrt{n}$ attributi per la classificazione
- $m = \frac{n}{3}$ attributi per la regressione

Ad alto livello, l'algoritmo di **Random Forest** funziona nel seguente modo:

1. Si generano N **campioni bootstrap** (bagging samples) di cardinalità n campionando dal training set originale **con replacement**.
2. Per ciascun campione, si addestra un **albero decisionale** considerando, ad ogni nodo, un sottoinsieme casuale di M feature tra quelle disponibili, e scegliendo lo **split ottimale** solo tra queste M .
3. Si aggregano le predizioni dei singoli alberi per ottenere la predizione finale: **media** nel caso di regressione, **voto di maggioranza** nel caso di classificazione.

Vantaggi delle Random Forest Le **Random Forest** presentano numerosi vantaggi che le rendono uno dei modelli ensemble più utilizzati in pratica:

- **Minima preparazione dei dati:** non richiedono scaling o normalizzazione e possono gestire feature numeriche, binarie e categoriali senza pre-elaborazione specifica.
- **Selezione implicita delle feature:** durante la costruzione degli alberi, il modello esegue di fatto una selezione automatica delle variabili più informative, fornendo anche una stima dell'importanza delle feature.
- **Ridotta sensibilità agli iperparametri:** le prestazioni sono stabili anche con impostazioni di default, e la probabilità di ottenere un modello "sbagliato" è bassa.

- **Buone prestazioni e robustezza:** difficilmente un singolo modello (ad esempio una rete neurale o un algoritmo di boosting) supera una Random Forest in modo significativo senza una complessa fase di tuning.
- **Versatilità e semplicità:** si adattano bene sia a problemi di regressione che di classificazione, e l'algoritmo è concettualmente semplice ma efficace.
- **Addestramento parallelo:** gli alberi possono essere addestrati in parallelo, rendendo il modello facilmente scalabile su più unità di calcolo. Inoltre alla loro efficienza contribuisce anche il fatto che ogni split considera meno feature della costruzione di un singolo albero decisionale.
- **Implementazioni mature:** esistono numerose implementazioni efficienti e open source (ad esempio in `scikit-learn`).
- **Overfitting:** la random forest è meno propensa all'overfitting

Extra-Trees: extremely randomized trees Gli **Extra-Trees** rappresentano una variante delle Random Forest che introduce un livello di **randomness** ancora maggiore nella costruzione degli alberi. A differenza delle Random Forest, in cui il miglior punto di split viene scelto tramite ricerca ottimale tra le feature selezionate, negli Extra-Trees:

- per ogni nodo si seleziona un sottoinsieme casuale di feature;
- per ciascuna feature, la soglia di split viene scelta in modo **casuale uniforme** entro il suo intervallo di valori;
- tra queste combinazioni casuali si sceglie quella che massimizza il criterio di separazione (ad esempio l'indice di Gini o l'entropia).

Questa maggiore casualità riduce ulteriormente la **correlazione tra gli alberi**.

In una **Random Forest** sono presenti diversi iperparametri, come il numero di alberi, la profondità massima e il numero massimo di feature considerate a ogni split. È possibile ricercare la combinazione ottimale di questi iperparametri minimizzando l'**out-of-bag (OOB) error**, ovvero l'errore medio commesso sui campioni che non sono stati inclusi nei corrispondenti campioni bootstrap utilizzati per addestrare ciascun albero.

6.3 Boosting

Ogni istanza del training set è pesata in modo differente (si parla di **weighted training set**). Ogni peso w_j indica il contributo della j -esima istanza del training set alla loss function, per cui un errore su un esempio con peso maggiore ha un impatto più elevato sul valore complessivo della loss.

- Il boosting parte ponendo i pesi di tutti gli esempi del training set uguali a 1. Da questo insieme di dati costruisce un primo modello h_1 , che commetterà determinati errori.
- Si aggiornano quindi i pesi in modo **proporzionale agli errori commessi**, assegnando un peso maggiore agli esempi classificati in modo errato, così che il modello successivo h_2 si concentri maggiormente su di essi.
- Il processo viene ripetuto per K iterazioni, ottenendo progressivamente modelli che correggono gli errori dei precedenti.

Il principale **drawback** del boosting è che si tratta di un algoritmo **sequenziale** (non parallelizzabile in modo naturale) e segue un approccio **greedy**, affinando la predizione in modo iterativo.

Per task di classificazione binaria o regressione si ha:

$$h(\mathbf{x}) = \sum_{i=1}^K z_i h_i(\mathbf{x})$$

dove z_i è un peso scalare che rappresenta il contributo della i -esima ipotesi alla predizione finale.

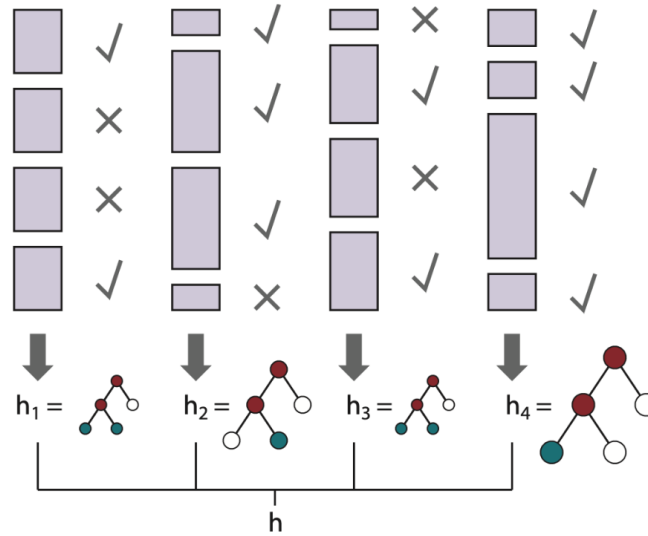


Figura 7: Funzionamento dell'algoritmo di boosting.

6.4 Gradient Boosting

È una forma di boosting che usa la discesa del gradiente. In questo caso si aggiungono nuove ipotesi, dette **boosting hypothesis**, che dipendono dal gradiente della loss function che confronta le risposte esatte con quelle predette dalle ipotesi precedenti. Necessitiamo dunque che la loss function sia **differenziabile** (e.g., mse per la regressione e log loss per la classificazione).

Per problemi di regressione e classificazione di dati tabulati si parla di **gradient boosting machines** (GBM) o **gradient boosting decision trees** (GBDT).

Con il gradient boosting non aggiorniamo i pesi dei modelli esistenti, ma aggiorniamo i parametri degli alberi successivi in modo da ridurre la loss function.

Si addestra quindi un modello F della forma $\hat{y} = F(x)$ ottenuto minimizzando la MSE:

$$\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$\hat{y}_i$ è la predizione del modello F per il sample i -esimo, y_i è il valore “osservato”.

Se consideriamo un algoritmo di gradient boosting con M iterazioni, a ogni stadio m , costruiamo

$$F_{m+1}(\mathbf{x}) = F_m(\mathbf{x}) + h_m(\mathbf{x})$$

per cui abbiamo il residuo dato da $y - F_m(\mathbf{x})$, e quella che andiamo a ottimizzare è la media dei residui

$$L_{\text{MSE}} = \frac{1}{n} (y - F(\mathbf{x}))^2 \implies -\frac{\partial L_{\text{MSE}}}{\partial F} = \frac{2}{n} (y - F(\mathbf{x})) \cdot (-1) = \frac{2}{n} (F(\mathbf{x}) - y) = \frac{2}{n} h_m(\mathbf{x})$$

6.5 Learning to Rank (LeToR)

Obiettivo Data una query q e un insieme di documenti $\{d\}$, si vuole apprendere una funzione che *ordini* i documenti per rilevanza rispetto a q (problema di tipo *ranking/ordinal*, non mera classificazione o regressione).

Segnali di base Un baseline classico è la **cosine similarity** tra vettori TF-IDF (term frequency - inverse document frequency) di query e documento:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\sum_i q_i d_i}{\|\mathbf{q}\| \|\mathbf{d}\|}.$$

In pratica si combinano molte feature (es. segnali di click, posizione dei termini nel documento, tempo/localizzazione, cronologia).

Famiglie di metodi

- **Pointwise:** si apprende un punteggio o livello di rilevanza per ogni coppia (q_i, d_j^i) ; poi si ordina per punteggio.

$$(q_i, d_j^i) \mapsto r_{ij} \in \{0, 1, 2, 3, 4\} \quad (\text{o } r_{ij} \in \mathbb{R}).$$

- **Pairwise:** si apprende una *preferenza* fra due documenti della stessa query, così da indurre un ordinamento.

$$(q_i, d_j, d_k) \mapsto y_{ijk} \in \{-1, +1\}, \quad y_{ijk} = \begin{cases} +1 & \text{se } r_{ij} > r_{ik} \quad (d_j \text{ preferito a } d_k) \\ -1 & \text{se } r_{ij} < r_{ik} \end{cases}$$

- **Listwise:** si ottimizza direttamente l'*ordine* dell'intera lista per una query, modellando relazioni globali tra documenti.

Questa visione “ordinata” è più naturale dell'approccio di pura classificazione per l'IR.

La metrica NDCG La metrica più comune per valutare il ranking è l'NDCG (Normalized Discounted Cumulative Gain). Questa metrica tiene conto della posizione dei documenti nella lista e del loro gain:

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}$$

$$\text{DCG} = \sum_{i=1}^n \frac{2^{r_i} - 1}{\log_2(i + 1)}$$

dove r_i è la rilevanza del documento i -esimo. La quantità IDCG (Ideal DCG) rappresenta il valore massimo della DCG, calcolato su una lista ideale in cui i documenti sono ordinati in base alla loro rilevanza decrescente.

RankNet (pairwise probabilistico) Dato uno score $s(q, d) \in \mathbb{R}$, per una query q e due documenti (d_i, d_j) con rilevanze $r_i > r_j$ vogliamo $s_i > s_j$. RankNet modella la *probabilità di ordine corretto* con una logistica:

$$P_{ij} = \sigma(\sigma_0(s_i - s_j)), \quad \sigma(x) = \frac{1}{1 + e^{-x}},$$

e minimizza una loss di *cross-entropy* sulle coppie. Definendo $\bar{P}_{ij}$ come la probabilità target ($\bar{P}_{ij} = 1$ se $r_i > r_j$), il costo C è:

$$C = -\bar{P}_{ij} \log(P_{ij}) - (1 - \bar{P}_{ij}) \log(1 - P_{ij})$$

Il gradiente rispetto ai punteggi (le cosiddette *lambdas*) è:

$$\lambda_{ij} = \frac{\partial C}{\partial s_i} = \sigma_0(P_{ij} - 1) = -\frac{\sigma_0}{1 + e^{\sigma_0(s_i - s_j)}}, \quad \frac{\partial C}{\partial s_j} = -\lambda_{ij}.$$

Intuizione: λ_{ij} spinge s_i verso l'alto e s_j verso il basso quando $r_i > r_j$, fornendo un surrogato liscio per ridurre le inversioni di ordine.

LambdaRank (allineare i gradienti alla metrica) Le metriche di ranking (es. NDCG) dipendono dalle *posizioni* in lista, mentre RankNet tratta tutte le coppie allo stesso modo. LambdaRank pesa i gradienti in base a quanto cambia la metrica se si scambiano d_i e d_j :

$$\lambda_{ij}^{(\lambda)} = \lambda_{ij} \cdot |\Delta \text{NDCG}_{ij}|,$$

dove $|\Delta \text{NDCG}_{ij}|$ è la variazione (in valore assoluto) di NDCG che si otterrebbe scambiando le posizioni correnti di d_i e d_j . Così gli aggiornamenti sono più grandi per le coppie che incidono maggiormente sulla metrica finale (tipicamente ai primi posti), avvicinando l'ottimizzazione alla NDCG.

7 Sistemi di raccomandazione

Il problema della raccomandazione è concettualmente simile a quello del **ranking**: dato un insieme di utenti e un insieme di oggetti (film, prodotti, articoli, ecc.), si vuole ordinare gli oggetti in base alla loro *rilevanza o utilità* per ciascun utente.

Un modo naturale per rappresentare queste interazioni è tramite una **matrice utente–oggetto**, dove ciascuna cella r_{ui} indica, ad esempio, la valutazione espressa dall'utente u sull'oggetto i (o una misura implicita di interesse, come un click o una visualizzazione).

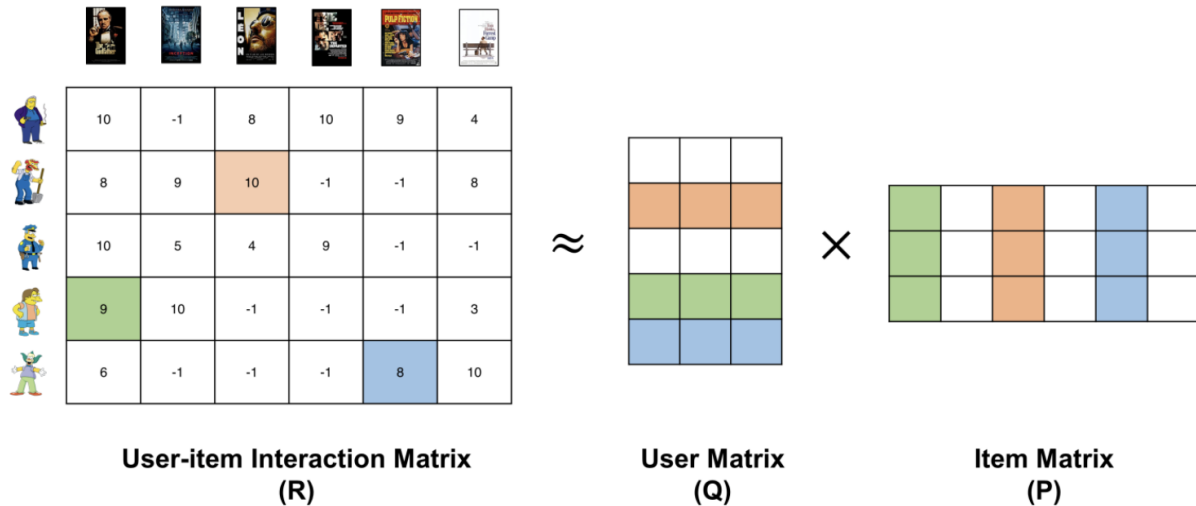


Figura 8: Rappresentazione a matrice delle interazioni utente–oggetto.

Tale matrice R è in genere **molto sparsa**, poiché ogni utente interagisce solo con una piccola parte degli oggetti disponibili. Per ridurre la dimensionalità e stimare i valori mancanti, si utilizza la **fattorizzazione della matrice** in due matrici di rango ridotto:

$$R \approx QP,$$

dove:

- $Q \in \mathbb{R}^{m \times k}$ rappresenta gli **utenti** nello spazio latente di k dimensioni,
- $P \in \mathbb{R}^{k \times n}$ rappresenta gli **oggetti** nello stesso spazio,
- l'obiettivo è minimizzare l'errore di ricostruzione $\|R - QP\|$, in modo che il prodotto QP approssimi le preferenze osservate in R .

La fattorizzazione permette quindi di apprendere **rappresentazioni latenti** (embedding) per utenti e oggetti, utili per predire interazioni non osservate e generare raccomandazioni personalizzate.

8 Neural Networks

8.1 Shallow Neural Networks

Le reti neurali shallow sono funzioni $\mathbf{y} = f(\mathbf{x}; \theta)$ con parametri θ , input multivariato $\mathbf{x}$ e output multivariato $\mathbf{y}$.

Esempio 8.1. Sia $f(\mathbf{x}; \theta)$ una rete neurale che mappa un input scalare $\mathbf{x}$ in un output scalare y e che ha 10 parametri

$$\phi = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$$

ovvero

$$y = f[\mathbf{x}; \phi] = \phi_0 + \phi_1 \underbrace{a[\theta_{10} + \theta_{11}x]}_{h_1} + \phi_2 \underbrace{a[\theta_{20} + \theta_{21}x]}_{h_2} + \phi_3 \underbrace{a[\theta_{30} + \theta_{31}x]}_{h_3}$$

dove a è la funzione di attivazione e i suoi argomenti sono i layer lineari. Chiamiamo h_1, h_2, h_3 **hidden units**. Combinandole otteniamo la funzione:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

Un esempio di funzione di attivazione è la ReLU:

$$a(z) = \begin{cases} z & \text{se } z > 0 \\ 0 & \text{se } z \leq 0 \end{cases}$$

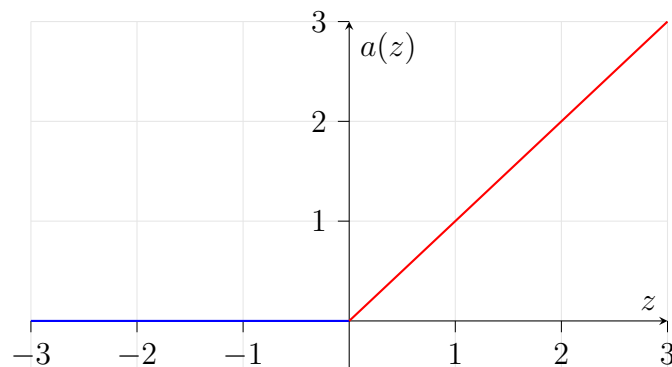


Figura 9: Funzione di attivazione ReLU (Rectified Linear Unit).

Ogni hidden unit contiene una funzione lineare rispetto all'input x e questa funzione viene clippata dalla funzione ReLU se è negativa. Le diverse h_i sono dunque pesate dai parametri ϕ_i e sommate per ottenere l'output y , assieme all'offset ϕ_0 , che controlla l'altezza della funzione.

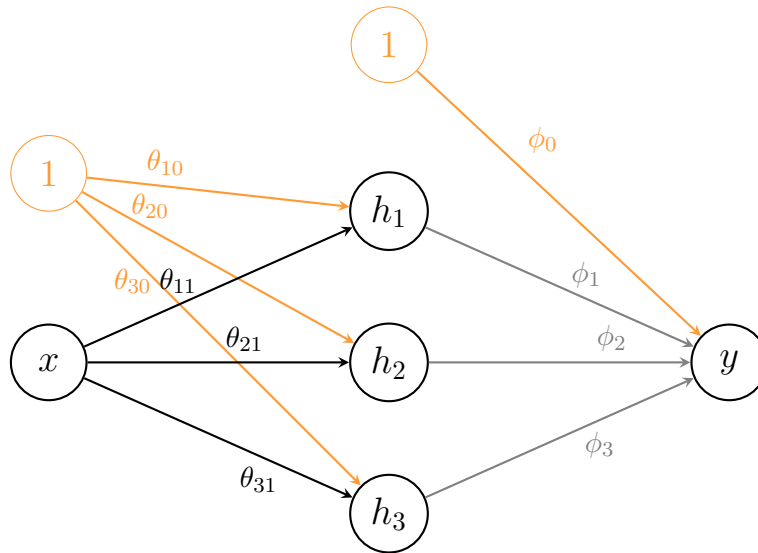


Figura 10: Architettura di una rete neurale shallow con un input scalare x , tre hidden units (h_1, h_2, h_3) e un output scalare y . I nodi arancioni rappresentano i bias (costanti pari a 1).

Teorema 8.1 (Teorema di approssimazione universale). *Con sufficiente capacità (unità nascoste), una rete neurale superficiale può descrivere qualsiasi funzione continua 1D definita su un sottoinsieme compatto della retta reale con precisione arbitraria.*

Formalmente, per ogni funzione continua $f : K \rightarrow \mathbb{R}$ dove $K \subset \mathbb{R}$ è compatto, e per ogni $\varepsilon > 0$, esiste una rete neurale superficiale tale che

$$\sup_{x \in K} |f(x) - y(x)| < \varepsilon$$

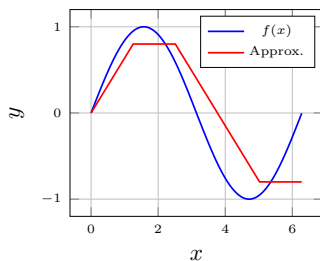
dove $y(x)$ è l'output della rete neurale.

Intuizione con ReLU. Consideriamo una rete shallow con D unità nascoste, dove ogni hidden unit h_d calcola $h_d = \text{ReLU}(\theta_{d0} + \theta_{d1}x)$, e l'output è dato da $y = \phi_0 + \sum_{d=1}^D \phi_d h_d$.

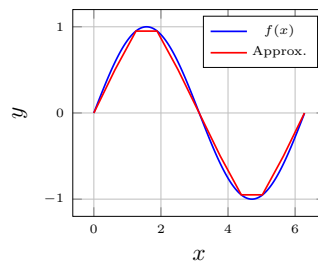
Con funzioni di attivazione ReLU, ogni unità nascosta introduce al massimo un *joint* (punto di non-differenziabilità) nell'output finale. Pertanto, una rete con D hidden units produce una funzione *piecewise linear* con al massimo D joints e $D + 1$ regioni lineari.

Il numero di unità nascoste misura quindi la **capacità** della rete: aggiungendo più hidden units, aumentiamo il numero di regioni lineari disponibili per approssimare la funzione target. All'aumentare di D , queste regioni diventano più numerose e piccole, permettendo di approssimare con precisione sempre maggiore qualsiasi funzione continua.

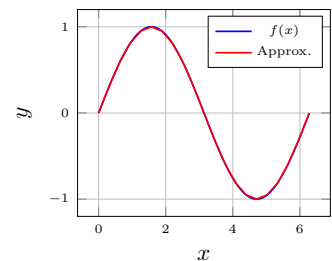
Nota: Sebbene il teorema garantisca l'esistenza di una rete capace di approssimare qualsiasi funzione continua, non fornisce garanzie sulla facilità di trovare i parametri ottimali tramite training.



5 regioni lineari



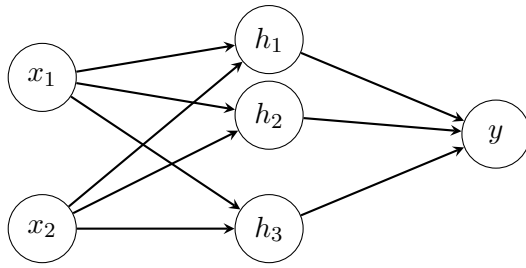
10 regioni lineari



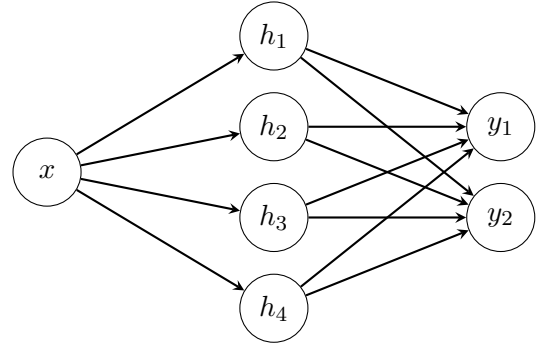
20 regioni lineari

Figura 11: Approssimazione piecewise linear di $f(x) = \sin(x)$ con un numero crescente di regioni lineari. La funzione originale (blu) viene approssimata sempre meglio all'aumentare della capacità della rete (rosso).

Il caso multivariato è analogo, semplicemente presenta più input o output. Ad esempio:



Multi-input: 2 input, 3 hidden units, 1 output



Multi-output: 1 input, 4 hidden units, 2 output

Figura 12: Esempi di reti neurali shallow con configurazioni multivariate. A sinistra, una rete con input multidimensionale; a destra, una rete con output multidimensionale.

La rete di destra si matematizza come:

$$\begin{aligned} y_1 &= \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4 \\ y_2 &= \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4 \end{aligned}$$

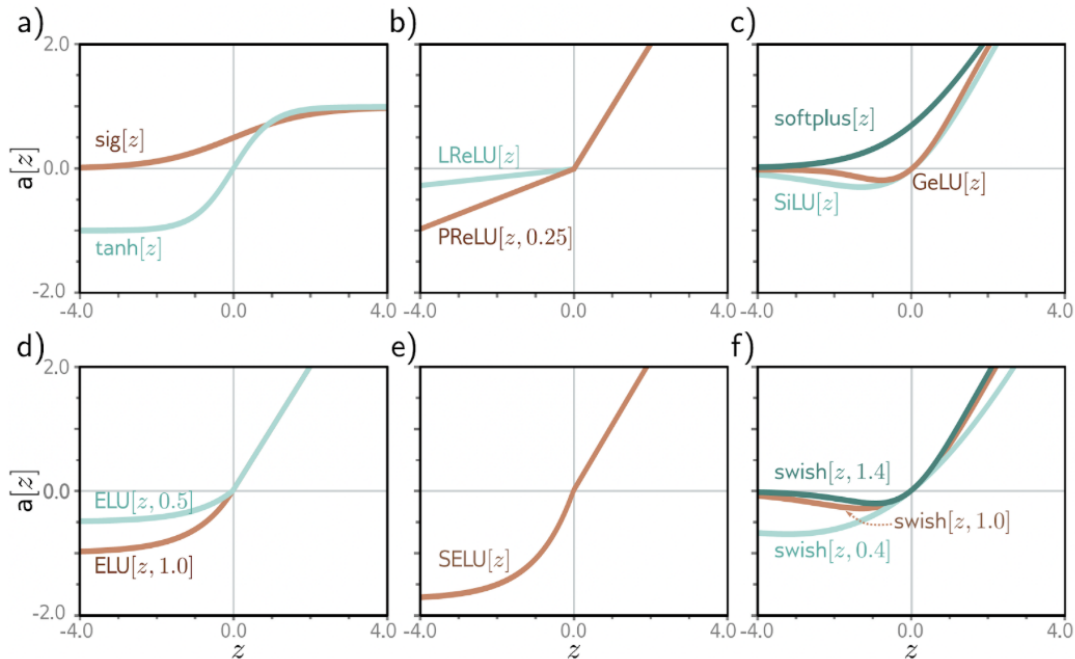
Possiamo **generalizzare** le shallow networks definendo, per una rete $\mathbf{f}[\mathbf{x}, \boldsymbol{\theta}]$

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right]$$

dove $\mathbf{x} \in \mathbb{R}^{D_i}$, $\mathbf{y} \in \mathbb{R}^{D_o}$ e $h \in \mathbb{R}^D$. Dunque si possono combinare linearmente per ottenere l'output:

$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d$$

Presentiamo nell'immagine seguente diverse funzioni di attivazione:



8.2 Deep Neural Networks

Si differenzia dalle shallow neural networks per il fatto che ha più layer di unità nascoste.

Comporre layer è l'equivalente di determinare delle feature, partendo dall'ipotesi che lo spazio dei dati sia più piccolo rispetto allo spazio originale.

Il deep learning vorrebbe che ogni layer si occupi di un sottoinsieme delle feature (e.g., bordi, colori, texture). Si intuisce che ad un maggior numero di layer corrisponda un maggior numero di feature.

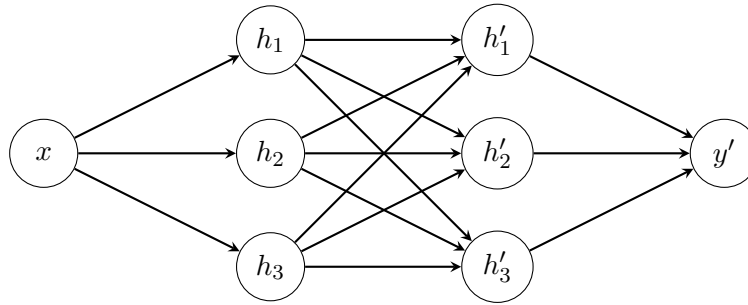


Figura 13: Esempio di Deep Neural Network (MLP) con 1 input, 2 hidden layers (ciascuno con 3 unità) e 1 output. La rete è fully connected tra layer adiacenti.

Le equazioni che descrivono questa rete sono:

First Layer:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

Second Layer:

$$h'_1 = a[\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3]$$

$$h'_2 = a[\psi_{20} + \psi_{21}h_1 + \psi_{22}h_2 + \psi_{23}h_3]$$

$$h'_3 = a[\psi_{30} + \psi_{31}h_1 + \psi_{32}h_2 + \psi_{33}h_3]$$

Output:

$$y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3$$

Notazione vettoriale

Le stesse equazioni possono essere riscritte in forma vettoriale:

First Layer:

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \mathbf{a} \left(\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x \right)$$

Second Layer:

$$\begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix} = \mathbf{a} \left(\begin{bmatrix} \psi_{10} \\ \psi_{20} \\ \psi_{30} \end{bmatrix} + \begin{bmatrix} \psi_{11} & \psi_{12} & \psi_{13} \\ \psi_{21} & \psi_{22} & \psi_{23} \\ \psi_{31} & \psi_{32} & \psi_{33} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} \right)$$

Output:

$$y' = \phi'_0 + [\phi'_1 \quad \phi'_2 \quad \phi'_3] \begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix}$$

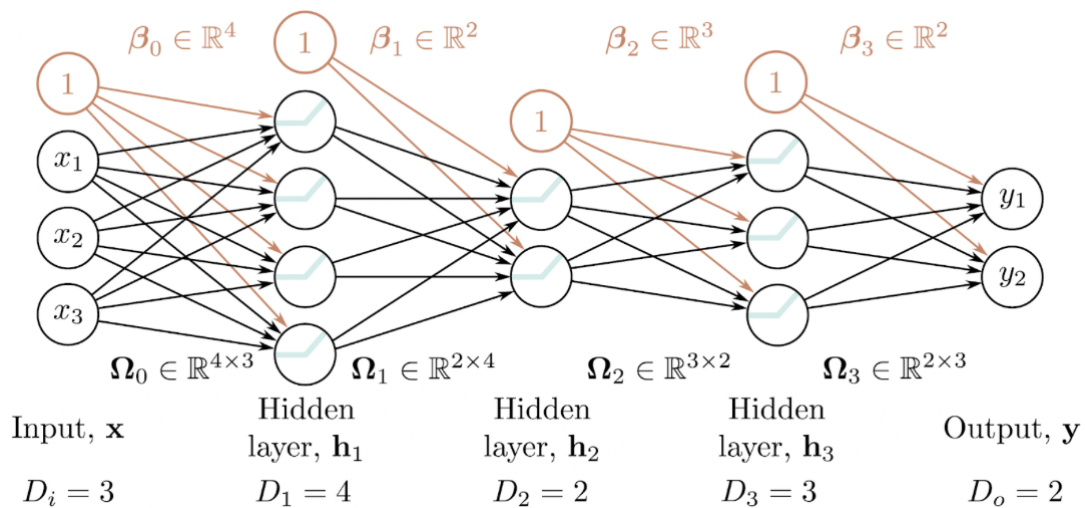
Notazione matriciale

In forma ancora più compatta, usando la notazione matriciale:

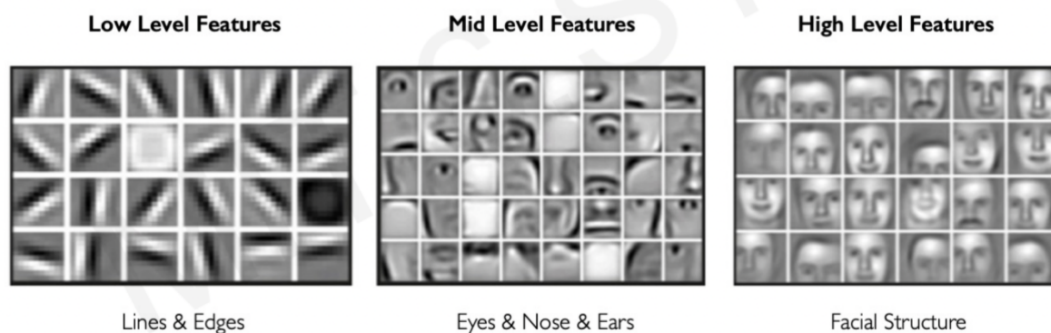
$$\begin{aligned}\mathbf{h} &= \mathbf{a}[\boldsymbol{\theta}_0 + \boldsymbol{\theta}x] \\ \mathbf{h}' &= \mathbf{a}[\boldsymbol{\psi}_0 + \boldsymbol{\Psi}\mathbf{h}] \\ y &= \phi'_0 + \phi'\mathbf{h}'\end{aligned}$$

dove:

- $\mathbf{h}, \mathbf{h}' \in \mathbb{R}^3$ sono i vettori delle hidden units
- $\boldsymbol{\theta}_0, \boldsymbol{\psi}_0 \in \mathbb{R}^3$ sono i vettori dei bias
- $\boldsymbol{\theta} \in \mathbb{R}^3$ è il vettore dei pesi del primo layer
- $\boldsymbol{\Psi} \in \mathbb{R}^{3 \times 3}$ è la matrice dei pesi del secondo layer
- $\phi' \in \mathbb{R}^3$ è il vettore dei pesi dell'output layer
- $\mathbf{a}[\cdot]$ indica l'applicazione element-wise della funzione di attivazione



Il deep learning riesce a battere modelli che si basano su features manuali perché queste sono difficili da mantenere e generalizzare, per cui portano a errori.



Chiaramente la costruzione di reti neurali profonde presenta spesso più di un layer nascosto (a volte persino migliaia). Il numero di hidden units definisce la larghezza (width) della rete, mentre il numero di hidden layer ne identifica la profondità (depth).

Il numero di hidden units ci dà una misura della capacità della rete. In generale indichiamo il numero di layer con K e il numero di hidden units in ogni layer con $D_1, D_2, \dots, D_K$.

Questi sono esempi di **iperparametri** che possono essere ottimizzati per ottenere i migliori outcome.

Per input larghi e strutturati si tende a evitare full connected layers (sequenziali), in quanto il numero di parametri diverrebbe proibitivo. La soluzione è processare differenti regioni dell'input in parallelo e integrare informazione in maniera graduale.

Empiricamente si osserva che a differenza delle shallow networks, le deep networks generalizzano meglio su dati nuovi.

9 Allenare reti neurali

9.1 Gradient Descent

Dati i parametri iniziali $\phi = \{\phi_0, \phi_1, \phi_2, \dots, \phi_n\}$ iteriamo due step:

1. Calcoliamo la derivata della loss function rispetto a tutti i parametri

$$\mathcal{L}[\phi] = \sum_{i=1}^N \ell_i \quad \frac{\partial \mathcal{L}}{\partial \phi} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \phi_0} \\ \frac{\partial \mathcal{L}}{\partial \phi_1} \\ \frac{\partial \mathcal{L}}{\partial \phi_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial \phi_n} \end{bmatrix}$$

2. Aggiorniamo i parametri in base al gradiente

$$\phi \leftarrow \phi - \alpha \cdot \frac{\partial \mathcal{L}}{\partial \phi}$$

dove α è il learning rate.

Nel caso di modelli del tipo $f(x, \phi)$ che mappano un input scalare x su un output scalare y con $\phi = [\phi_0, \phi_1]^T$ abbiamo:

$$y = f[x, \phi] = \phi_0 + \phi_1 x$$

e la loss function:

$$\mathcal{L}[\phi] = \sum_{i=1}^N \ell_i = \sum_{i=1}^N (f[x_i, \phi] - y_i)^2 = \sum_{i=1}^N (\phi_0 + \phi_1 x_i - y_i)^2$$

per cui la derivata della loss rispetto ai parametri può essere scomposta nella somma delle derivate rispetto a ciascun parametro:

$$\frac{\partial \mathcal{L}}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^N \ell_i = \sum_{i=1}^N \frac{\partial \ell_i}{\partial \phi}$$

nel caso specifico abbiamo:

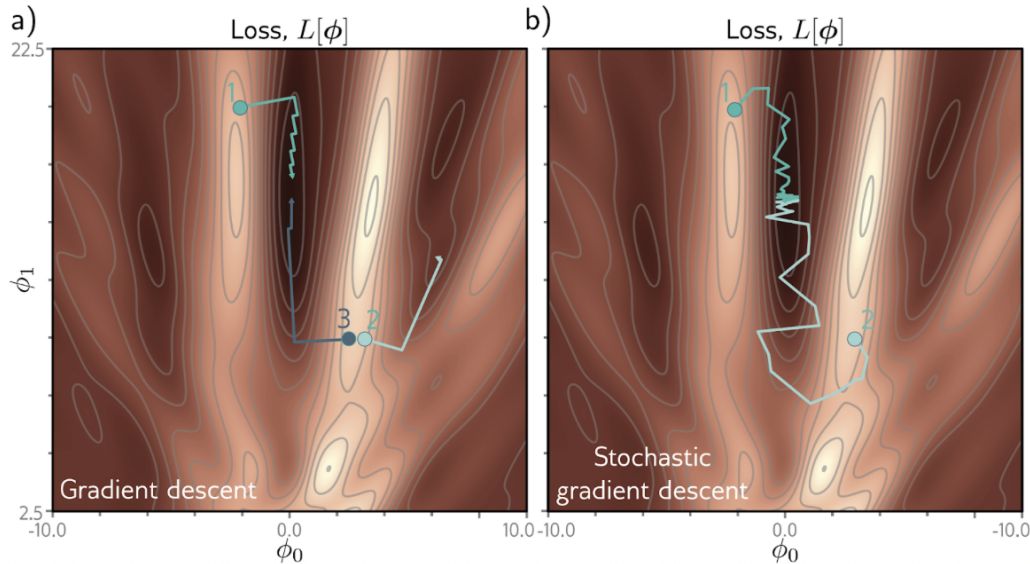
$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2(\phi_0 + \phi_1 x_i - y_i)x_i \end{bmatrix}$$

9.2 Stochastic Gradient Descent

Invece di calcolare il gradiente rispetto a tutti i parametri, calcoliamo il gradiente rispetto a un singolo parametro:

$$\frac{\partial \mathcal{L}}{\partial \phi_i} = \frac{\partial \mathcal{L}}{\partial \phi_i}$$

dove i è un indice casuale tra 1 e N .



Batch & Epochs Possiamo scegliere di usare ad ogni iterazione un sottoinsieme del training set e questo è detto **batch**. In questo caso la regola di aggiornamento dei parametri diventa:

$$\phi_{t+1} = \phi_t - \alpha \cdot \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

Il batch è solitamente scelto senza rimpiazzamento. Si parla di **epoca** quando il modello ha visto tutto il dataset di training.

La scelta del batch size, learning rate schedule, learning algorithm e coefficienti del momentum sono tutti iperparametri che possono essere ottimizzati per ottenere prestazioni migliori.

9.3 Backpropagation

È un algoritmo che ottimizza tutte le famiglie di algoritmi di gradient descent.

Esempio 9.1. Sia $f(\mathbf{x}; \phi)$ una rete con input multivariato $\mathbf{x}$, parametri ϕ e tre layer nascosti h_1, h_2, h_3 .

$$h_1 = a[\beta_0 + \Omega_0 \cdot \mathbf{x}]$$

$$h_2 = a[\beta_1 + \Omega_1 \cdot h_1]$$

$$h_3 = a[\beta_2 + \Omega_2 \cdot h_2]$$

$$f[\mathbf{x}; \phi] = \beta_3 + \Omega_3 \cdot h_3$$

e abbiamo la loss function:

$$\mathcal{L}[\phi] = \sum_{i=1}^N \ell_i$$

assumendo di usare SGD, occorrerebbe calcolare le derivate

$$\frac{\partial \ell_i}{\partial \beta_k} \quad \text{e} \quad \frac{\partial \ell_i}{\partial \Omega_k}$$

e questo viene fatto in maniera efficiente tramite **backpropagation**.

Forward pass Nel forward pass calcoliamo l'output della rete dato un input $\mathbf{x}_i$, propagando i valori in avanti attraverso i layer.

Per l'esempio precedente, il forward pass consiste nel calcolare sequenzialmente:

$$\begin{aligned} f_0 &= \beta_0 + \Omega_0 \cdot \mathbf{x}_i \\ h_1 &= a[f_0] \\ f_1 &= \beta_1 + \Omega_1 \cdot h_1 \\ h_2 &= a[f_1] \\ f_2 &= \beta_2 + \Omega_2 \cdot h_2 \\ h_3 &= a[f_2] \\ f_3 &= \beta_3 + \Omega_3 \cdot h_3 \end{aligned}$$

L'output finale è $\hat{y}_i = f[\mathbf{x}_i; \phi] = f_3$, che confrontiamo con il valore target y_i per calcolare la loss $\ell_i = \ell(\hat{y}_i, y_i)$.

Nota: Nel forward pass memorizziamo tutti i valori intermedi f_k e h_k , che serviranno per il backward pass.

Backward pass Nel backward pass calcoliamo i gradienti della loss rispetto ai parametri, propagando all'indietro attraverso la rete usando la **chain rule**.

L'idea chiave è calcolare sequenzialmente le derivate parziali:

1. Calcoliamo la derivata della loss rispetto all'output: $\frac{\partial \ell_i}{\partial f_3}$
2. Per ogni layer k (da $K - 1$ a 0, all'indietro), calcoliamo:

- La derivata rispetto ai parametri del layer:

$$\begin{aligned} \frac{\partial \ell_i}{\partial \beta_k} &= \frac{\partial \ell_i}{\partial f_k} \\ \frac{\partial \ell_i}{\partial \Omega_k} &= \frac{\partial \ell_i}{\partial f_k} \cdot h_{k-1}^T \end{aligned}$$

- La derivata rispetto all'input del layer precedente (per propagare il gradiente):

$$\frac{\partial \ell_i}{\partial h_{k-1}} = \Omega_k^T \cdot \frac{\partial \ell_i}{\partial f_k}$$

- La derivata attraverso la funzione di attivazione:

$$\frac{\partial \ell_i}{\partial f_{k-1}} = \frac{\partial \ell_i}{\partial h_{k-1}} \odot a'[f_{k-1}]$$

dove $\odot$ indica il prodotto element-wise (Hadamard product) e $a'[\cdot]$ è la derivata della funzione di attivazione.

Chain rule: Il principio fondamentale è che per calcolare $\frac{\partial \ell_i}{\partial \beta_k}$ usiamo:

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial \ell_i}{\partial f_3} \cdot \frac{\partial f_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial f_2} \cdots \frac{\partial f_{k+1}}{\partial h_k} \cdot \frac{\partial h_k}{\partial f_k} \cdot \frac{\partial f_k}{\partial \beta_k}$$

Durante il backward pass vengono aggiornati i pesi del modello. Backward è l'algoritmo che implementa il gradient descent.

Grafo computazionale Un modo efficace per visualizzare e comprendere la backpropagation è attraverso il **grafo computazionale** (computational graph), una rappresentazione grafica che mostra le operazioni matematiche e le dipendenze tra variabili.

Nel grafo computazionale:

- I **nodi** rappresentano variabili (input, parametri, valori intermedi, output)
- Gli **archi diretti** rappresentano le operazioni e il flusso dei dati
- Il **forward pass** segue la direzione degli archi (da sinistra a destra)
- Il **backward pass** procede nella direzione opposta (da destra a sinistra), propagando i gradienti

Esempio 9.2 (Grafo computazionale per una rete shallow semplice). Consideriamo una rete neurale molto semplice con un solo hidden unit:

$$f_0 = \theta_0 + \theta_1 \cdot x$$

$$h = \text{ReLU}(f_0)$$

$$\hat{y} = \phi_0 + \phi_1 \cdot h$$

$$\ell = (\hat{y} - y)^2$$

Il grafo computazionale corrispondente è:

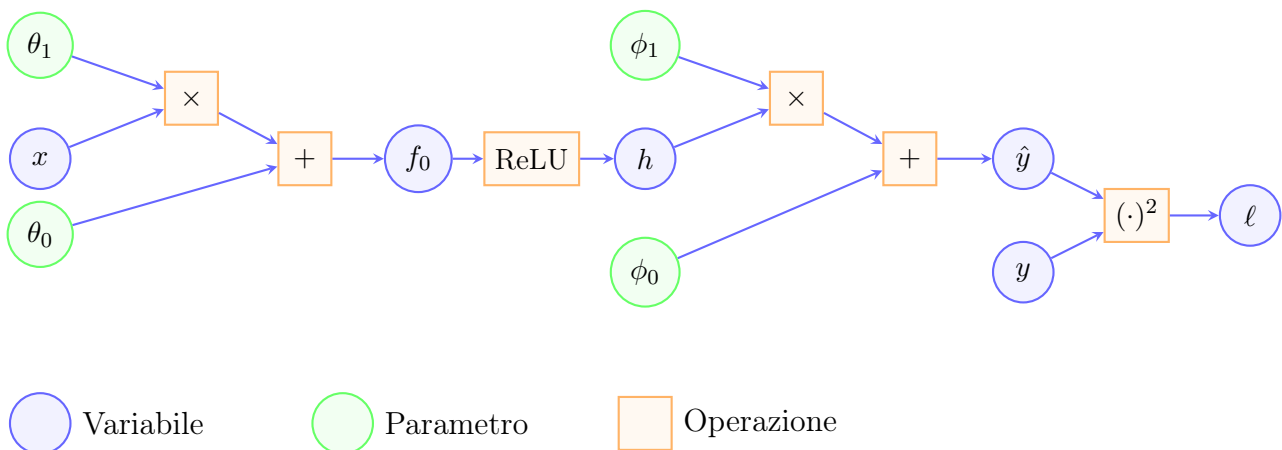


Figura 14: Grafo computazionale per una rete shallow con un hidden unit. Il forward pass procede da sinistra (input) a destra (loss), mentre il backward pass procede da destra a sinistra, calcolando i gradienti usando la chain rule.

Forward pass sul grafo: Partiamo dai nodi di input (x, θ_0, θ_1) e calcoliamo i valori dei nodi seguenti seguendo gli archi. Ogni nodo calcola il suo valore applicando l'operazione ai suoi input.

Backward pass sul grafo: Partiamo dal nodo finale (ℓ) con $\frac{\partial \ell}{\partial \ell} = 1$ e propagiamo all'indietro. Per ogni arco che va da un nodo A a un nodo B , se conosciamo $\frac{\partial \ell}{\partial B}$, calcoliamo $\frac{\partial \ell}{\partial A}$ usando:

$$\frac{\partial \ell}{\partial A} = \frac{\partial \ell}{\partial B} \cdot \frac{\partial B}{\partial A}$$

Quando un nodo ha più archi in uscita (come h nell'esempio), sommiamo i contributi di tutti i percorsi:

$$\frac{\partial \ell}{\partial h} = \sum_{\text{nodi } N \text{ che dipendono da } h} \frac{\partial \ell}{\partial N} \cdot \frac{\partial N}{\partial h}$$

Questo approccio garantisce che ogni derivata parziale sia calcolata esattamente una volta, rendendo l'algoritmo efficiente anche per reti molto profonde.

Esempio 9.3 (Backward pass concreto). Consideriamo l'esempio precedente con 3 hidden layers. Il backward pass procede così:

Step 1 - Output layer:

$$\begin{aligned}\frac{\partial \ell_i}{\partial f_3} &= \frac{\partial \ell_i}{\partial \hat{y}_i} \quad (\text{dipende dalla loss function}) \\ \frac{\partial \ell_i}{\partial \beta_3} &= \frac{\partial \ell_i}{\partial f_3} \\ \frac{\partial \ell_i}{\partial \Omega_3} &= \frac{\partial \ell_i}{\partial f_3} \cdot h_3^T\end{aligned}$$

Step 2 - Hidden layer 3:

$$\begin{aligned}\frac{\partial \ell_i}{\partial h_3} &= \Omega_3^T \cdot \frac{\partial \ell_i}{\partial f_3} \\ \frac{\partial \ell_i}{\partial f_2} &= \frac{\partial \ell_i}{\partial h_3} \odot a'[f_2] \\ \frac{\partial \ell_i}{\partial \beta_2} &= \frac{\partial \ell_i}{\partial f_2} \\ \frac{\partial \ell_i}{\partial \Omega_2} &= \frac{\partial \ell_i}{\partial f_2} \cdot h_2^T\end{aligned}$$

Step 3 - Hidden layer 2:

$$\begin{aligned}\frac{\partial \ell_i}{\partial h_2} &= \Omega_2^T \cdot \frac{\partial \ell_i}{\partial f_2} \\ \frac{\partial \ell_i}{\partial f_1} &= \frac{\partial \ell_i}{\partial h_2} \odot a'[f_1] \\ \frac{\partial \ell_i}{\partial \beta_1} &= \frac{\partial \ell_i}{\partial f_1} \\ \frac{\partial \ell_i}{\partial \Omega_1} &= \frac{\partial \ell_i}{\partial f_1} \cdot h_1^T\end{aligned}$$

Step 4 - Hidden layer 1:

$$\begin{aligned}\frac{\partial \ell_i}{\partial h_1} &= \Omega_1^T \cdot \frac{\partial \ell_i}{\partial f_1} \\ \frac{\partial \ell_i}{\partial f_0} &= \frac{\partial \ell_i}{\partial h_1} \odot a'[f_0] \\ \frac{\partial \ell_i}{\partial \beta_0} &= \frac{\partial \ell_i}{\partial f_0} \\ \frac{\partial \ell_i}{\partial \Omega_0} &= \frac{\partial \ell_i}{\partial f_0} \cdot \mathbf{x}_i^T\end{aligned}$$

Efficienza computazionale La backpropagation è efficiente perché:

- Riutilizza i valori calcolati nel forward pass (h_k, f_k)
- Evita di ricalcolare la stessa derivata più volte grazie alla chain rule
- La complessità è $O(K)$ dove K è il numero di layer, rendendo il training di reti profonde computazionalmente fattibile

In pratica, framework come PyTorch e TensorFlow implementano la backpropagation automaticamente attraverso l'**automatic differentiation**, costruendo un grafo computazionale durante il forward pass e calcolando automaticamente i gradienti nel backward pass.

Dropout Il dropout è una tecnica che viene usata per prevenire che specifiche unità nascoste si adattino troppo ai dati di training e perdere generalizzazione. Questa tecnica consiste nel settare alcuni elementi dell'input tensor a 0 con una certa probabilità p .

Questa pratica previene il co-adattamento (**co-adaptation**) dei pesi delle unità nascoste: ovvero neuroni che diventano molto dipendenti da altri neuroni.

10 Sentiment Analysis

La sentiment analysis fa riferimento all'uso del processamento del linguaggio naturale, testuale e audio allo scopo di analizzare, estrarre e quantificare l'opinione che viene espressa. Questo task è un **task di classificazione multiclasse**:

- **Neutral:** 0
- **Positive:** 1
- **Negative:** 2

Come metriche si usano accuracy, precision, recall, F1-score e ROC-AUC.

Gli input sono testi che vengono trasformati in vettori di token dove ogni dimensione rappresenta un token nel vocabolario (si può fare binario). Questo modello usa **bag of words** per rappresentare i testi.

Esempio 10.1. $V = \{ "a", "cat", "dog", "file", "string" \}$, testo da trasformare: "A string file".

$$\text{one-hot-encoding}(t) = [1 \ 0 \ 0 \ 1 \ 1]^T$$

In `scikit-learn` si può usare la classe `CountVectorizer` per convertire i testi in vettori di token. Questo può essere fatto in modalità binaria o contatore (in una frase conta occorrenze di ogni token/parola). Possiamo usare un'altra tecnica per rappresentare i testi: **TF-IDF** (Term Frequency - Inverse Document Frequency).

Si rappresenta ogni documento/frase con un vettore a lunghezza fissa e ogni termine del vettore indica il prodotto di due termini:

- **Term Frequency (TF):** quante volte un termine appare nel documento
- **Inverse Document Frequency (IDF):** quante volte un termine appare in tutti i documenti

$$\text{IDF}(t) = \log \left(\frac{\text{Total documents}}{\text{Number of documents containing term } t} \right)$$

Conosciamo adesso l'**hash trick** (o Feature Hashing) che risolve il problema della dimensionalità esplosiva del Bag of Words.

Nel Bag of Words classico, dobbiamo mantenere un dizionario globale che mappa ogni parola a un indice univoco, e la dimensione dei vettori cresce linearmente con il vocabolario. Con l'Hashing Trick, invece, non manteniamo alcun dizionario. Utilizziamo una **funzione di hash** $h(\cdot)$ per mappare direttamente ogni token in un indice di un vettore a dimensione fissa D (decisa a priori):

$$\text{index} = h(\text{token}) \pmod{D}$$

- **Vantaggi:**

- **Efficienza di memoria:** La dimensione dei vettori è fissa (D), indipendentemente dalla grandezza del vocabolario.

- **Stateless**: Non serve costruire il vocabolario in anticipo (utile per dati in streaming o dataset enormi).

- **Svantaggi**:

- **Collisioni**: Parole diverse possono finire nello stesso "bucket" (indice), creando rumore. Tuttavia, con un D sufficientemente grande, le collisioni sono rare e spesso tollerate dai modelli.
- **Irreversibilità**: Non si può risalire facilmente dal vettore alle parole originali.

Sentences	Vinay	likes	cricket	Vivek	watching	when	Kohli	bats	Sujeet	playing	enjoys	Cover-drive
Vinay likes cricket.	1	1	1	0	0	0	0	0	0	0	0	0
Vivek likes watching cricket when Kohli bats.	0	1	1	1	1	1	1	1	0	0	0	0
Sujeet likes playing cricket.	0	1	1	0	0	0	0	0	1	1	0	0
Vivek enjoys cover-drive of Kohli.	0	0	0	1	0	0	1	0	0	0	1	1

➔ **Bag-of-Word**

	Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Feature 6	Feature 7	Feature 8
Vinay likes cricket	0	0	0	0	1	1	0	1
Vivek likes watching cricket when Kohli bats	0	1	2	1	1	1	0	1
Sujeet likes playing cricket	0	0	0	0	2	0	0	2
Vivek enjoys cover-drive of Kohli	1	1	1	0	0	0	1	0

➔ **Feature hashing**

Figura 15: Confronto tra Bag of Words (sopra) e Feature Hashing (sotto). Nel Feature Hashing, i token vengono mappati in un numero fisso di bucket tramite una funzione di hash, riducendo la dimensionalità.

11 Modeling a language

Il natural language processing (NLP) fa uso di Statistical Language Models (LMs) per modellare il linguaggio. Questi modelli sono usati per raccogliere informazioni, riconoscimento vocale, spell checking, traduzione e generazione di testo.

Definizione 11.1 (LM). Un Language Model (LM) definisce una distribuzione di probabilità su “token” per misurare la verosimiglianza di una sequenza di token.

$$p(s) = p(t_1, t_2, \dots, t_n)$$

In questo senso i task NLP possono essere visti come il calcolo di uno score di probabilità per componenti del task:

- **Information Retrieval**: Probabilità di un documento data una query:

$$p(d|q) = p(q|d) \frac{p(d)}{p(q)}$$

- **Translation** (Noisy channel model):

$$p(d_{eng}|d_{ita}) \propto p(d_{eng}, d_{ita}) = p(d_{ita}|d_{eng})p(d_{eng})$$

Definizione 11.2 (K-Gram Language Models). Un K-Gram Language Model (KLM) cerca di stimare la probabilità di una sequenza. L’approccio naive basato sulla frequenza relativa dell’intera frase è impraticabile a causa della scarsità dei dati:

$$p(s) = \frac{\text{count}(s)}{\text{count}(\text{all sentences})} \quad (\text{impossibile da calcolare})$$

Si usa quindi la **chain rule** approssimata all'ordine k : assumiamo che la probabilità di un token dipenda solo dai $k - 1$ token precedenti (assunzione di Markov):

$$p(t_m | t_{m-1}, \dots, t_1) \approx p(t_m | t_{m-1}, \dots, t_{m-k+1})$$

Un caso particolare molto diffuso è il **Bigram** ($k = 2$), dove la probabilità dipende solo dal token immediatamente precedente:

$$p(t_m | t_{m-1}) = \frac{\text{count}(t_{m-1}, t_m)}{\text{count}(t_{m-1})}$$

Definizione 11.3 (Smoothing). Smoothing è una tecnica che viene usata per evitare di avere probabilità zero per sequenze rare o mai viste, ovvero situazioni in cui $p(w) = 0$.

Lo **smoothing** consiste nell'aggiungere una piccola quantità di probabilità a tutte le sequenze.

$$p_{\text{smooth}}(w_m | w_{m-1}) = \frac{\text{count}(w_{m-1}, w_m) + \alpha}{\sum_{w' \in \mathcal{V}} \text{count}(w_{m-1}, w') + \alpha |\mathcal{V}|} \quad (\text{Lidstone smoothing})$$

dove α è un parametro di smoothing e $|\mathcal{V}|$ è la cardinalità del vocabolario. Laplace smoothing è un caso particolare di smoothing dove $\alpha = 1$, Jeffreys-Perks law prevede $\alpha = 0.5$.

11.0.1 Language Models in IR

Una delle applicazioni principali dei LM è nell'Information Retrieval. Dato una query q e una collezione di documenti, l'obiettivo è ordinare i documenti calcolando la probabilità a posteriori $P(d|q)$ (quanto è probabile che il documento d sia rilevante data la query q).

Ogni documento d viene modellato come un Language Model M_d (una distribuzione di probabilità sulle parole). Usando la regola di Bayes:

$$P(d|q) = \frac{P(q|d)P(d)}{P(q)}$$

Poiché il nostro obiettivo è solo *ordinare* i documenti per una data query, possiamo semplificare:

- $P(q)$ è costante per tutti i documenti (non influenza il ranking).
- $P(d)$ è la probabilità a priori del documento. Spesso si assume uniforme (tutti i documenti sono ugualmente probabili a priori), quindi può essere ignorata.

Concludiamo che il ranking dipende da:

$$P(d|q) \propto P(q|d)$$

Questo approccio è detto **Generative Model**: calcoliamo la probabilità che il Language Model del documento d generi la sequenza di parole della query q .

Definizione 11.4 (OOV: Out of Vocabulary). Se conosciamo in anticipo tutte le parole allora possiamo fissare la lunghezza $|V|$ del vocabolario, ma questi sono casi più unici che rari e invece si usa la **OOV** (Out of Vocabulary) per gestire le parole che non sono presenti nel vocabolario, creando un token $\langle \text{UNK} \rangle$ per le parole sconosciute.

Si trainano dunque le probabilità $\langle \text{UNK} \rangle$ e si crea un vocabolario esteso $\mathcal{V}$. In fase di normalizzazione, ogni training word non presente in $\mathcal{V}$ viene sostituita con $\langle \text{UNK} \rangle$. Dunque si tratta $\langle \text{UNK} \rangle$ come una parola reale.

11.1 Neural Language Models

L'idea chiave dei modelli neurali del linguaggio è trattare la predizione come un task di apprendimento discriminativo: vogliamo predire la probabilità di una parola target w_i dato un contesto di parole C_j .

$$P(w_i | C_j) = \frac{e^{\theta_{w_i} \cdot \theta_{C_j}}}{\sum_{w' \in \mathcal{V}} e^{\theta_{w'} \cdot \theta_{C_j}}} = \text{softmax}(\dots)$$

11.1.1 Codifica delle parole

Il modo più semplice per rappresentare le parole in ingresso a una rete è il **One-Hot Encoding**. Se abbiamo un vocabolario V di dimensione $|V|$, ogni parola è un vettore di dimensione $|V|$ con tutti zeri tranne un 1 nella posizione corrispondente all'ID della parola.

$$e(w_1) = [1, 0, 0, \dots], \quad e(w_2) = [0, 1, 0, \dots]$$

Questa rappresentazione è però inefficiente: vettori sparsi, ortogonali tra loro (nessuna nozione di similarità) e dimensione enorme.

11.1.2 Approccio con MLP

Un'idea semplice è concatenare i vettori one-hot delle parole di contesto e passarli a un Multi-Layer Perceptron (MLP).

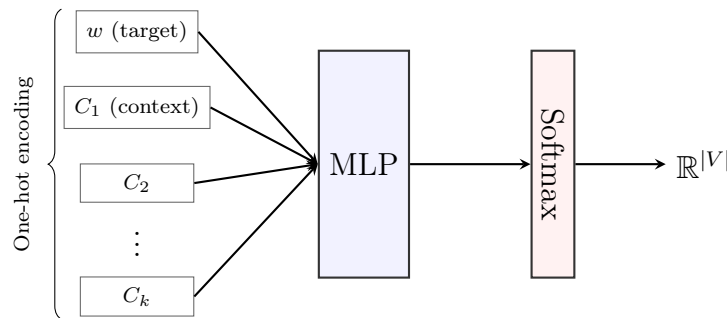


Figura 16: Schema di un semplice Neural LM basato su MLP. L'input è la concatenazione dei vettori one-hot del target e del contesto.

Il problema di questo approccio è l'**esplosione dei parametri**. L'input della rete è la concatenazione del vettore target w e dei k vettori di contesto C_j . Dato che ogni vettore è codificato one-hot (lunghezza $|V|$), la dimensione totale dell'input è:

$$\text{Dim Input} = (1 + k) \cdot |V|$$

Se il primo hidden layer ha dimensione $|V|$, la matrice dei pesi che connette l'input a questo layer avrà dimensione:

$$W \in \mathbb{R}^{((1+k)|V|) \times |V|}$$

Il numero totale di parametri scala quindi con $|V|^2$, rendendo il modello intrattabile per vocabolari realistici (dove $|V| \approx 10^5$). Questo motiva la necessità di rappresentazioni più dense e compatte (**embeddings**).

11.2 Recurrent Neural Networks LMs

Le Recurrent Neural Networks (RNN) sono progettate specificamente per gestire dati sequenziali. A differenza degli MLP che processano input a dimensione fissa, le RNN mantengono uno **stato nascosto** (hidden state) che funge da memoria del passato.

11.2.1 Funzionamento

Ad ogni passo temporale t , la rete:

1. Prende in input la parola corrente w_t (codificata come one-hot).
2. La trasforma in un vettore denso x_t tramite una matrice di embedding Φ .

3. Calcola il nuovo stato nascosto h_t combinando l'input corrente x_t e lo stato nascosto precedente h_{t-1} .
4. Produce un output y_t (predizione della parola successiva).

Le equazioni che governano il processo sono:

$$x_t = \Phi w_t \quad (\text{Embedding lookup})$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \quad (\text{Aggiornamento stato})$$

$$y_t = W_{hy}h_t \quad (\text{Predizione / Logits})$$

Dove:

- $h_t \in \mathbb{R}^k$ (dimensione hidden)
- $x_t \in \mathbb{R}^d$ (dimensione embedding)
- $\Phi \in \mathbb{R}^{d \times |V|}$ è la matrice di embedding.
- W_{hh}, W_{xh}, W_{hy} sono matrici di pesi condivise per tutti i time steps.

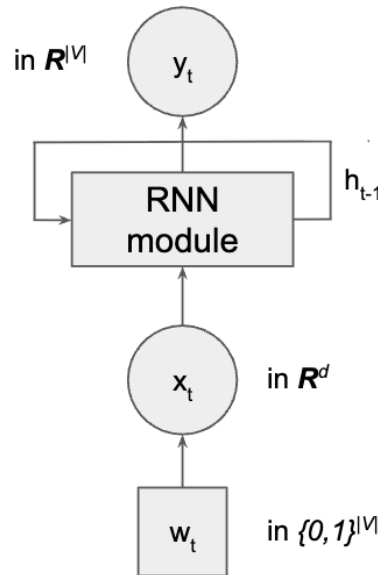


Figura 17: Modulo RNN: l'input viene processato insieme allo stato precedente per generare output e nuovo stato.

11.2.2 Efficienza dei parametri

Il vantaggio enorme rispetto agli MLP one-hot è la riduzione dei parametri. Mentre un MLP su one-hot richiedeva $\approx |V|^2$ parametri, una RNN richiede:

$$\text{Params} \approx d|V| + dk + k^2 + k|V|$$

Con valori realistici ($|V| = 10^5, d = 100, k = 100$), si passa da $\approx 10^{11}$ a $\approx 10^7$ parametri, rendendo il training fattibile.

11.2.3 Unrolling e Training

Per addestrare una RNN, si utilizza la tecnica dell'**Unrolling** (o srotolamento). La rete viene replicata per ogni time step della sequenza, condividendo gli stessi pesi. L'addestramento avviene tramite **Backpropagation Through Time (BPTT)**, calcolando i gradienti attraverso tutti i passi temporali passati.

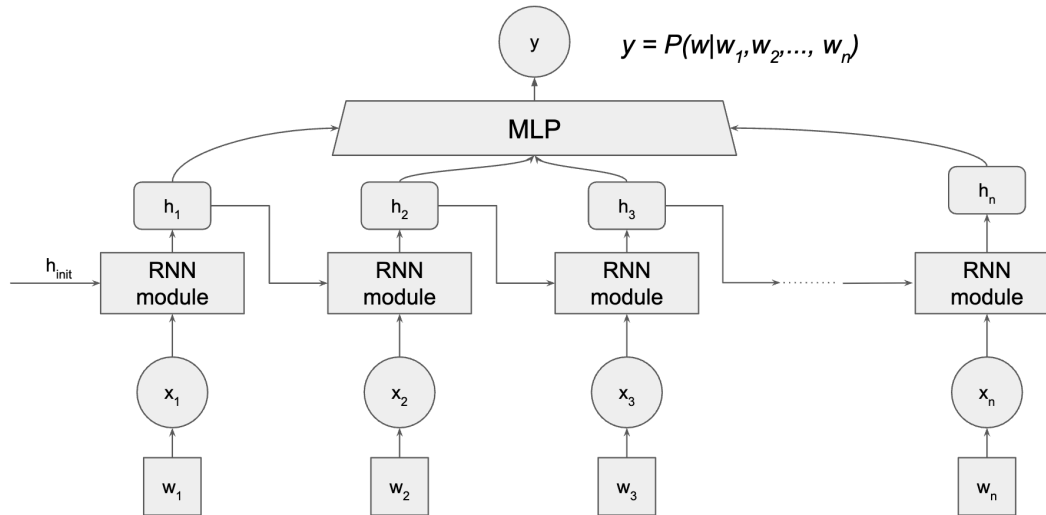


Figura 18: Unrolling di una RNN per il training di un Language Model. Ogni output h_t contribuisce alla predizione e allo stato successivo.

11.3 Word Embeddings

Le rappresentazioni one-hot hanno due grossi limiti: l'altissima dimensionalità (pari alla grandezza del vocabolario $|V|$) e l'incapacità di catturare la similarità semantica (tutti i vettori sono ortogonali).

I **Word Embeddings** risolvono questi problemi mappando ogni parola in un vettore denso di dimensione ridotta d (es. $d = 300$), in modo tale che parole con significati simili abbiano vettori vicini nello spazio geometrico.

11.3.1 Word2Vec

Word2Vec è una famiglia di algoritmi efficienti per apprendere questi embeddings. Il problema principale nei modelli neurali classici è il calcolo della Softmax finale su tutto il vocabolario, che ha costo $O(|V|)$. Word2Vec aggira questo problema trasformando il task in una classificazione binaria tramite **Negative Sampling** (o Noise Contrastive Estimation): invece di predire la parola esatta tra tutte le possibili, il modello impara a distinguere tra coppie (parola, contesto) "reali" e coppie "rumore" generate casualmente.

Esistono due architetture principali per Word2Vec:

CBOW (Continuous Bag of Words) Il modello predice la **parola target** basandosi sulla media delle parole del **contesto** (finestra temporale).

- **Input:** Vettori delle parole di contesto.
- **Hidden Layer:** Media dei vettori di input (da qui "Bag of Words", l'ordine non conta nella media).
- **Output:** Probabilità della parola target.
- **Caratteristiche:** Più veloce da addestrare, funziona bene per parole frequenti.

Esempio 11.1 (CBOW). Consideriamo la frase: “Today is a **pleasant** day”. Se il target è “pleasant” e la finestra è di dimensione 2, il contesto sarà:

$$C = \{\text{“Today”, “is”, “a”, “day”}\}$$

L’obiettivo è massimizzare la probabilità:

$$P(\text{“pleasant”} \mid \text{“Today”, “is”, “a”, “day”})$$

calcolata mediando i vettori embedding delle parole di contesto e passandoli alla softmax.

Formalmente, l’obiettivo è massimizzare:

$$\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_{t-m}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+m})$$

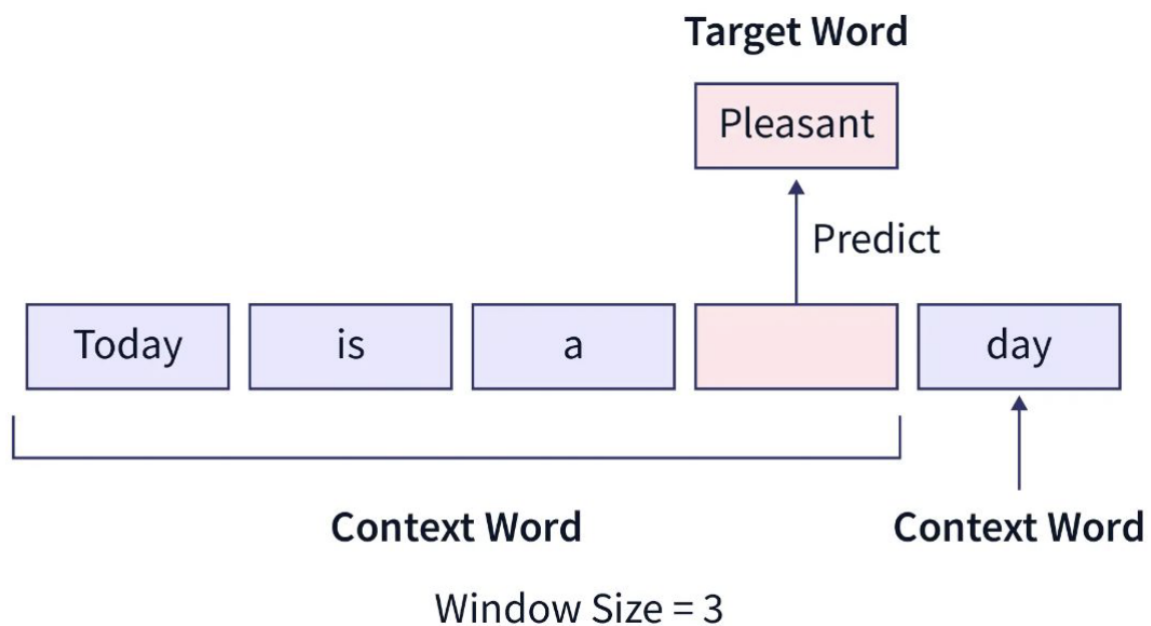


Figura 19: CBOW: le parole del contesto vengono usate per predire la parola target.

Skip-Gram Il modello fa l’opposto: data una **parola target** centrale w_t , cerca di predire le parole del **contesto** circostante.

- **Input:** Parola target (es. “banking”).
- **Output:** Probabilità per ogni parola del contesto (es. “problems”, “turning”, “into”, “crises”).
- **Caratteristiche:** Funziona meglio con dataset piccoli e rappresenta meglio le parole rare, ma è computazionalmente più costoso.

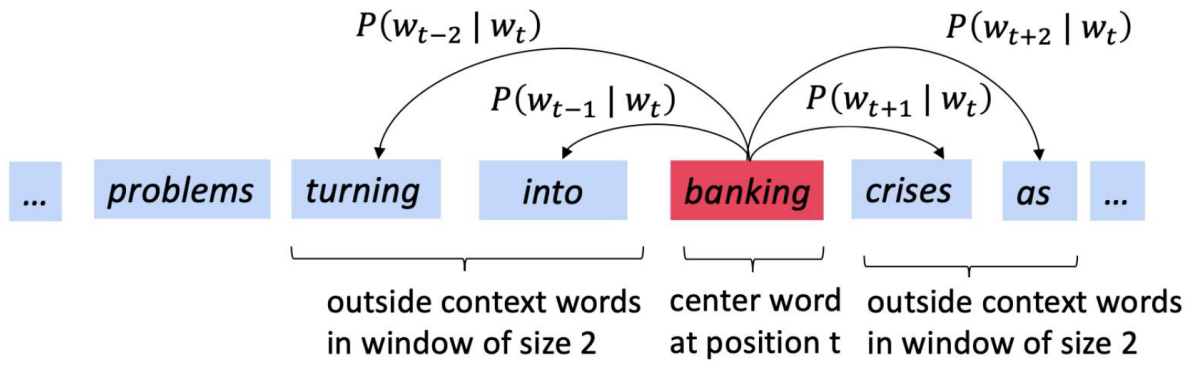


Figura 20: Skip-Gram: la parola target viene usata per predire le parole del contesto circostante.

Formalmente, dato un corpus di parole $w_1, w_2, \dots, w_T$, Skip-Gram massimizza la probabilità logaritmica media:

$$\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} | w_t)$$

dove m è la dimensione della finestra di contesto. La probabilità $P(w_{out} | w_{in})$ è modellata tramite **softmax** usando i vettori di embedding v_w (vettore input) e u_w (vettore contesto):

$$P(w_{out} | w_{in}) = \frac{\exp(u_{out}^T v_{in})}{\sum_{w \in V} \exp(u_w^T v_{in})}$$

In pratica, il termine al denominatore è costoso da calcolare e viene approssimato con il **Negative Sampling**.

11.3.2 Proprietà degli Embeddings e Valutazione

Una delle caratteristiche più interessanti dei Word Embeddings è che catturano relazioni semantiche e sintattiche tramite operazioni algebriche nello spazio vettoriale.

Analogie Vettoriali Le relazioni tra concetti tendono a corrispondere a direzioni costanti nello spazio. È quindi possibile risolvere analogie del tipo “A sta a B come C sta a D” calcolando:

$$v_B - v_A + v_C \approx v_D$$

L'esempio più celebre è:

$$v_{\text{King}} - v_{\text{Man}} + v_{\text{Woman}} \approx v_{\text{Queen}}$$

Altre relazioni catturate includono:

- Capitali-Statì: $v_{\text{Paris}} - v_{\text{France}} + v_{\text{Italy}} \approx v_{\text{Rome}}$
- Sintassi (comparativi): $v_{\text{Better}} - v_{\text{Good}} + v_{\text{Bad}} \approx v_{\text{Worse}}$

Valutazione (Evaluation) Esistono due approcci principali per misurare la qualità degli embeddings:

- **Intrinsic Evaluation:** Valuta gli embedding in isolamento su task specifici (es. analogie, similarità). È veloce ma non garantisce prestazioni migliori nell'applicazione finale.
- **Extrinsic Evaluation:** Valuta gli embedding impiegandoli in task reali (es. Machine Translation) e misurando la loss/accuracy finale. È il metodo più affidabile ma computazionalmente costoso.

12 Recurrent Neural Networks

Le reti neurali ricorrenti (RNN) sono un tipo di rete neurale specializzato nel processare **sequenze** di valori $x_1, x_2, \dots, x_T$. L'elemento chiave è il **parameter sharing**: lo stesso set di parametri (pesi) viene utilizzato ad ogni passo temporale. Questo permette al modello di:

- Gestire sequenze di input di lunghezza variabile (non fissata a priori).
- Generalizzare pattern appresi in una posizione della sequenza anche ad altre posizioni.

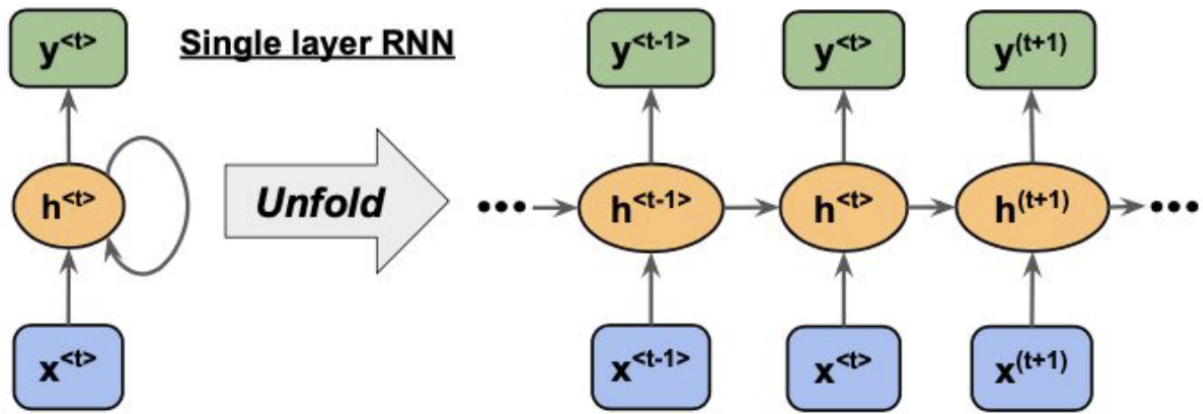
12.1 Unfolding (Srotolamento)

Le RNN possono essere descritte come sistemi dinamici definiti dalla ricorrenza:

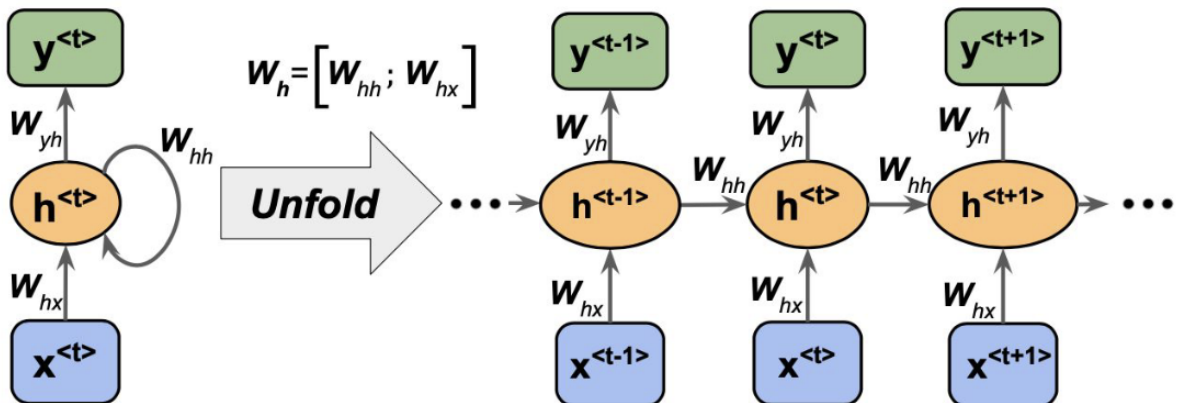
$$h^{(t)} = f_W(h^{(t-1)}, x^{(t)})$$

dove $h^{(t)}$ è lo stato nascosto (cell state) al tempo t , che funge da memoria della rete, $x^{(t)}$ è l'input al tempo t , e f_W è una funzione con parametri W .

Per una sequenza finita di lunghezza τ , questa formula può essere “srotolata” (unfolded) applicando la definizione $\tau - 1$ volte, trasformando la ricorrenza temporale in una struttura a grafo computazionale profondo.



In forma matriciale



Una RNN vanilla può avere inizializzazione

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}; \theta)$$

dove

$$h_t = g_\theta(h_{t-1}, x_t) \quad h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \quad o_t = \text{softmax}(W_{ho}h_t)$$

12.2 Addestramento: Backpropagation Through Time

Per addestrare una RNN, si considera il dataset come una sequenza ordinata di coppie input-output. È necessario specificare uno stato nascosto iniziale h_0 , tipicamente inizializzato come un vettore di zeri.

L'algoritmo utilizzato è la **Backpropagation Through Time (BPTT)**, che estende la backpropagation classica alla struttura temporale:

- **Unfolding:** La rete viene srotolata nel tempo, trattando ogni step come un layer che condivide gli stessi parametri.
- **Gradienti accumulati:** Poiché i pesi sono condivisi, il gradiente della loss totale rispetto ai pesi è la somma dei gradienti calcolati ad ogni istante temporale t :

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L^{(t)}}{\partial W_{hh}}$$

Truncated BPTT: Per gestire sequenze molto lunghe ed evitare costi computazionali eccessivi, si utilizza una versione troncata. La retropropagazione dell'errore viene limitata a un numero fisso di passi indietro (s step), ignorando le dipendenze temporali più remote per l'aggiornamento corrente. Questo approccio rende l'apprendimento più efficiente e mitiga problemi come l'esplosione del gradiente.

12.3 Vanishing ed Exploding Gradient

L'addestramento delle RNN su sequenze lunghe soffre di instabilità numerica dovuta alla natura ricorsiva del gradiente. Considerando la relazione base della RNN $h^{(t)} = \sigma(W_{hh}h^{(t-1)} + W_{hx}x^{(t)})$, e assumendo per semplicità una funzione di attivazione lineare $\sigma(x) = x$, la derivata dello stato corrente rispetto al precedente è data direttamente dalla matrice dei pesi ricorrenti:

$$\frac{\partial h^{(t)}}{\partial h^{(t-1)}} = W_{hh}$$

Applicando la regola della catena per calcolare il gradiente della loss al tempo i rispetto a uno stato passato j (con distanza $\ell = i - j$), si ottiene una produttoria che coinvolge la potenza della matrice dei pesi:

$$\frac{\partial L^{(i)}}{\partial h^{(j)}} = \frac{\partial L^{(i)}}{\partial h^{(i)}} \prod_{j < t \leq i} \frac{\partial h^{(t)}}{\partial h^{(t-1)}} \propto W_{hh}^\ell$$

Analizzando W_{hh} tramite la decomposizione in autovalori (se diagonalizzabile $W_{hh} = Q^{-1}\Delta Q$), la potenza diventa $W_{hh}^\ell = Q^{-1}\Delta^\ell Q$. Il comportamento del gradiente dipende quindi dagli autovalori λ nella matrice diagonale Δ :

- **Vanishing Gradient** ($\lambda < 1$): Il termine λ^ℓ tende esponenzialmente a zero. Il contributo degli input lontani nel tempo svanisce, rendendo difficile apprendere dipendenze a lungo termine.
- **Exploding Gradient** ($\lambda > 1$): Il termine λ^ℓ cresce esponenzialmente. I gradienti diventano enormi, causando instabilità numerica e aggiornamenti distruttivi dei pesi.

12.4 Long Short-Term Memory (LSTM)

Possiamo osservare per le RNN che quando si fa backpropagation sui gradienti, i parametri più lontani nel tempo vengono aggiornati molto meno rispetto a quelli più vicini.

Le LSTM usano oltre all'hidden state $\mathbf{h}^{(t)}$, anche una **cell state** $\mathbf{c}^{(t)}$, che funge da memoria a **lungo termine** e i modelli possono dunque interagirci cancellando, scrivendo o leggendone il contenuto.

I **gate** permettono di controllare il flusso di informazioni nella cell state. Ad ogni step temporale t un gate può essere aperto (1), chiuso (0) o trovarsi in uno stato intermedio (0,1).

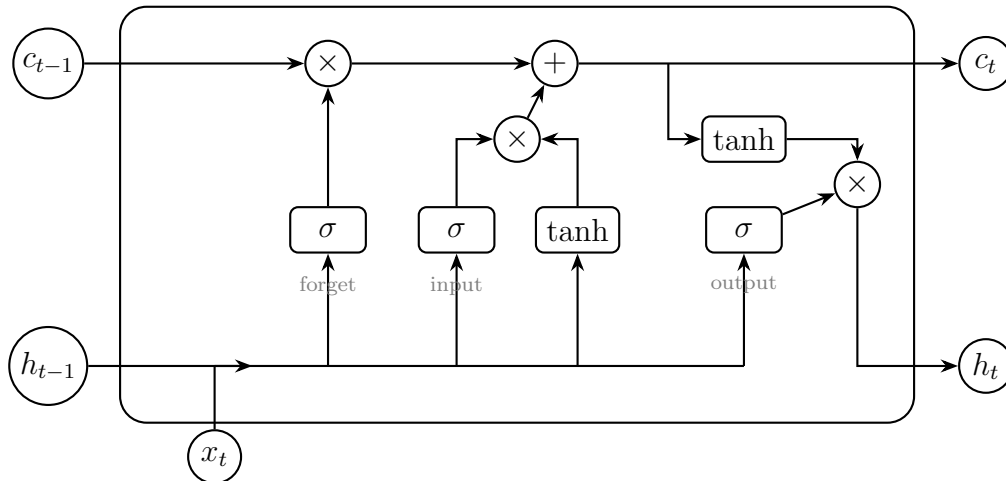
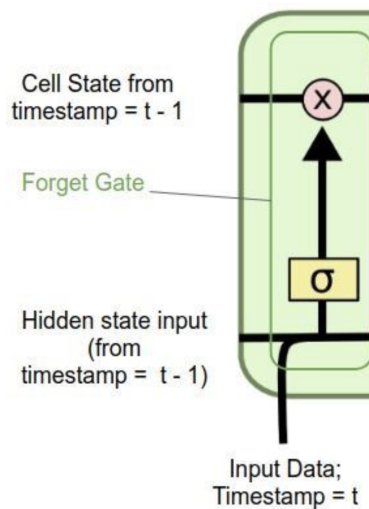


Figura 21: Modulo LSTM: i gate (forget, input, output) controllano il flusso di informazioni attraverso la cell state c_t e producono l'hidden state h_t .

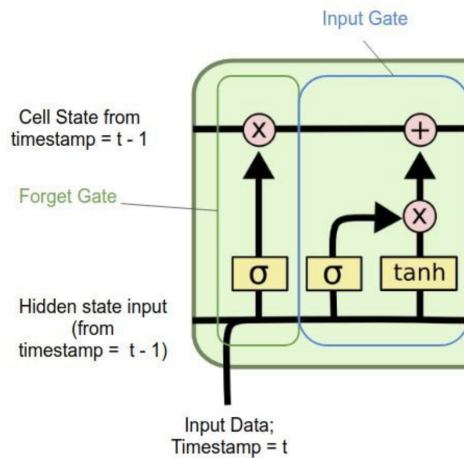
Forget gate: controlla cosa viene tenuto e cosa dimenticato dalla cell state precedente.

$$f^{(t)} = \sigma(W_f \cdot h^{(t-1)} + U_f \cdot x^{(t)} + b_f)$$



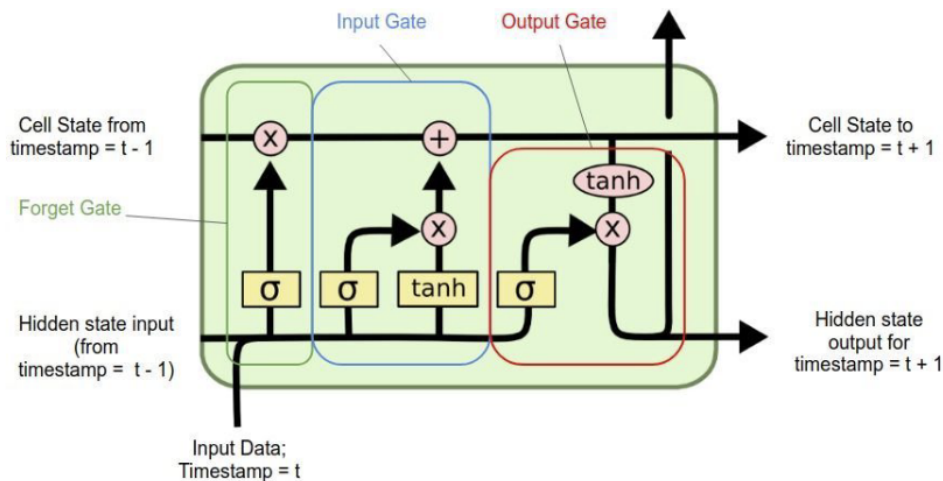
Input gate: controlla quanta parte dell'input corrente $x^{(t)}$ viene scritta nella cell state.

$$i^{(t)} = \sigma(W_i \cdot h^{(t-1)} + U_i \cdot x^{(t)} + b_i)$$



Output gate: controlla quanta parte della cell state viene resa disponibile all'output.

$$o^{(t)} = \sigma(W_o \cdot h^{(t-1)} + U_o \cdot x^{(t)} + b_o)$$



Candidato cell state: il valore candidato da aggiungere alla memoria, prodotto dalla tanh nell'input gate:

$$\tilde{c}^{(t)} = \tanh(W_c \cdot h^{(t-1)} + U_c \cdot x^{(t)} + b_c)$$

Aggiornamento cell state: combina il forget (cosa dimenticare) e l'input (cosa aggiungere):

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

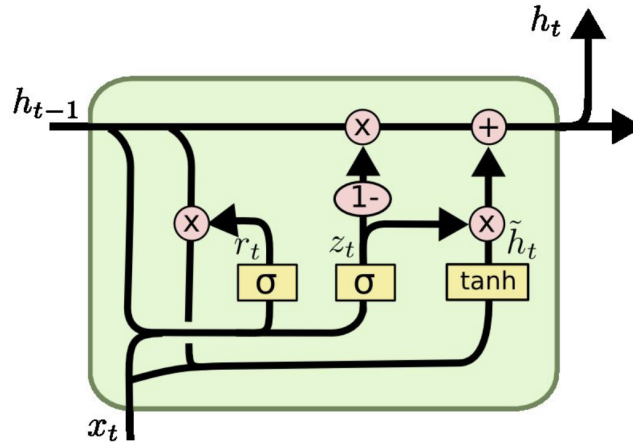
Hidden state: l'output gate modula il cell state per produrre l'output:

$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

Perché le LSTM mitigano il vanishing gradient? Nella formula del cell state, l'informazione può fluire attraverso il tempo tramite **somme** (non solo prodotti). Se il forget gate $f^{(t)} \approx 1$, il gradiente può propagarsi quasi inalterato lungo la “highway” del cell state, evitando il decadimento esponenziale tipico delle RNN vanilla.

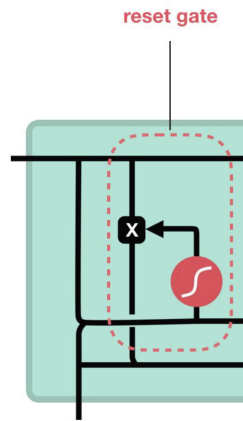
12.5 Gated Recurrent Unit (GRU)

Le GRU sono un'alternativa più semplice alle LSTM: non usano cell state, ma ad ogni iterazione è mantenuto solo un hidden state $\mathbf{h}^{(t)}$.



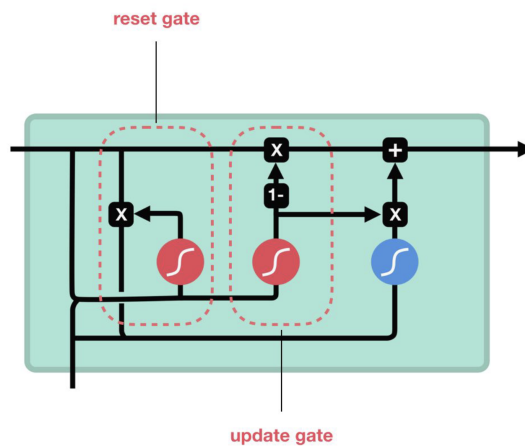
Reset gate: controlla quanta parte dell'hidden state precedente $h^{(t-1)}$ viene utilizzata per calcolare il candidato hidden state.

$$r^{(t)} = \sigma(W_r \cdot h^{(t-1)} + U_r \cdot x^{(t)} + b_r)$$



Update gate: controlla quale parte dell'hidden state è aggiornata e quale mantenuta.

$$u^{(t)} = \sigma(W_u \cdot h^{(t-1)} + U_u \cdot x^{(t)} + b_u)$$



Candidato hidden state: il reset gate seleziona quali parti dello stato precedente usare per calcolare il nuovo contenuto:

$$\tilde{h}^{(t)} = \tanh(W_r \cdot (r^{(t)} \odot h^{(t-1)}) + U_h \cdot x^{(t)} + b_h)$$

Hidden state: l'update gate controlla **simultaneamente** cosa mantenere del passato e cosa aggiornare:

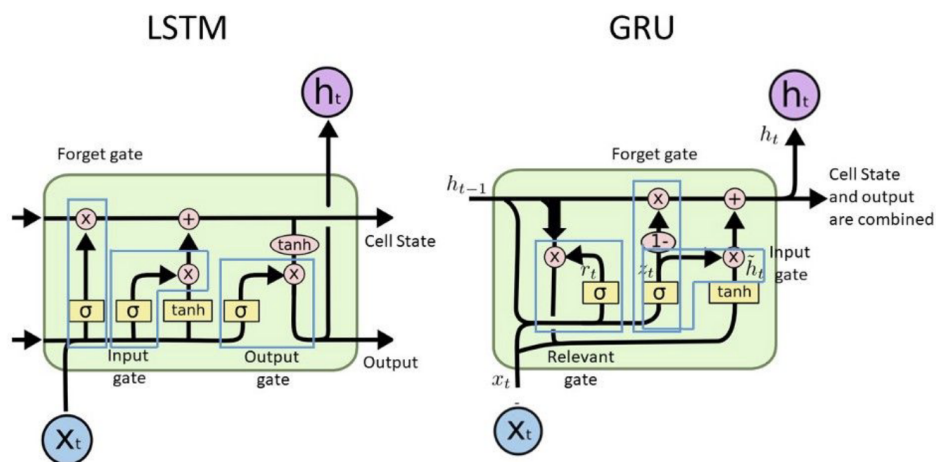
$$h^{(t)} = (1 - u^{(t)}) \odot h^{(t-1)} + u^{(t)} \odot \tilde{h}^{(t)}$$

Quando $u^{(t)} \approx 0$, lo stato precedente viene preservato (utile per dipendenze a lungo termine). Quando $u^{(t)} \approx 1$, lo stato viene aggiornato con il candidato.

GRU vs LSTM

- **GRU:** 2 gate (reset, update), nessun cell state separato, meno parametri
- **LSTM:** 3 gate (forget, input, output), cell state dedicato, più espressivo

In pratica, le prestazioni sono spesso comparabili; le GRU sono preferite quando si vuole un modello più leggero.



12.6 Sequence to Sequence (Seq2Seq)

Di base le RNN sono modelli che trasformano sequenze di input in sequenze di output. A seconda della cardinalità di input e output, si distinguono diverse architetture:

- **Many-to-one:** sequenza in input, singolo output (es. sentiment analysis)
- **One-to-many:** singolo input, sequenza in output (es. image captioning)
- **Many-to-many:** sequenza in input, sequenza in output. Due varianti:
 - Output sincronizzato con l'input (es. POS tagging)
 - Output generato dopo aver processato tutto l'input (es. traduzione)

Paradigma Encoder-Decoder Per task many-to-many con lunghezze diverse tra input e output, si usa l'architettura encoder-decoder:

- **Encoder:** processa la sequenza di input $(x_0, x_1, \dots, x_n)$ e la comprime in un **feature vector** (o context vector) di dimensione fissa.
- **Decoder:** a partire dal feature vector, genera la sequenza di output $(y_0, y_1, \dots, y_m)$ in modo autoregressivo.

Nella versione deep learning, sia encoder che decoder possono essere composti da più layer impilati (stacked), aumentando la capacità rappresentativa del modello.

Limitazione: information bottleneck Comprimere un'intera sequenza in un singolo vettore di dimensione fissa crea un collo di bottiglia informativo. Per sequenze lunghe, il context vector potrebbe non catturare tutti i dettagli rilevanti. Questo problema ha motivato l'introduzione del meccanismo di **Attention**.

12.7 Modelli Autoregressivi

Un modello è **autoregressivo** quando genera l'output un elemento alla volta, condizionando ogni predizione sugli elementi già generati. La probabilità di una sequenza viene fattorizzata come:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$$

Questo è esattamente ciò che fanno i language model e i decoder nei modelli Seq2Seq: ad ogni step t , il modello predice la distribuzione sul prossimo token x_t dato il contesto precedente.

Teacher Forcing Durante il **training**, invece di usare l'output generato dal modello come input per lo step successivo, si usa il **ground truth** (la sequenza target corretta). Questo accelera la convergenza e stabilizza l'addestramento. Altrimenti si può decidere di usare il ground truth con probabilità p , mentre con probabilità $1 - p$ si usa l'output generato dal modello.

Durante l'**inferenza**, non avendo accesso al ground truth, il modello usa i propri output precedenti (modalità autoregressiva pura). Questo può causare *exposure bias*: errori nei primi step si propagano e amplificano.

Osservazione 12.1 (Word2Vec drawback). *Word2Vec è un modello di embedding di parole che associa un vettore ad ogni parola. Tuttavia questo non tiene conto della struttura sintattica e semantica delle parole, per cui la stessa parola può assumere più significati diversi a seconda del contesto.*

12.8 ELMo

ELMo (*Embeddings from Language Models*) genera **embeddings contestuali**: la stessa parola riceve rappresentazioni diverse a seconda del contesto in cui appare.

L'architettura usa una **biLSTM** (bidirezionale):

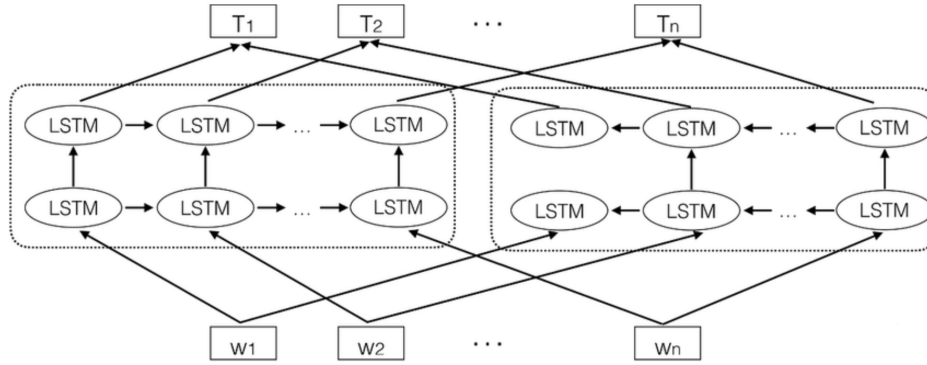
- **Forward LSTM**: processa la sequenza da sinistra a destra
- **Backward LSTM**: processa la sequenza da destra a sinistra

Il modello è composto da più layer. L'embedding finale di una parola è una **combinazione pesata** delle rappresentazioni di tutti i layer:

$$\mathbf{ELMo}_k = \gamma \sum_{j=0}^L s_j \mathbf{h}_{k,j}$$

dove $\mathbf{h}_{k,j}$ è la rappresentazione della parola k al layer j , s_j sono pesi softmax-normalizzati (appresi per il task), e γ è un fattore di scala.

I layer catturano informazioni diverse: quelli più bassi tendono a catturare aspetti sintattici, quelli più alti aspetti semantici.



13 Attention Mechanism

I modelli Seq2Seq classici comprimono l'intera sequenza di input in un singolo context vector di dimensione fissa. Questo crea un **information bottleneck**: per sequenze lunghe, il modello tende a dimenticare le prime parti dell'input.

L'**Attention mechanism** risolve questo problema permettendo al decoder di “guardare” direttamente tutti gli hidden state dell'encoder, pesandoli dinamicamente ad ogni step di decodifica.

13.1 Encoder Bidirezionale

L'encoder è una RNN bidirezionale che produce, per ogni posizione i , due hidden state:

- $\vec{h}_i$: forward (da sinistra a destra)
- $\overleftarrow{h}_i$: backward (da destra a sinistra)

La rappresentazione finale è la concatenazione: $h_i = [\vec{h}_i^\top; \overleftarrow{h}_i^\top]^\top$, che cattura il contesto sia precedente che successivo.

13.2 Context Vector Dinamico

Invece di un unico context vector, l'attention calcola un **context vector diverso** c_t per ogni output y_t :

$$c_t = \sum_{i=1}^n \alpha_{t,i} h_i$$

dove $\alpha_{t,i}$ sono i **pesi di attenzione** (o alignment weights), che indicano quanto l'output al tempo t deve “prestare attenzione” all'input in posizione i .

13.3 Calcolo degli Alignment Scores

I pesi $\alpha_{t,i}$ sono calcolati tramite softmax su uno score di allineamento:

$$\alpha_{t,i} = \frac{\exp(\text{score}(s_{t-1}, h_i))}{\sum_{i'=1}^n \exp(\text{score}(s_{t-1}, h_{i'}))}$$

dove s_{t-1} è lo stato del decoder allo step precedente. La funzione $\text{score}(\cdot)$ misura la compatibilità tra lo stato del decoder e ogni hidden state dell'encoder.

Additive Attention (Bahdanau) Una scelta comune per lo score è l'*additive attention* che misura la similarità:

$$\text{score}(s_{t-1}, h_i) = v^\top \tanh(W_s s_{t-1} + W_h h_i)$$

dove v , W_s , W_h sono parametri appresi.

13.4 Decoder con Attention

Lo stato del decoder dipende ora dal context vector dinamico:

$$s_t = f(s_{t-1}, y_{t-1}, c_t)$$

In questo modo, ad ogni step il decoder può focalizzarsi sulle parti dell'input più rilevanti per generare l'output corrente. I pesi $\alpha_{t,i}$ possono essere visualizzati come una “mappa di attenzione” che mostra quali parole dell'input influenzano maggiormente ogni parola dell'output.

Name	Alignment score function	Citation
Content-base attention	$\text{score}(s_t, h_i) = \text{cosine}[s_t, h_i]$	Graves2014
Additive(*)	$\text{score}(s_t, h_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a [s_t; h_i])$	Bahdanau2015
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a s_t)$ Note: This simplifies the softmax alignment to only depend on the target position.	Luong2015
General	$\text{score}(s_t, h_i) = s_t^\top \mathbf{W}_a h_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer.	Luong2015
Dot-Product	$\text{score}(s_t, h_i) = s_t^\top h_i$	Luong2015
Scaled Dot-Product(^)	$\text{score}(s_t, h_i) = \frac{s_t^\top h_i}{\sqrt{n}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.	Vaswani2017

13.5 Self-Attention

La **self-attention** (o *intra-attention*) è un meccanismo che mette in relazione diverse posizioni della **stessa sequenza** per calcolare una rappresentazione contestuale. A differenza dell'attention encoder-decoder, qui query, chiavi e valori provengono tutti dalla stessa sequenza.

Query, Key, Value L'input viene proiettato in tre rappresentazioni:

- **Query** (Q): “cosa sto cercando?”
- **Key** (K): “cosa contiene ogni posizione?”
- **Value** (V): “quale informazione restituire?”

Per ogni posizione, si calcola la similarità tra la sua query e tutte le chiavi, e si usa il risultato per pesare i valori corrispondenti (si calcola attenzione sempre sullo stesso input, per questo viene detto *self*). Per ogni parola in input si calcola la probabilità di quanto essa debba essere considerata nella generazione dell'output.

Scaled Dot-Product Attention La formulazione usata nei Transformer è:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

dove d_k è la dimensione delle chiavi. La divisione per $\sqrt{d_k}$ evita che i dot-product diventino troppo grandi (causando gradienti piccoli dopo la softmax).

13.6 Transformer

Il **Transformer** (Vaswani et al., 2017) è un'architettura seq2seq che elimina completamente le RNN, basandosi solo su meccanismi di attention.

Caratteristiche principali:

- **Self-attention:** permette di catturare dipendenze a qualsiasi distanza in tempo costante (vs. lineare per le RNN)
- **Multi-Head Attention:** più “teste” di attention in parallelo, ognuna che cattura relazioni diverse
- **Positional Encoding:** poiché non c'è ricorrenza, l'informazione posizionale viene aggiunta esplicitamente agli embedding
- **Parallelizzazione:** a differenza delle RNN, il training può essere completamente parallelizzato

13.7 Multi-Head Attention

La **multi-head attention** esegue la scaled dot-product attention più volte in parallelo (ogni esecuzione parallela è detta “testa” o *head*). L'idea è che ogni testa possa specializzarsi nel catturare tipi diversi di relazioni (ad esempio, una testa potrebbe focalizzarsi sulla sintassi, un'altra sulle relazioni semantiche a lungo raggio).

Formalmente:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

dove ogni testa è calcolata proiettando query, key e value in sottospazi diversi:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

I risultati delle singole teste vengono concatenati e proiettati linearmente tramite W^O per ottenere l'output finale.

Intuizione Con una singola testa di attention, la media pesata potrebbe “appiattire” informazioni diverse. Multi-head attention permette al modello di focalizzarsi congiuntamente su informazioni provenienti da diversi sottospazi rappresentazionali in posizioni differenti.

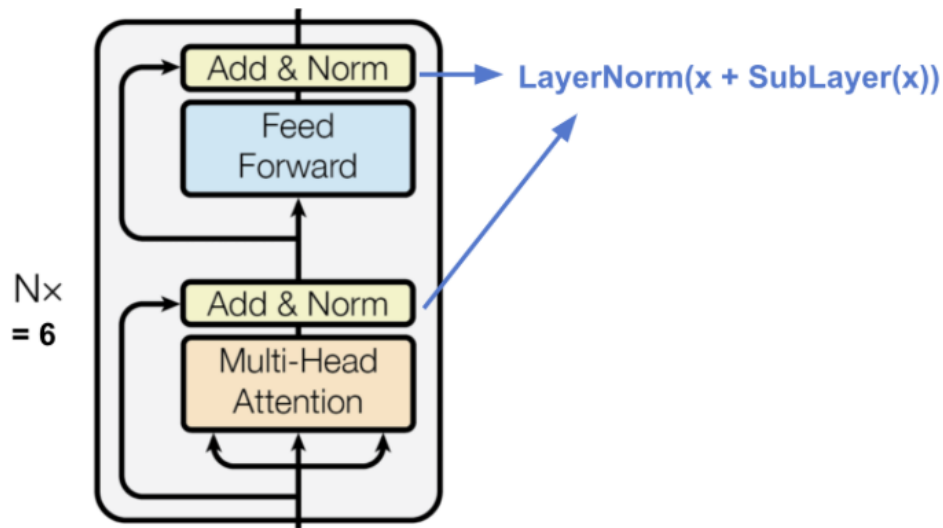
13.8 Transformer Architecture

L'architettura completa del Transformer segue il paradigma encoder-decoder, ma sostituendo le ricorrenze con layer di self-attention.

Encoder Composto da uno stack di $N = 6$ layer identici. Ogni layer ha due sotto-strati:

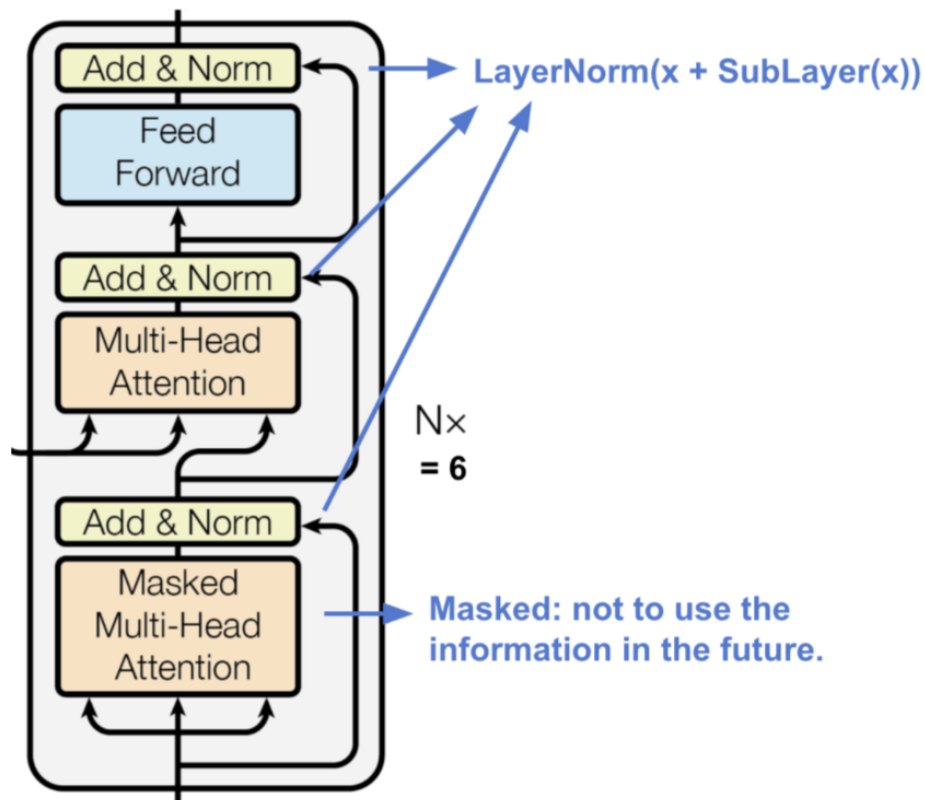
1. **Multi-Head Self-Attention:** permette a ogni posizione di attendere a tutte le altre posizioni dell'input.
2. **Position-wise Feed-Forward Network:** una rete fully connected applicata indipendentemente a ogni posizione.

Ogni sotto-strato usa una connessione residua seguita da **Layer Normalization**: $\text{LayerNorm}(x + \text{Sublayer}(x))$. L'output di ogni sotto-strato ha dimensione $d_{\text{model}} = 512$.



Decoder Anche il decoder è uno stack di $N = 6$ layer identici. Oltre ai due sotto-strati dell'encoder, ne inserisce un terzo:

1. **Masked Multi-Head Self-Attention:** impedisce alle posizioni di attendere a posizioni future (causalità), mascherando gli elementi successivi a t .
2. **Multi-Head Cross-Attention:** esegue l'attention sull'output dell'encoder (qui le Query vengono dal decoder, mentre Key e Value dall'encoder).
3. **Position-wise Feed-Forward Network.**



Full Architecture

- Input e output vengono convertiti in embedding e sommati al **Positional Encoding** (per preservare l'ordine).

- L'output dell'ultimo encoder viene passato a **tutti** i decoder stack (tramite la cross-attention).
- L'output finale passa per un layer lineare e una softmax per generare le probabilità dei token successivi.

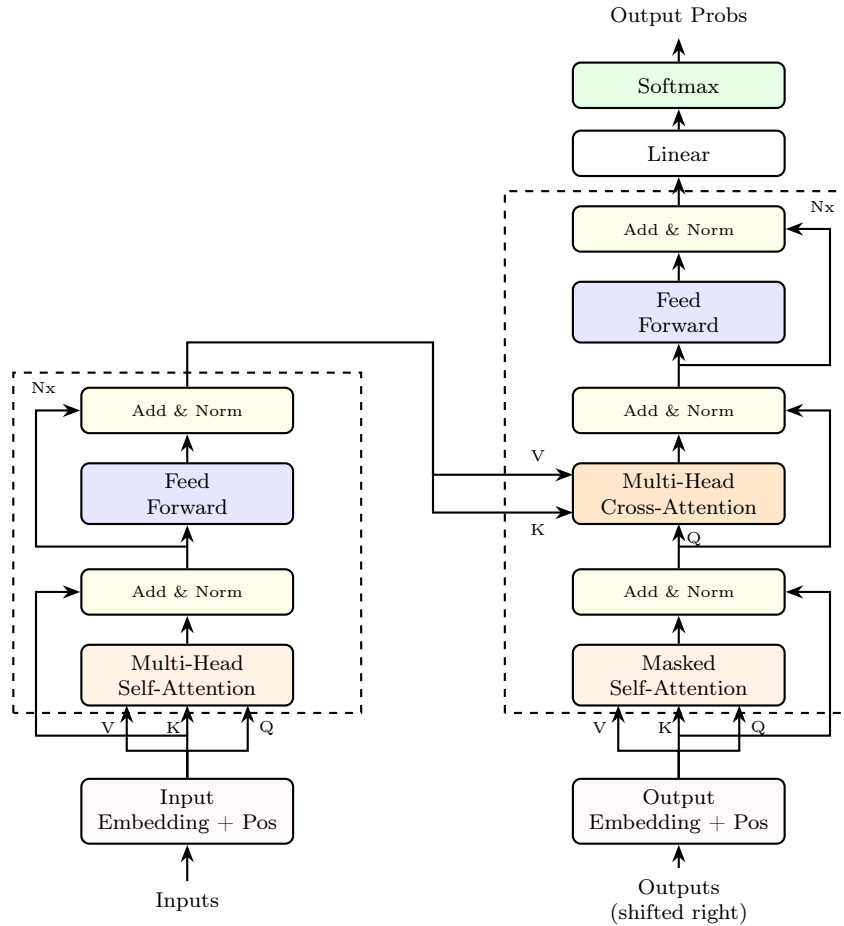


Figura 22: Architettura del Transformer. A sinistra l'Encoder, a destra il Decoder. Si notino le connessioni residue (freccie laterali) e la cross-attention che collega i due blocchi.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Figura 23: Confronto complessità computazionale tra Transformer e RNN.

13.9 BERT

BERT (Bidirectional Encoder Representations from Transformers) è un modello basato sull'Encoder del Transformer, progettato per apprendere rappresentazioni profonde e bidirezionali pre-addestrandolo su enormi corpus di testo non etichettato.

A differenza dei modelli precedenti (come GPT che è unidirezionale left-to-right, o ELMo che concatena indipendentemente left-to-right e right-to-left), BERT è **profondamente bidirezionale**.

Masked Language Model (MLM) Nei modelli bidirezionali standard, se si provasse a predire la parola successiva, i layer superiori potrebbero "vedere" la parola target stessa, rendendo la predizione banale. Per risolvere questo problema, BERT utilizza il **Masking**:

- Il 15% dei token in input viene scelto a caso.
- Questi token vengono sostituiti con un token speciale [MASK] (80% delle volte), con un token casuale (10%) o lasciati invariati (10%).
- L'obiettivo del modello è predire i token originali basandosi solo sul contesto (le altre parole della frase).

Next Sentence Prediction (NSP) Oltre al MLM, BERT viene addestrato per capire le relazioni tra frasi (utile per task come Question Answering).

- Il modello riceve in input coppie di frasi (A, B) .
- Il 50% delle volte B è la frase che segue effettivamente A .
- Il 50% delle volte B è una frase casuale dal corpus.
- L'obiettivo è predire se B è la continuazione di A .

Grazie a questo pre-training, BERT può essere poi **fine-tunato** su specifici task con l'aggiunta di un semplice layer di output, ottenendo risultati state-of-the-art.

In questo modo BERT può apprendere rappresentazioni profonde e bidirezionali di ogni token in base al contesto, senza dover essere addestrato specificamente per ogni task.

Fine-Tuning Il modello pre-addestrato viene adattato a task supervisionati specifici aggiungendo un layer di output all'encoder e aggiornando tutti i parametri. Ecco le principali applicazioni:

- **Sequence Classification** (es. Sentiment Analysis): L'obiettivo è predire una classe per l'intera sequenza. Poiché l'output di BERT è una sequenza di vettori, si utilizza solo l'embedding corrispondente al token speciale [CLS] (inserito sempre all'inizio). Questo vettore viene passato a un layer fully-connected + softmax per ottenere la classificazione.
- **Named Entity Recognition (NER)**: L'obiettivo è assegnare un tag (es. Persona, Luogo) a ogni parola. In questo caso, l'hidden state di **ogni token** viene passato a un classificatore (lo stesso per tutti i token) che predice la classe di appartenenza usando una softmax/argmax.
- **Natural Language Inference (NLI)**: Date due frasi (es. Testo e Ipotesi), bisogna determinare la loro relazione logica (es. implicazione, contraddizione). Le due frasi vengono separate dal token speciale [SEP] e distinte tramite *segment embeddings*. Come per la classificazione, si usa l'output del token [CLS] per predire la relazione.
- **Grounded Common Sense Inference**: Data una frase, bisogna scegliere la continuazione più plausibile tra 4 opzioni. Si costruiscono 4 sequenze di input concatenando la frase originale con ciascuna delle possibili continuazioni. Ogni sequenza viene passata a BERT e, tramite il token [CLS], si calcola uno score. Una softmax normalizza questi score per identificare la scelta migliore.

14 CNN - Convolutional Neural Networks

14.1 Introduzione e Motivazioni

Le reti neurali fully-connected (MLP) sono approssimatori universali, ma presentano limitazioni significative quando applicate a dati ad alta dimensionalità come le immagini: il numero di parametri esplode, rendendo difficile l'ottimizzazione e la generalizzazione. Le **Convolutional Neural Networks (CNN)** sono progettate per sfruttare le proprietà strutturali intrinseche delle immagini:

- **Self-similarity:** I dati tendono ad essere auto-simili attraverso il dominio (pattern ripetuti).
- **Translation Invariance:** Il riconoscimento di un oggetto non dovrebbe dipendere dalla sua posizione nell'immagine.
- **Deformation Invariance:** Capacità di gestire deformazioni e variazioni locali.
- **Hierarchy and Compositionality:** Le immagini sono composte gerarchicamente (pixel $\rightarrow$ bordi $\rightarrow$ forme $\rightarrow$ oggetti).

Le CNN introducono quindi uno specifico **inductive bias**, assumendo che i dati abbiano queste caratteristiche di località e invarianza.

14.2 Operazione di Convoluzione

L'operazione fondamentale alla base delle CNN è la convoluzione. Possiamo definirla in diversi modi a seconda del dominio:

Definizione 14.1 (Convoluzione). Date due funzioni f e g definite nell'intervallo $[-\pi, \pi]$, la loro convoluzione $(f \star g)$ è definita come l'integrale:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$

In questo contesto, una delle due funzioni agisce come input (il segnale) e l'altra come filtro (o kernel). Il risultato è una nuova funzione, detta **Feature Map**, che rappresenta la risposta del segnale al filtro.

Convoluzione Discreta (1D) Nel dominio discreto, come nel caso di sequenze o vettori (es. testo, audio), dati due vettori $\mathbf{x}$ e $\mathbf{w}$, la convoluzione è definita dalla sommatoria:

$$(\mathbf{x} * \mathbf{w})_i = \sum_{k=0}^{n-1} w_{i-k \bmod n} x_k$$

dove $\mathbf{w}$ rappresenta i pesi del kernel e $\mathbf{x}$ l'input.

Convoluzione Discreta in Domini 2D Per le immagini, lavoriamo su domini 2D (o 3D considerando i canali colore). Se consideriamo un'immagine $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ (es. RGB), per ogni canale la convoluzione discreta è:

$$(f \star g)[m, n] = \sum_k \sum_{\ell} f[k, \ell]g[m-k, n-\ell]$$

Questa operazione può essere interpretata intuitivamente come una "finestra mobile" (moving window) del kernel f che scorre sopra l'immagine g . Ad ogni posizione, viene calcolato il prodotto elemento per elemento e la somma risultante costituisce un pixel della Feature Map di output.

Estensione a 3D e oltre Il concetto si generalizza facilmente a domini euclidei di dimensione superiore $f: \mathbb{R}^k \rightarrow \mathbb{R}^m$, dove la convoluzione è ben definita a meno delle condizioni al contorno.

14.3 Iperparametri della Convoluzione

Oltre ai pesi del kernel, diversi iperparametri regolano l'applicazione della convoluzione e determinano la dimensione dell'output:

- **Kernel Size (K):** La dimensione spaziale o temporale della finestra. Un kernel più grande cattura dipendenze su un raggio più ampio (campo recettivo maggiore).
- **Stride (S):** Il passo con cui il kernel scorre sull'input.
 - $S = 1$: Spostamento unitario (massima sovrapposizione).
 - $S > 1$: Il kernel "salta" degli elementi, riducendo la dimensione dell'output (effetto *downsampling*).
- **Padding (P):** La gestione dei bordi dell'input.
 - *Valid* (No Padding): Il kernel si ferma quando colpisce il bordo. L'output è più piccolo dell'input ($N - K + 1$).
 - *Same* (Zero Padding): Vengono aggiunti zeri ai bordi per mantenere la dimensione dell'output uguale a quella dell'input (assumendo $S = 1$).
 - *Full Padding*: Espande ulteriormente l'input aggiungendo zeri.

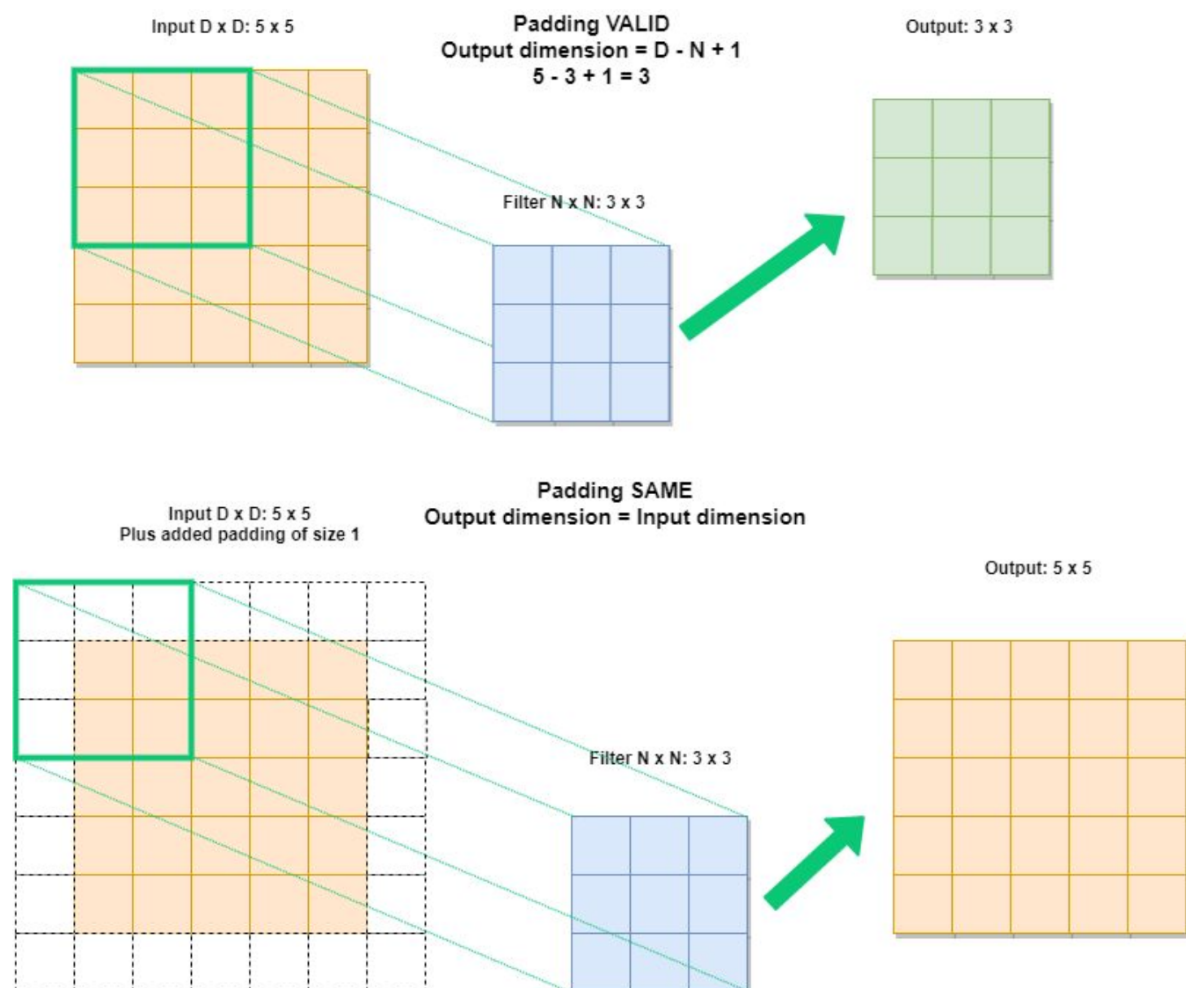


Figura 24: Valid vs Same padding.

- **Dilation:** Introduce dei "buchi" nel kernel per espandere il campo recettivo senza aumentare il numero di parametri (convoluzione *atrous*).

14.4 1D ConvNets

Le reti convoluzionali 1D (ConvNets) rappresentano un adattamento dell'operazione di convoluzione per dati sequenziali (come serie temporali, audio o testo), dove il kernel scorre lungo un'unica dimensione spaziale.

Il Layer Convoluzionale A differenza di un layer *Fully Connected* (o denso), dove ogni input è connesso a ogni output con un peso distinto, un layer convoluzionale sfrutta la struttura locale dei dati. L'operazione base è la **convoluzione discreta**. Dato un input vettoriale $\mathbf{x}$ e un filtro (o *kernel*) $\mathbf{w}$ di dimensione K , l'output z (detto *feature map*) è calcolato come una somma pesata scorrevole ("sliding window"):

$$z_i = \sum_{j=1}^K w_j \cdot x_{i+j-1}$$

Ad ogni unità di output viene poi aggiunto un bias β e applicata una funzione di attivazione non lineare $a(\cdot)$ (es. ReLU):

$$h_i = a \left[\beta + \sum_{j=1}^K w_j \cdot x_{i+j-1} \right]$$

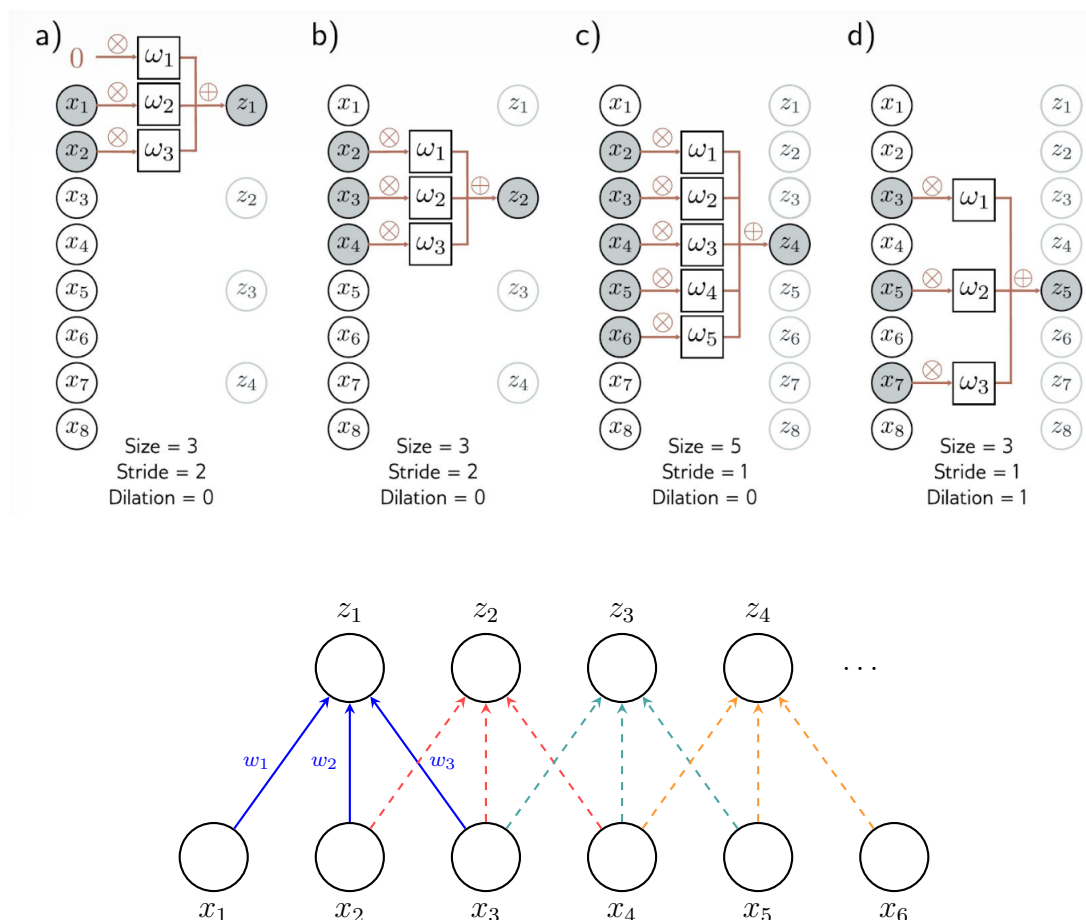


Figura 25: Rappresentazione di una Convoluzione 1D con Kernel Size $K = 3$ e Stride $S = 1$. Si noti come gli stessi pesi vengano riutilizzati ("Parameter Sharing").

Canali Nelle reti moderne, sia l'input che l'output hanno una dimensione di "profondità" (i **canali**). Se l'input ha C_{in} canali (es. le 3 componenti RGB in un'immagine, o la dimensione dell'embedding nel testo), il kernel non è più un vettore ma una matrice di dimensione $C_{in} \times K$. L'output a sua volta avrà C_{out} canali (feature maps), ottenuti applicando C_{out} filtri diversi in parallelo. Il tensore dei pesi avrà quindi dimensione:

$$\mathbf{W} \in \mathbb{R}^{C_{out} \times C_{in} \times K}$$

Receptive Field e Gerarchia Il **Receptive Field** è la porzione dell'input originale che influenza il valore di uno specifico neurone in un layer profondo.

- Nei primi layer, il receptive field è piccolo e cattura feature di **basso livello** (es. bordi, linee).
- Andando in profondità, componendo più layer convoluzionali e di pooling, il receptive field cresce, permettendo alla rete di "vedere" porzioni più ampie dell'input e di estrarre feature di **alto livello** e semantiche (es. forme complesse, oggetti).

Struttura di una ConvNet Una tipica architettura CNN alterna blocchi composti da:

1. **Convoluzione:** Estrazione delle feature.
2. **Non-linearità (ReLU):** Per apprendere funzioni complesse.
3. **Pooling:** Operazione di sottocampionamento (es. *Max Pooling*) che riduce la dimensione spaziale, rendendo la rappresentazione più astratta e robusta a piccole variazioni.

Infine, le feature map vengono appiattite (*Flattening*) e passate a un MLP classico per la classificazione finale.

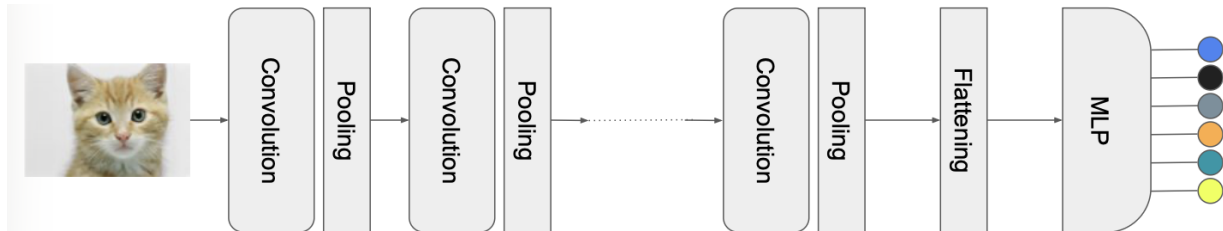


Figura 26: Struttura di una ConvNet.

14.5 Input 2D e Layer Conv2D

L'applicazione più comune delle CNN riguarda dati bidimensionali, come le immagini. In questo contesto, l'input è un tensore con dimensioni spaziali (Altezza H e Larghezza W) e una profondità data dai canali.

Convoluzione 2D Il kernel diventa una matrice bidimensionale $K \times K$ che scorre sull'input sia in orizzontale che in verticale. Dato un input X e un kernel $\mathbf{W}$, l'unità di output $h_{i,j}$ (trascurando per un attimo i canali multipli) è calcolata come:

$$h_{i,j} = a \left[\beta + \sum_{m=1}^K \sum_{n=1}^K w_{m,n} \cdot x_{i+m-1,j+n-1} \right]$$

Questa operazione corrisponde a una somma pesata su una regione locale quadrata dell'input.

Gestione dei Canali (Input Volumetrici) Nelle immagini a colori, l'input non è una semplice matrice ma un tensore (o volume) di dimensione $H \times W \times C_{in}$ (dove $C_{in} = 3$ per RGB). La convoluzione gestisce la profondità in questo modo:

1. **Il filtro è un volume:** Ogni singolo kernel $\mathbf{w}$ deve avere la stessa profondità dell'input. Se l'input ha 3 canali, il kernel avrà dimensione $K \times K \times 3$.
2. **Collasso dei canali (Single Filter Output):** Quando il kernel scorre sull'immagine, in ogni posizione esegue una somma pesata che coinvolge *tutti* i canali di input contemporaneamente. Matematicamente, questo "schiaffia" la dimensione della profondità: l'applicazione di un **singolo filtro** produce quindi una Feature Map piatta (una matrice 2D), non un volume.
3. **Generazione del volume di output:** Per ottenere un output che sia a sua volta un tensore (e preservare l'informazione complessa), utilizziamo un banco di C_{out} filtri diversi in parallelo.
4. **Stacking:** Ognuno dei C_{out} filtri produce la propria Feature Map 2D. Queste mappe vengono infine impilate ("stacked") lungo la dimensione dei canali per formare il tensore di output finale di dimensione $H_{out} \times W_{out} \times C_{out}$.

Per generare C_{out} canali in output, il tensore dei pesi totale avrà dimensione:

$$\mathbf{W} \in \mathbb{R}^{C_{out} \times C_{in} \times K \times K}$$

Calcolo delle Dimensioni La dimensione spaziale dell'output ($H_{out} \times W_{out}$) dipende da Padding (P), Dilation (D), Kernel Size (K) e Stride (S). La formula generale (per l'altezza) è:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - D(K - 1) - 1}{S} + 1 \right\rfloor$$

Pooling (Downsampling) Per ridurre progressivamente le dimensioni spaziali della rappresentazione e guadagnare invarianza alle piccole traslazioni, si utilizzano layer di **Pooling**. L'operazione avviene indipendentemente su ogni canale. Le varianti principali sono:

- **Max Pooling:** Seleziona il valore massimo all'interno della finestra locale. È il metodo più diffuso perché preserva le feature più salienti.
- **Average Pooling:** Calcola la media dei valori nella finestra.

Possiamo fare downsampling anche usando una convoluzione con stride maggiore di 1.

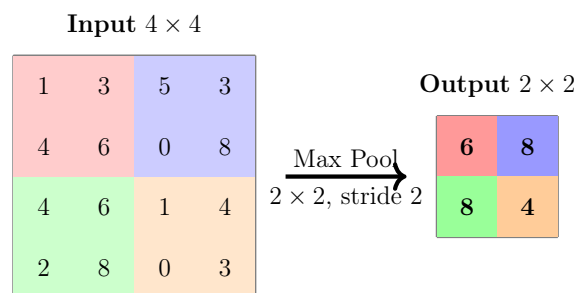


Figura 27: Esempio di Max Pooling con kernel 2×2 e stride 2. La dimensione spaziale viene dimezzata.

Upsampling In alcune architetture è necessario aumentare la risoluzione spaziale. Tecniche comuni includono:

- **Bilinear Interpolation:** Metodo classico di interpolazione.
- **Max Unpooling:** Utilizza gli indici salvati durante il Max Pooling per riposizionare i valori nelle posizioni originali, riempiendo il resto con zeri.

Convoluzione 1×1 Un caso particolare e molto potente è l'uso di un kernel di dimensione 1×1 . Poiché non considera i vicini spaziali, questo layer agisce solo lungo la profondità. Viene utilizzato principalmente per:

1. Modificare il numero di canali (aumentarli o diminuirli) senza alterare le dimensioni spaziali H e W .
2. Aggiungere non-linearità e combinare le informazioni tra i vari canali in modo economico.

14.6 Applicazioni delle ConvNets

Le CNN sono state primariamente usate per fare **image classification** (e.g., ImageNet - più di un milione di immagini 224×224 RGB con 1000 categorie).

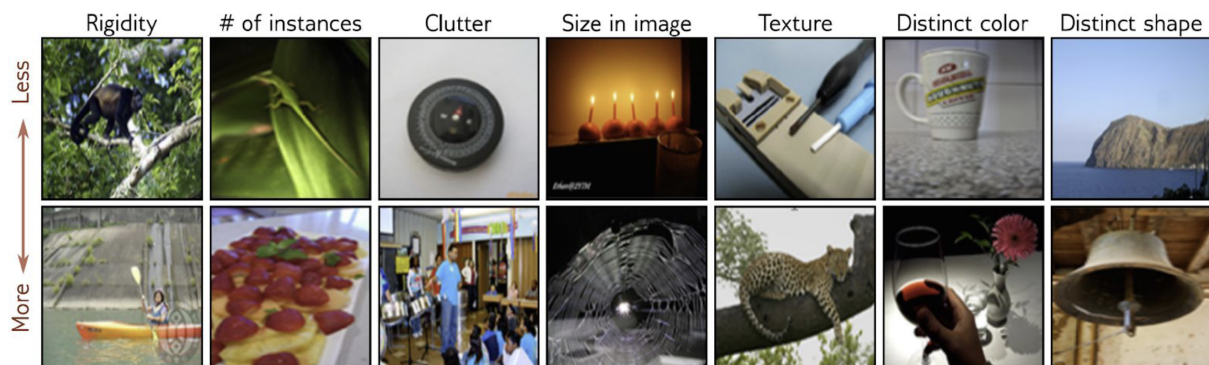


Figura 28: Dataset ImageNet.

AlexNet AlexNet è la rete neurale convoluzionale che ha segnato la rinascita del Deep Learning, vincendo con ampio margine la competizione ILSVRC (ImageNet) del 2012. Questa architettura ha dimostrato per la prima volta l'efficacia delle CNN su dataset di grandi dimensioni.

Il modello è composto da 8 layer totali: 5 strati convoluzionali seguiti da 3 strati fully-connected, per un totale di circa 60 milioni di parametri e 650.000 neuroni. Tra le principali innovazioni introdotte o rese popolari da AlexNet troviamo:

- **ReLU (Rectified Linear Units)**: l'uso della funzione di attivazione $f(x) = \max(0, x)$ al posto di tanh o sigmoide ha permesso di velocizzare drasticamente la convergenza dell'addestramento.
- **Dropout**: introdotto nei layer fully-connected per prevenire l'overfitting, spegnendo casualmente i neuroni durante il training (con probabilità 0.5).
- **Data Augmentation**: tecniche come traslazioni, riflessioni e crop casuali per aumentare artificialmente la dimensione del dataset.
- **GPU Training**: l'addestramento è stato parallelizzato su due GPU per gestire l'elevato carico computazionale.
- **Local Response Normalization (LRN)**: una forma di normalizzazione laterale (oggi meno usata, sostituita spesso dalla Batch Normalization) per favorire la competizione tra neuroni adiacenti.

L'input della rete prevede immagini di dimensione 224×224 e l'output finale è una distribuzione di probabilità su 1000 classi.

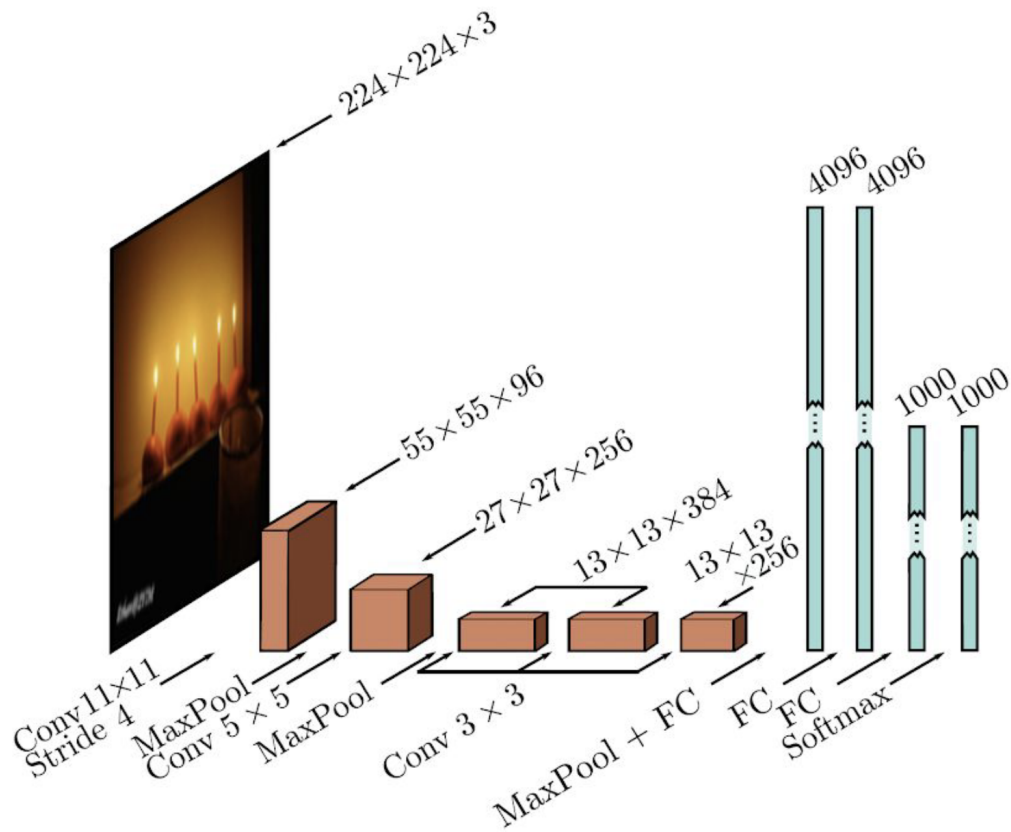


Figura 29: Architettura AlexNet.

VGGNet La VGGNet (Visual Geometry Group) è un'architettura che ha standardizzato il design delle CNN profonde. A differenza di AlexNet, che usava filtri di dimensioni variabili (e.g., 11×11 , 5×5), VGGNet introduce un principio di design molto semplice e uniforme: usare **solo filtri convoluzionali molto piccoli** di dimensione 3×3 , ma aumentando notevolmente la profondità della rete.

Le configurazioni più famose sono la **VGG-16** (13 layer convoluzionali + 3 fully-connected) e la **VGG-19** (16 conv + 3 FC). Le caratteristiche principali sono:

- **Small Receptive Fields:** L'intuizione chiave è che una pila di due layer 3×3 ha lo stesso campo recettivo effettivo di un singolo layer 5×5 , ma con meno parametri ($2 \cdot 3^2 = 18$ vs $1 \cdot 5^2 = 25$) e più non-linearità intermedie, rendendo la funzione di decisione più discriminativa.
- **Uniformità:** Tutti i layer convoluzionali hanno kernel 3×3 , stride 1 e padding 1 (per mantenere la dimensione spaziale). Il downsampling è delegato esclusivamente ai layer di Max Pooling (2×2 , stride 2).
- **Elevato numero di parametri:** A causa dei tre layer fully-connected finali (il primo dei quali ha dimensione 4096), la VGG-16 ha circa **138 milioni di parametri**, rendendola computazionalmente più pesante e costosa in termini di memoria.

Nonostante il costo computazionale, VGG ha dimostrato che la *profondità* è una componente critica per ottenere alte performance.

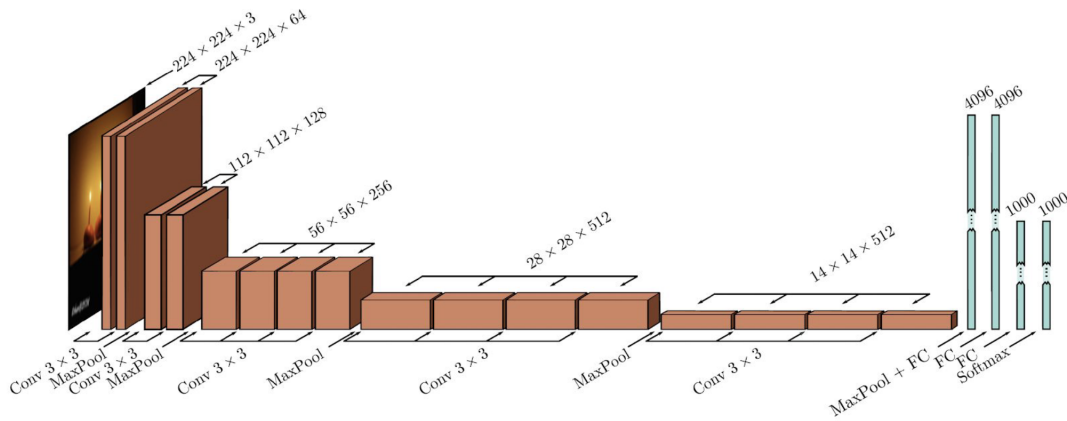


Figura 30: Schema a blocchi dell'architettura VGGNet.

Object Detection: YOLO Mentre la classificazione assegna un'etichetta all'immagine intera, l'**Object Detection** mira a identificare e localizzare (con bounding box) più oggetti all'interno della stessa immagine.

Uno dei primi metodi basati su CNN a rivoluzionare questo campo è stato **YOLO** (You Only Look Once). Il nome deriva dal fatto che, a differenza di approcci precedenti che scansionavano l'immagine più volte o in più parti, YOLO elabora l'intera immagine in un unico passaggio della rete neurale (1 solo frame).

L'architettura, ispirata a VGGNet, è composta da:

- **Input:** Immagine RGB 448×448 .
- **Backbone (Feature Extraction):** 24 layer convoluzionali che estraggono le feature. La rete riduce progressivamente la dimensione spaziale usando Max Pooling, mentre aumenta la profondità (numero di canali).
- **Head (Prediction):** L'ultimo layer convoluzionale (7×7 con 1024 canali) viene appiattito e passato a due layer Fully Connected. Il primo ha 4096 neuroni, il secondo produce l'output finale (coordinate e classi).

Semantic Segmentation L'obiettivo della **Semantic Segmentation** è assegnare un'etichetta semantica a *ogni singolo pixel* dell'immagine, in base all'oggetto a cui appartiene. Se un pixel non corrisponde ad alcuna classe nota nel dataset di training, gli viene assegnata un'etichetta "background" o nessuna etichetta.

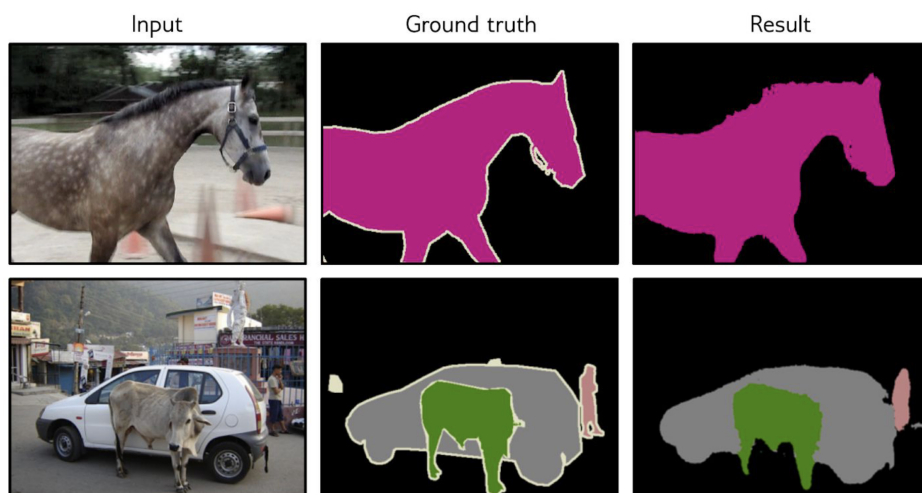


Figura 31: Esempio di Semantic Segmentation: ogni pixel viene classificato (es. cavallo, auto).

A differenza dell'Object Detection, che approssima la posizione degli oggetti con dei rettangoli (bounding box), la segmentazione semantica fornisce i contorni precisi degli oggetti, offrendo una comprensione molto più dettagliata della scena. Tuttavia, non distingue tra istanze diverse della stessa classe.

Queste reti convoluzionali processano l'immagine in maniera sequenziale e possono dunque essere pensate come funzioni di funzioni.

$$\mathbf{y} = f_4 [f_3 [f_2 [f_1(\mathbf{x}, \theta_1), \theta_2], \theta_3], \theta_4]$$

Storicamente (fino al 2014) si è osservato che aumentando la profondità della rete gli errori tendono a diminuire (come visto nel passaggio da AlexNet a VGG e GoogleNet). Tuttavia, esperimenti successivi hanno evidenziato un problema inaspettato: continuando ad aggiungere layer a una rete convoluzionale standard ("plain"), le prestazioni iniziano a degradare.

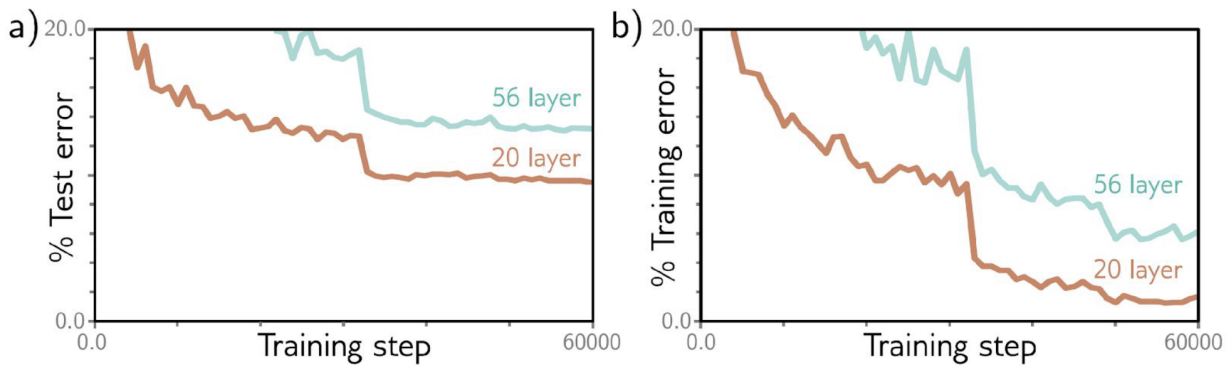


Figura 32: Degradation problem: una rete più profonda (56 layer) performa peggio di una meno profonda (20 layer) sia sul test set che sul training set.

Come si nota dai grafici, una rete a 56 layer ha un errore più alto (sia di training che di test) rispetto a una rete a 20 layer. Questo fenomeno, noto come **degradation problem**, dimostra che non basta impilare più layer per ottenere modelli migliori: l'aumento di profondità rende l'ottimizzazione molto più difficile, e il modello non riesce nemmeno ad apprendere bene quanto la sua controparte meno profonda (non è overfitting, dato che l'errore di training aumenta).

Residual Connection La soluzione al degradation problem è la **Residual Connection** (o *skip connection*). L'idea è semplice ma potente: invece di far apprendere direttamente a un blocco la trasformazione desiderata $\mathcal{H}(\mathbf{x})$, gli si fa apprendere solo il *residuo* $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$.

L'output del blocco diventa quindi:

$$\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x}, \phi)$$

dove $\mathbf{x}$ è l'input che "salta" il blocco tramite la skip connection e viene sommato all'output della trasformazione $\mathcal{F}$.

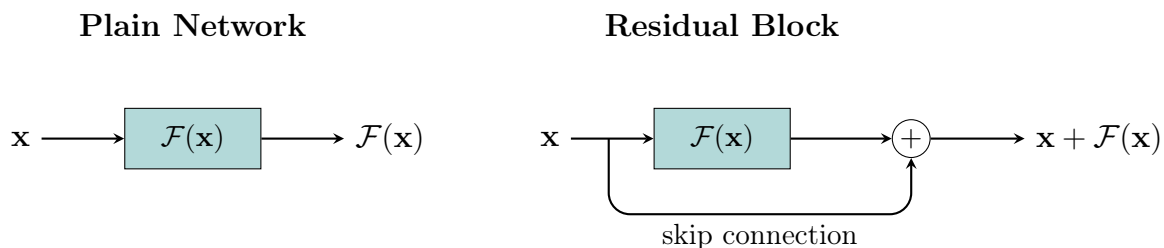


Figura 33: Confronto tra blocco plain e blocco residuale. La skip connection permette all'input di bypassare la trasformazione.

Se i layer aggiuntivi non sono utili, è più facile per la rete apprendere $\mathcal{F}(\mathbf{x}) \approx 0$ (identity mapping) piuttosto che apprendere direttamente $\mathcal{H}(\mathbf{x}) = \mathbf{x}$. Inoltre, durante il backpropagation il gradiente può fluire direttamente attraverso la skip connection, mitigando il problema del *vanishing gradient*. Di fatto stiamo anche implementando un ensemble, per cui la varianza è ridotta.

Un altro vantaggio fondamentale delle residual connections è che rendono la **loss surface** molto più *smooth* (liscia). Visualizzando la superficie di loss nello spazio dei parametri, si osserva che:

- Con skip connections:** la superficie è regolare, con un minimo ben definito e senza irregolarità significative. Questo facilita l'ottimizzazione via SGD.
- Senza skip connections:** la superficie presenta molti picchi, valli strette e minimi locali, rendendo l'ottimizzazione molto più difficile e instabile.

Una loss surface liscia rende il training più robusto a piccole perturbazioni nei parametri, migliorando la capacità di generalizzazione del modello.

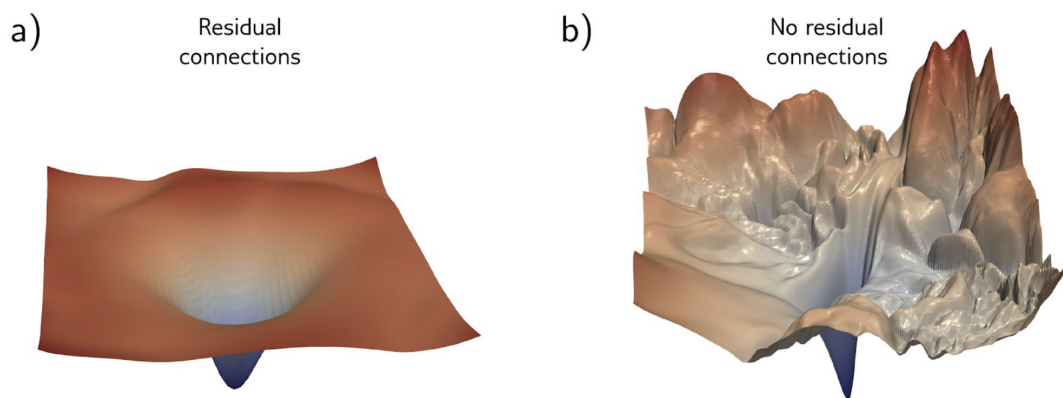


Figura 34: Confronto tra loss surface con e senza skip connections.

Formulazione ricorsiva Considerando una sequenza di blocchi residuali, le attivazioni si propagano come:

$$\begin{aligned} \mathbf{h}_1 &= \mathbf{x} + f_1[\mathbf{x}, \phi_1] \\ \mathbf{h}_2 &= \mathbf{h}_1 + f_2[\mathbf{h}_1, \phi_2] \\ \mathbf{h}_3 &= \mathbf{h}_2 + f_3[\mathbf{h}_2, \phi_3] \\ \mathbf{y} &= \mathbf{h}_3 + f_4[\mathbf{h}_3, \phi_4] \end{aligned}$$

Espandendo completamente l'output:

$$\mathbf{y} = \mathbf{x} + f_1[\mathbf{x}] + f_2[\mathbf{x} + f_1[\mathbf{x}]] + f_3[\dots] + f_4[\dots]$$

Questa struttura mostra come l'output finale contenga sempre l'input originale $\mathbf{x}$, più una serie di correzioni residuali.

Struttura di un blocco residuale tipico Un blocco residuale standard è composto da più layer. Esistono diverse varianti:

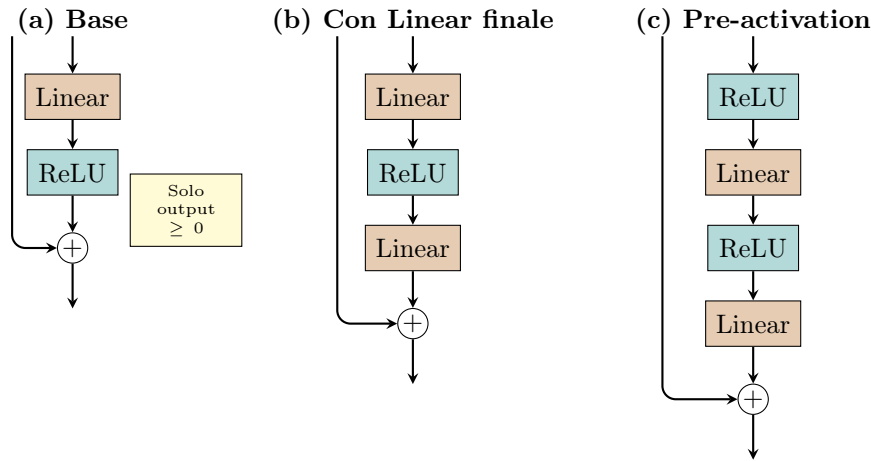


Figura 35: Varianti di blocchi residuali: (a) base con problema di output solo positivi, (b) con layer lineare finale per gestire valori negativi, (c) pre-activation con ReLU prima dei layer lineari.

ResNet ResNet (Residual Network) è l'architettura che ha introdotto le residual connections su larga scala, vincendo la competizione ILSVRC 2015. La sua innovazione principale è stata dimostrare che, grazie alle skip connections, è possibile allenare reti estremamente profonde (fino a 152 layer) senza incorrere nel degradation problem.

L'architettura segue una struttura modulare:

- **Stem:** Una convoluzione iniziale 7×7 con stride 2, seguita da Max Pooling. L'input $224 \times 224 \times 3$ viene ridotto a $56 \times 56 \times 64$.
- **Stage 1–4:** Quattro gruppi di blocchi residuali con canali crescenti (64, 128, 256, 512). Ogni stage raddoppia i canali e dimezza la risoluzione spaziale tramite subsampling (stride 2).
- **Head:** Global Average Pooling seguito da un singolo layer Fully Connected con Softmax per la classificazione su 1000 classi.

Bottleneck Block Nelle varianti più profonde (ResNet-50, 101, 152), viene utilizzato il **bottleneck block** per ridurre il costo computazionale. Invece di due convoluzioni 3×3 , si usa una sequenza:

$$\text{Conv } 1 \times 1 \rightarrow \text{Conv } 3 \times 3 \rightarrow \text{Conv } 1 \times 1$$

La prima 1×1 riduce i canali (bottleneck), la 3×3 opera su meno canali, e l'ultima 1×1 li espande nuovamente. Ogni convoluzione è preceduta da Batch Normalization e ReLU (schema pre-activation).

Varianti Le configurazioni standard sono:

- **ResNet-18/34:** Blocchi base (due conv 3×3), circa 11M/21M parametri.
- **ResNet-50:** Bottleneck blocks, circa 25M parametri.
- **ResNet-101/152:** Bottleneck blocks più profondi, circa 44M/60M parametri.

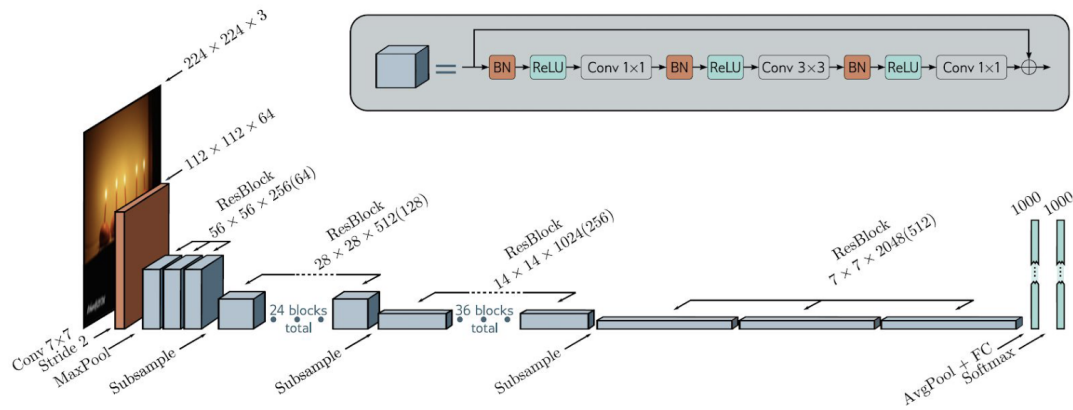


Figura 36: Schema a blocchi dell'architettura ResNet.

14.7 Architetture per Semantic Segmentation

Per la semantic segmentation l'output non è un singolo vettore di probabilità, ma una **mappa densa** della stessa dimensione spaziale dell'input, dove ogni pixel viene classificato. Questo richiede architetture specializzate di tipo **encoder-decoder**.

Loss Function: Pixel-wise Cross-Entropy Nella semantic segmentation, ogni pixel p dell'immagine viene trattato come un problema di classificazione indipendente su C classi. La loss totale è la media delle cross-entropy su tutti i pixel:

$$\mathcal{L} = -\frac{1}{N} \sum_{p=1}^N \sum_{c=1}^C y_{p,c} \log(\hat{y}_{p,c})$$

dove N è il numero totale di pixel, $y_{p,c}$ è la ground truth one-hot per il pixel p e classe c , e $\hat{y}_{p,c}$ è la probabilità predetta (output del softmax). In pratica, per ogni pixel si calcola la cross-entropy categorica e si fa la media su tutta l'immagine.

Hourglass Network L'Hourglass Network (o rete a "clessidra") è un'architettura encoder-decoder simmetrica per la semantic segmentation. Il nome deriva dalla sua forma: l'encoder comprime progressivamente la rappresentazione spaziale, mentre il decoder la espande nuovamente.

- **Encoder:** Struttura simile a VGG con convoluzioni 3×3 e Max Pooling. L'immagine 224×224 viene compressa fino a $7 \times 7 \times 512$.
- **Bottleneck:** Due layer Fully Connected (4096 neuroni ciascuno) che catturano informazione globale sull'intera immagine.
- **Decoder:** Specchia l'encoder usando **Max Unpooling** (che ricorda le posizioni dei massimi) e **Transposed Convolutions** (deconvoluzioni) per ricostruire la risoluzione originale.
- **Output:** Mappa $224 \times 224 \times C$ dove C è il numero di classi (es. 21 per PASCAL VOC). Ogni pixel contiene le probabilità di appartenenza alle diverse classi.

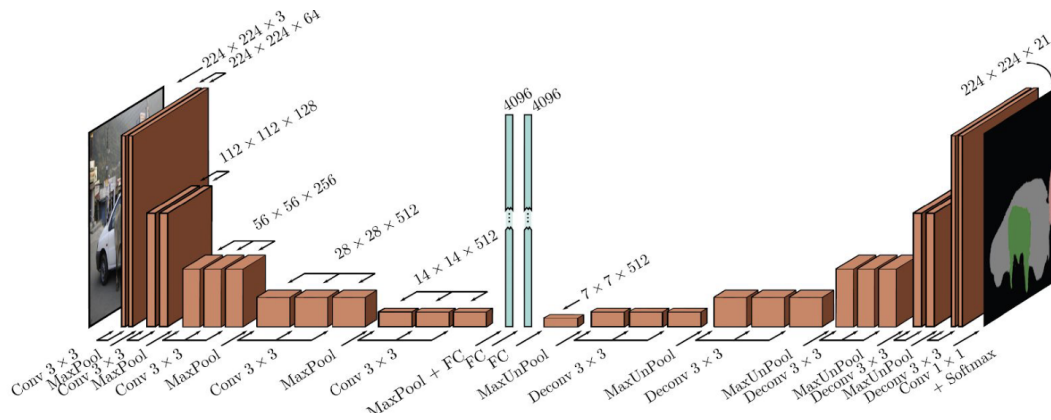


Figura 37: Schema a blocchi dell'architettura Hourglass Network.

U-Net U-Net è un'architettura encoder-decoder sviluppata specificamente per la segmentazione di immagini biomediche, dove spesso i dataset sono piccoli ma è richiesta alta precisione spaziale.

La caratteristica distintiva di U-Net sono le **skip connections laterali** che collegano direttamente ogni livello dell'encoder al corrispondente livello del decoder tramite concatenazione:

- **Encoder (Contracting Path):** Sequenza di blocchi [Conv $3 \times 3 \rightarrow \text{ReLU} \rightarrow \text{Conv } 3 \times 3 \rightarrow \text{ReLU} \rightarrow \text{Max Pool } 2 \times 2$]. Ad ogni step la risoluzione si dimezza e i canali raddoppiano ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$).
- **Decoder (Expanding Path):** Sequenza di [Transposed Conv $2 \times 2 \rightarrow \text{Concatenazione con feature map dell'encoder} \rightarrow \text{Conv } 3 \times 3 \rightarrow \text{ReLU} \rightarrow \text{Conv } 3 \times 3 \rightarrow \text{ReLU}$]. La risoluzione raddoppia ad ogni step.
- **Skip Connections:** Le feature map dell'encoder vengono ritagliate (*crop*) e concatenate a quelle del decoder alla stessa scala. Questo permette di combinare informazione semantica di alto livello (dal decoder) con dettagli spaziali precisi (dall'encoder).
- **Output:** Convoluzione 1×1 finale per mappare i canali al numero di classi.

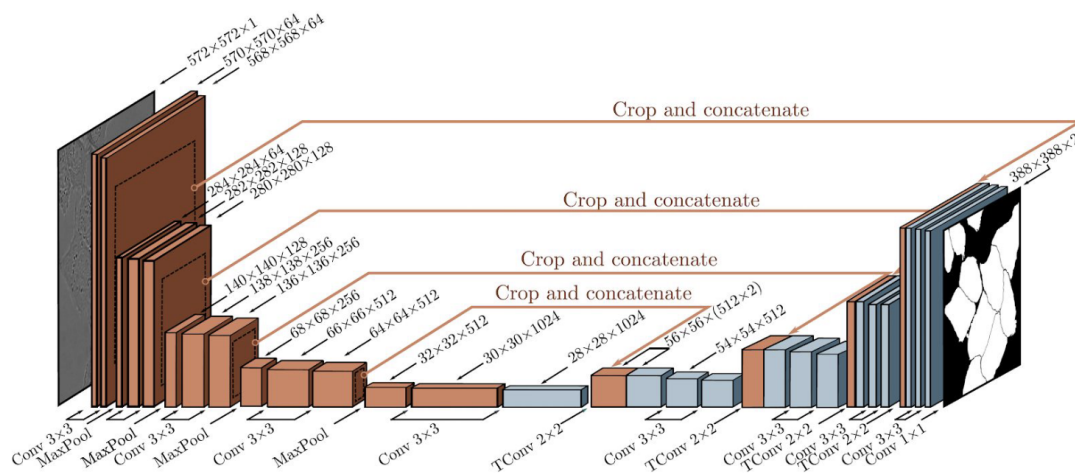


Figura 38: Schema a blocchi dell'architettura U-Net.

La differenza principale rispetto all'Hourglass è che U-Net non usa layer fully connected (è *fully convolutional*) e soprattutto possiede **skip connections laterali** che collegano direttamente encoder e decoder. Hourglass, invece, è un encoder-decoder *puro*: tutta l'informazione deve passare attraverso il bottleneck, perdendo dettagli spaziali fini. In U-Net, le skip connections **concatenano** le feature map (non le sommano come in ResNet), preservando così i dettagli ad alta risoluzione necessari per una segmentazione precisa e permettendo di andare ancora più in profondità.